

Programación Web

Programación IV



Ing. Alejandro Di Battista

PROGRAMACIÓN IV

Programacion Web

De los datos a las soluciones

Ing. Alejandro Di Battista

Índice

Parte I. Entender el lenguaje y sus valores

1. Del problema al programa
2. Variables, alcance y expresiones
3. El tipo boolean
4. Los tipos number y bigint
5. El tipo string y Unicode
6. undefined, null y la ausencia
7. Conversión y coerción
8. El tipo symbol y los protocolos

Parte II. Modelar colecciones y entidades

9. Arrays y matrices
10. Objetos, propiedades y referencias
11. Map, Set y colecciones especializadas

Parte III. Organizar el comportamiento

12. Estructuras de control
13. Errores y excepciones
14. Funciones
15. Programación funcional y pipelines
16. Recursividad y árboles binarios

Parte IV. Aplicar e integrar

17. Fundamentos de computación digital
18. Expresiones regulares
19. Gestión de archivos con Node.js

Apéndices

- A. Byte Pair Encoding: construir un tokenizador
- B. Evaluar expresiones mediante pilas

BLOQUE 1

Parte I. Entender el lenguaje y sus valores

8 capítulos en este bloque.

CAPÍTULO 1

Del problema al programa

Idea central

JavaScript es un lenguaje; para convertirlo en una solución necesita un motor que lo ejecute y un entorno que le permita interactuar con el mundo. Distinguir esas tres capas evita buena parte de la confusión inicial y permite decidir si un programa debe vivir en el navegador, en Node.js o en ambos.

JavaScript no es “el lenguaje de una etiqueta `<script>`” ni un sinónimo de navegador. Es una especificación con múltiples implementaciones, anfitriones y herramientas. Comprender ese ecosistema permite interpretar correctamente mensajes de error, documentación y decisiones de arquitectura.

Una tarea concreta antes que un lenguaje

Imaginemos una inscripción a una materia. Recibimos el nombre y la edad de una persona, validamos los datos y producimos una respuesta. El problema existe antes del código:

entrada → reglas → salida

JavaScript permite expresar esas reglas:

```
function evaluarInscripcion({ nombre, edad }) {  
  if (!nombre.trim()) return { aceptada: false, motivo: "Falta el nombre" };  
  if (edad < 16) return { aceptada: false, motivo: "Edad insuficiente" };  
  
  return { aceptada: true, mensaje: `Bienvenido, ${nombre}` };  
}
```

Este pequeño programa ya contiene las piezas que recorreremos en el libro: valores, una estructura de datos, una función, condiciones y un resultado explícito.

Por qué apareció JavaScript

La Web inicial publicaba documentos enlazados. Cada interacción importante requería pedir una página nueva al servidor. Los navegadores necesitaban un lenguaje pequeño que pudiera reaccionar cerca del usuario: validar un formulario, cambiar contenido y responder a eventos sin recargar todo.

JavaScript fue creado en 1995 dentro de Netscape. Su primer prototipo se desarrolló con enorme rapidez y el lenguaje cambió de nombre durante su lanzamiento. Su nombre final aprovechó la popularidad de Java, pero JavaScript y Java son lenguajes distintos:

- tienen sistemas de tipos y modelos de objetos diferentes;
- se ejecutan mediante plataformas diferentes;
- compartir parte de la sintaxis superficial no vuelve equivalentes sus programas.

La estandarización llegó mediante Ecma International. El estándar se llama **ECMAScript** y JavaScript es su implementación más conocida.

Una línea de tiempo mínima

- **1995:** nace el lenguaje en el navegador.
- **1997:** se publica la primera edición de ECMAScript.
- **1999:** ECMAScript 3 consolida características que dominaron la Web durante años.
- **2009:** ECMAScript 5 formaliza modo estricto, JSON y métodos modernos de arrays, entre otras mejoras.
- **2015:** ECMAScript 2015 introduce módulos, clases, `let`, `const`, promesas, iteradores, generadores, flechas y una gran actualización del lenguaje.
- **Etapas posteriores:** el estándar adopta entregas regulares e incrementales en lugar de esperar muchos años entre grandes versiones.

El lenguaje actual conserva compatibilidad con una enorme cantidad de código antiguo. Esa continuidad explica algunas peculiaridades como `typeof null`, `var` y la igualdad flexible.

Las tres capas de JavaScript

Cuando decimos "JavaScript", solemos mezclar tres cosas diferentes:

1. **El lenguaje** define la sintaxis y el comportamiento de valores, objetos, funciones y operadores. Su estándar se llama ECMAScript.
2. **El motor** lee y ejecuta ese código. V8 es un motor usado por Chrome y Node.js; otros entornos pueden utilizar motores diferentes.

3. **El anfitrión** ofrece capacidades externas. El navegador aporta el DOM, eventos y `localStorage`; Node.js aporta archivos, procesos y red.

La especificación ECMAScript define valores como `Map`, `Promise`, `Symbol` y `Array`.

Documentos separados definen APIs web como `fetch`, `document` y `setTimeout`. Node.js implementa algunas APIs web y aporta sus propios módulos. Por eso una función puede existir en varios entornos sin pertenecer originalmente al núcleo del lenguaje.

Por eso `document.querySelector()` funciona en una página web pero no es parte del lenguaje, y `readFile()` funciona en Node.js pero no aparece por arte de magia en el navegador.

```
// Lenguaje: funciona en distintos entornos.
const total = [10, 20, 30].reduce((suma, valor) => suma + valor, 0);

// API del navegador.
document.querySelector("#total").textContent = total;

// API de Node.js.
import { writeFile } from "node:fs/promises";
await writeFile("total.txt", String(total), "utf8");
```

Cuando una API “no existe”, la pregunta productiva es: ¿pertenece al lenguaje, al anfitrión, a una biblioteca instalada o a una versión diferente del entorno?

Motores y anfitriones diferentes

Entre los motores conocidos están V8, SpiderMonkey y JavaScriptCore. Todos buscan implementar ECMAScript, aunque pueden diferir durante la incorporación de características nuevas, en optimizaciones y en herramientas de diagnóstico.

El mismo motor puede aparecer en anfitriones distintos. V8 ejecuta código dentro de Chrome y también dentro de Node.js, pero Chrome ofrece DOM y Node.js ofrece `node:fs`. Compartir motor no iguala las APIs.

El anfitrión también coordina tareas alrededor del motor. Un bucle de eventos decide cuándo ejecutar callbacks de temporizadores, red o interfaz. JavaScript puede ejecutar una tarea a la vez en un hilo principal y, aun así, coordinar operaciones asincrónicas realizadas por el entorno.

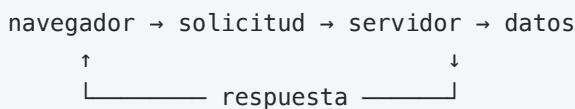
Del navegador a una plataforma completa

JavaScript nació para agregar comportamiento a páginas web y evolucionó hasta convertirse en un lenguaje de propósito general. Hoy puede participar en toda una aplicación:

- en el **frontend**, responde a la interacción del usuario y presenta información;

- en el **backend**, aplica reglas, protege recursos y accede a datos;
- en herramientas de línea de comandos, automatiza tareas;
- en aplicaciones híbridas, comparte modelos y validaciones entre capas.

El lugar de ejecución importa. El frontend está cerca del usuario, pero su código y sus datos pueden inspeccionarse. Las reglas de seguridad, las credenciales y las decisiones que no deben manipularse pertenecen al servidor.



La validación del navegador mejora la experiencia; la validación del servidor conserva la integridad del sistema. Una no reemplaza a la otra.

Frontend: código cerca del usuario

El frontend puede:

- leer eventos de teclado, ratón o tacto;
- modificar la representación de la página;
- validar rápidamente una entrada;
- almacenar preferencias locales;
- solicitar datos a servicios remotos.

Ese código se descarga en el dispositivo del usuario. No debe contener secretos ni ser la única barrera de autorización. Ocultar un botón no impide llamar a la API correspondiente.

Backend: código cerca de datos y reglas

El backend puede:

- autenticar y autorizar;
- aplicar invariantes del negocio;
- acceder a bases de datos y sistemas internos;
- usar credenciales protegidas;
- coordinar operaciones entre usuarios.

Node.js permite escribir ese backend en JavaScript. Usar el mismo lenguaje en ambos extremos facilita compartir conocimientos y algunos contratos, pero no elimina la frontera de seguridad ni convierte en confiables los datos que cruzan la red.

Módulos y dependencias

Un programa real se divide en módulos. Los módulos ECMAScript declaran explícitamente qué exportan e importan:

```
// precios.js
export function calcularTotal(precio, cantidad) {
  return precio * cantidad;
}
```

```
// aplicacion.js
import { calcularTotal } from "./precios.js";

console.log(calcularTotal(100, 3));
```

El especificador `./precios.js` señala un módulo local. Nombres como `node:path` señalan módulos integrados de Node.js. Otros nombres pueden referirse a paquetes instalados y resueltos por el entorno o una herramienta de construcción.

Los módulos crean límites de nombres, permiten reutilización y vuelven visibles las dependencias. Una importación no garantiza que el código sea seguro o compatible: las dependencias externas también requieren evaluación, versiones y mantenimiento.

Qué ocurre al ejecutar

Un motor moderno no se limita a interpretar cada línea de la misma manera. Analiza el código, produce una representación ejecutable y puede optimizar las partes que se usan con frecuencia mediante compilación JIT (*just in time*). Si las suposiciones de una optimización dejan de cumplirse, el motor puede descartarla.

De forma conceptual, el recorrido incluye:

1. **lectura y análisis léxico:** el texto se separa en unidades significativas;
2. **análisis sintáctico:** se comprueba la gramática y se construye una representación del programa;
3. **creación de ámbitos:** se preparan declaraciones y vínculos;
4. **ejecución:** un intérprete o código generado realiza las operaciones;
5. **observación y optimización:** las rutas frecuentes pueden compilarse con supuestos más específicos;
6. **desoptimización:** si los supuestos dejan de cumplirse, el motor vuelve a una representación general.

Un error de sintaxis aparece antes de ejecutar el bloque afectado:

```
// const = 10; // SyntaxError
```

Un error de ejecución surge al alcanzar una operación inválida:

```
const usuario = null;  
// usuario.nombre; // TypeError al ejecutar esta línea
```

Un error lógico no necesariamente lanza nada: el programa corre y produce una respuesta equivocada. Las pruebas y ejemplos son esenciales para detectarlo.

La consecuencia práctica no es “programar para el motor”. Es mantener datos y operaciones comprensibles. Primero se escribe código correcto y medible; la optimización prematura suele aumentar complejidad sin demostrar una mejora.

Interpretación, compilación y transpilación

Las categorías no son excluyentes. Un motor puede interpretar una representación intermedia y compilar partes durante la misma ejecución. “JavaScript es interpretado” describe una visión histórica incompleta; “JavaScript es compilado” también requiere aclarar cuándo y hacia qué representación.

Transpilar suele significar transformar código fuente a otro código fuente de un nivel parecido. Una herramienta puede convertir sintaxis moderna a una forma compatible con entornos antiguos. Eso no agrega automáticamente las APIs que el entorno no posee: una sintaxis puede transformarse, mientras que una funcionalidad ausente necesita un *polyfill* o una estrategia alternativa.

JavaScript y TypeScript

JavaScript tiene tipado dinámico: una variable no queda asociada para siempre a un tipo, aunque cada valor sí tiene uno.

```
let resultado = 42;  
resultado = "cuarenta y dos"; // válido en JavaScript
```

TypeScript agrega un análisis estático que ayuda a detectar inconsistencias antes de ejecutar:

```
let resultado: number = 42;  
resultado = "cuarenta y dos"; // error del verificador
```

El navegador y Node.js ejecutan JavaScript. Por eso TypeScript se transforma —habitualmente se dice que se transpila— a JavaScript. Sus tipos mejoran las herramientas y documentan contratos, pero desaparecen en tiempo de ejecución. Los datos que llegan desde un formulario, archivo o red siguen necesitando validación real.

Dos dimensiones que suelen confundirse

Estático frente a dinámico pregunta cuándo se comprueban tipos:

- estático: una herramienta razona antes de ejecutar;
- dinámico: las comprobaciones principales ocurren con valores durante la ejecución.

Fuerte frente a débil intenta describir cuánto permite el lenguaje mezclar tipos y qué conversiones realiza. No existe una única escala universal. JavaScript conserva distinciones importantes —no puede sumarse directamente `bigint` y `number`, por ejemplo— y también aplica coerciones que sorprenden:

```
"5" + 1; // "51"
"5" - 1; // 4
```

En lugar de etiquetarlo con una sola palabra, conviene aprender las reglas concretas de cada operador y convertir en los bordes.

Los tipos atraviesan fronteras

Un formulario entrega texto:

```
const edadTexto = formulario.elements.edad.value;
```

El frontend puede convertir y mostrar una advertencia. Al serializar JSON, el número viaja sin información de TypeScript. El backend recibe bytes, interpreta JSON y vuelve a validar:

```
function validarSolicitud(datos) {
  if (!Number.isSafeInteger(datos.edad) || datos.edad < 0) {
    throw new TypeError("Edad inválida");
  }

  return datos;
}
```

Una anotación como `datos: Solicitud` expresa lo que el desarrollador espera; no transforma una entrada desconocida en una solicitud válida.

Modo estricto

El modo estricto elimina o convierte en errores varios comportamientos históricos problemáticos:

```
"use strict";
```

Los módulos ECMAScript son estrictos de forma automática. Entre otros efectos, una asignación a un identificador no declarado lanza error y `this` en una llamada de función simple queda `undefined` en lugar de convertirse en el objeto global.

No reemplaza validación ni estilo, pero ofrece una semántica más segura para código moderno.

Un método productivo para comenzar

Antes de escribir código, respondé cuatro preguntas:

1. ¿Qué datos entran?
2. ¿Qué resultado debe salir?
3. ¿Qué reglas conectan ambos extremos?
4. ¿Qué puede fallar y cómo lo observaremos?

Agregá dos más cuando la solución cruza entornos:

1. ¿En qué anfitrión se ejecuta cada parte y qué APIs necesita?
2. ¿Qué datos atraviesan una frontera y deben volver a validarse?

Después construí la versión más pequeña que recorra el camino completo. Para la inscripción:

```
const solicitud = { nombre: "Ada", edad: 20 };  
const respuesta = evaluarInscripcion(solicitud);  
console.log(respuesta);
```

Recién entonces agregá más reglas. Este recorrido vertical entrega evidencia temprana y reduce el costo de corregir una idea equivocada.

Leer documentación con el modelo de capas

Ante una API, identificá:

- si está definida por ECMAScript, la Web, Node.js o una biblioteca;
- qué versiones del entorno la implementan;
- si es síncrona, devuelve una promesa o usa callbacks;
- qué tipos acepta y produce;
- qué errores o resultados de ausencia forman parte del contrato.

Este hábito evita copiar ejemplos de un anfitrión a otro sin comprender por qué fallan.

Para recordar

- Lenguaje, motor y anfitrión son capas distintas.
- ECMAScript estandariza el núcleo; las APIs del entorno pertenecen a especificaciones y plataformas adicionales.
- Frontend y backend resuelven responsabilidades diferentes.
- TypeScript comprueba tipos durante el desarrollo; no valida datos externos por sí solo.
- Los motores analizan, ejecutan y pueden optimizar; sintaxis, ejecución y lógica fallan en momentos diferentes.
- Un programa productivo comienza por el contrato de entrada y salida, no por una lista de características del lenguaje.

CAPÍTULO 2

Variables, alcance y expresiones

Idea central

Una variable no es una caja: es un nombre vinculado con un valor durante una parte determinada de la ejecución. Para escribir programas previsibles conviene crear ese vínculo lo más cerca posible de su uso, preferir `const`, reservar `let` para cambios deliberados y elegir nombres que expresen el papel del dato.

Esta conclusión se sostiene en tres ideas:

1. declaración, inicialización y asignación son momentos diferentes;
2. el alcance determina dónde puede utilizarse un nombre y la vida del dato determina cuánto tiempo existe;
3. una expresión produce un valor, y los operadores determinan cómo se construye ese resultado.

De un valor anónimo a un dato con propósito

Un programa puede operar directamente sobre literales:

```
1500 * 3;
```

El cálculo es válido, pero no comunica qué representan esos números. Al darles nombre aparece la intención:

```
const precioUnitario = 1500;  
const cantidad = 3;  
const subtotal = precioUnitario * cantidad;
```

Una **declaración** introduce el nombre. La **inicialización** le asigna su primer valor. Una **reasignación** reemplaza posteriormente el valor vinculado.

```
let total;           // declaración; su valor inicial es undefined  
total = 100;         // asignación  
total = 150;         // reasignación  
  
const iva = 0.21;    // declaración e inicialización juntas
```

`const` exige inicialización inmediata porque el vínculo no podrá cambiar.

Elegir entre `const`, `let` y `var`

La regla de trabajo es:

1. empezá con `const` ;
2. cambiá a `let` solo si la reasignación expresa una evolución real del algoritmo;
3. evitá `var` en código nuevo.

`const` : el nombre conserva su vínculo

```
const moneda = "ARS";  
// moneda = "USD"; // TypeError
```

`const` no vuelve inmutable el valor referenciado. Si el valor es un objeto, sus propiedades todavía pueden cambiar:

```
const pedido = { estado: "pendiente", total: 1000 };  
pedido.estado = "pagado"; // permitido
```

Lo que no puede hacerse es vincular `pedido` con otro objeto:

```
// pedido = { estado: "nuevo", total: 0 };
```

Esta diferencia —vínculo constante frente a valor inmutable— será esencial al estudiar arrays y objetos.

`let` : el cambio forma parte del modelo

```
let saldo = 10_000;  
saldo -= 2_500;  
saldo -= 1_000;
```

`let` es apropiado para acumuladores, índices, estados que avanzan y referencias que se reemplazan. Que pueda cambiar no significa que deba hacerlo desde muchos lugares. Cuanto menor sea su alcance, más fácil será reconstruir su historia.

var : una regla histórica diferente

`var` tiene alcance de función, no de bloque, permite redeclaración y su inicialización se comporta de manera diferente durante la elevación:

```
function ejemplo() {  
  if (true) {  
    var mensaje = "visible en toda la función";  
  }  
  
  console.log(mensaje);  
}
```

Con `let` o `const`, `mensaje` solo existiría dentro del `if`. Comprender `var` sigue siendo útil para leer código antiguo, pero rara vez mejora un diseño nuevo.

Alcance: desde dónde puede verse un nombre

El **alcance léxico** se deduce de la estructura del código. Un bloque interior puede consultar nombres exteriores, pero el exterior no puede consultar los nombres privados del interior.

```
const recargoGeneral = 0.05;  
  
function cotizar(precio) {  
  const subtotal = precio * (1 + recargoGeneral);  
  
  if (subtotal > 10_000) {  
    const envio = 0;  
    return subtotal + envio;  
  }  
  
  return subtotal + 800;  
}
```

Aquí aparecen varios niveles:

- `recargoGeneral` está en el alcance exterior;
- `precio` y `subtotal` viven en el alcance de la función;
- `envio` solo existe en el bloque del `if`.

Alcance global

Un nombre global puede consultarse desde muchas partes. Esa comodidad tiene un costo: cualquier cambio puede afectar a consumidores lejanos. Preferí módulos y parámetros explícitos para compartir datos.

En navegadores, algunas declaraciones globales históricas se reflejan en `globalThis` o `window`, pero no todas lo hacen del mismo modo. En módulos de JavaScript, los nombres del archivo tienen alcance de módulo y no se convierten automáticamente en propiedades globales.

Alcance de función

Parámetros y declaraciones internas pertenecen a cada llamada:

```
function sumarImpuesto(precio, tasa) {  
  const impuesto = precio * tasa;  
  return precio + impuesto;  
}  
  
sumarImpuesto(100, 0.21);  
sumarImpuesto(200, 0.10);
```

Las dos llamadas crean contextos independientes. El `impuesto` de una no se mezcla con el de la otra.

Alcance de bloque

`let`, `const` y `class` respetan las llaves de `if`, `for`, `while`, `switch` y bloques sueltos:

```
for (let indice = 0; indice < 3; indice += 1) {  
  const posicionHumana = indice + 1;  
  console.log(posicionHumana);  
}  
  
// indice y posicionHumana ya no existen aquí
```

Sombreado: un nombre puede ocultar a otro

Un bloque puede declarar un nombre que ya existe fuera:

```
const estado = "global";  
  
function informar() {  
  const estado = "local";  
  return estado;  
}
```

Esto es válido y a veces útil, pero puede dificultar la lectura si ambos valores participan en la misma operación. Usá nombres diferentes cuando representen conceptos diferentes.

Elevación y zona muerta temporal

Las declaraciones se procesan antes de ejecutar el bloque, pero no todas quedan utilizables de la misma forma.

```
// console.log(total); // ReferenceError
const total = 100;
```

Desde el comienzo del bloque hasta la inicialización, `total` está en la **zona muerta temporal**. No se trata de que el nombre sea desconocido: existe, pero todavía no puede leerse.

Con `var`, la declaración se eleva e inicializa con `undefined`:

```
console.log(totalHistorico); // undefined
var totalHistorico = 100;
```

Eso puede ocultar el uso prematuro de un dato. Las declaraciones de función completas también se elevan, mientras que una función almacenada en `const` sigue las reglas de `const`.

La práctica más clara es declarar cerca del primer uso y no depender de la elevación para organizar el archivo.

Vida de un valor y recolección de memoria

El alcance indica dónde puede escribirse un nombre; la **vida** indica durante cuánto tiempo el valor sigue siendo alcanzable. Un valor local suele dejar de ser necesario al terminar la función. El recolector de basura puede liberar su memoria cuando ya no existe ninguna referencia accesible.

Una clausura puede prolongar esa vida:

```
function crearSecuencia() {
  let siguiente = 1;

  return () => siguiente++;
}

const obtenerId = crearSecuencia();
obtenerId(); // 1
obtenerId(); // 2
```

Aunque `crearSecuencia` terminó, la función devuelta conserva acceso a `siguiente`. Volveremos sobre este mecanismo en el capítulo de funciones.

Identificadores válidos y nombres útiles

Un identificador puede contener letras Unicode, dígitos, `_` y `$`, pero no puede comenzar con un dígito ni coincidir con una palabra reservada.

```
const año = 2026;  
const _interno = true;  
const $elemento = null;  
// const 2curso = "A";
```

Las convenciones más habituales son:

- `camelCase` para variables y funciones;
- `PascalCase` para clases y constructores;
- mayúsculas con guiones bajos para constantes verdaderamente globales e inmutables, como `MAX_INTENTOS`.

Un nombre describe el papel, no solo el tipo:

```
const datos = 15;           // ¿qué datos?  
const diasHastaVencimiento = 15; // intención visible
```

Para booleanos funcionan bien prefijos como `es`, `tiene`, `puede` y `debe`. Para funciones, un verbo expresa la acción: `calcularTotal`, `buscarAlumno`, `guardarArchivo`.

Expresiones y sentencias

Una **expresión** produce un valor:

```
2 + 3  
precio * cantidad  
edad >= 18  
usuario?.nombre ?? "Anónimo"
```

Una **sentencia** realiza una acción estructural, como declarar, decidir o repetir:

```
const total = precio * cantidad;  
  
if (total > limite) {  
  console.log("Requiere autorización");  
}
```

Una expresión puede aparecer dentro de una sentencia. Reconocer esta diferencia ayuda a leer funciones flecha, operadores condicionales y asignaciones.

Operadores: aridad, precedencia y asociatividad

Un operador puede ser:

- unario: `!activo`, `typeof valor`, `-cantidad`;
- binario: `a + b`, `x === y`;
- ternario: `condicion ? valorA : valorB`.

La **precedencia** indica qué operador se agrupa primero:

```
2 + 3 * 4; // 14, porque * tiene mayor precedencia
```

La **asociatividad** decide cómo se agrupan operadores del mismo nivel:

```
10 - 3 - 2; // (10 - 3) - 2 = 5  
2 ** 3 ** 2; // 2 ** (3 ** 2) = 512
```

La asignación se asocia desde la derecha:

```
let a;  
let b;  
a = b = 10;
```

No conviene convertir estas reglas en un acertijo. Los paréntesis documentan la intención:

```
const total = (precio * cantidad) - descuento;  
const habilitado = esCliente && (tieneSaldo || tieneCredito);
```

Asignaciones abreviadas e incremento

```
saldo += deposito;  
stock -= vendido;  
factor *= 2;  
indice += 1;
```

El incremento prefijo cambia y luego devuelve; el sufijo devuelve el valor anterior y luego cambia:

```
let n = 5;
const anterior = n++; // anterior = 5, n = 6
const actual = ++n;    // n = 7, actual = 7
```

Cuando el valor producido importa, `n += 1` seguido de una lectura explícita suele ser más fácil de entender.

Un método productivo para revisar variables

Al leer una función, preguntá por cada nombre:

1. ¿Qué representa?
2. ¿Quién puede cambiarlo?
3. ¿Durante cuánto tiempo debe existir?
4. ¿Podría calcularse en lugar de almacenarse?
5. ¿Su unidad está en el nombre o en el contrato?

```
function calcularDemoraHoras(inicioMs, finMs) {
  const MILISEGUNDOS_POR_HORA = 3_600_000;
  const duracionMs = finMs - inicioMs;
  return duracionMs / MILISEGUNDOS_POR_HORA;
}
```

El nombre evita confundir milisegundos con horas y la constante explica el factor de conversión.

Errores frecuentes

Crear que `const` vuelve profundo e inmutable al objeto

`const` evita reasignar la variable; las propiedades siguen siendo modificables.

Declarar todo al comienzo

Aleja la creación del uso, amplía innecesariamente el alcance y obliga a mantener estados parciales.

Reutilizar una variable para conceptos distintos

```
let resultado = leerEntrada();
resultado = Number(resultado);
resultado = resultado * 1.21;
```

Es más claro distinguir `entrada`, `precio` y `precioConIva`.

Depender de precedencia difícil de reconocer

Aunque el código sea correcto, los paréntesis pueden evitar una revisión lenta o una modificación incorrecta.

Para recordar

- Una variable vincula un nombre con un valor dentro de un alcance.
- `const` es la elección inicial; `let` expresa evolución; `var` queda para comprender código histórico.
- Alcance y vida no son lo mismo: una clausura puede mantener vivo un dato local.
- Una expresión produce un valor; los operadores lo construyen según precedencia y asociatividad.
- Los nombres, las unidades y el alcance reducido son herramientas de corrección, no decoración.

CAPÍTULO 3

El tipo boolean

Idea central

JavaScript permite usar cualquier valor como condición, pero una condición verdadera no implica que su resultado sea el booleano `true`. Para evitar errores hay que distinguir booleanos de valores *truthy* y *falsy*, y recordar que `&&` y `||` seleccionan operandos mediante cortocircuito.

Esta distinción permite usar los operadores lógicos de forma productiva sin confundir validación, ausencia y selección de valores.

Dos valores para representar una decisión

El tipo `boolean` tiene solo dos valores:

```
true  
false
```

Las comparaciones producen booleanos:

```
10 > 3;           // true  
5 === "5";        // false  
"ana" !== "Ana";  // true
```

Un nombre booleano debería permitir leer la condición como una frase:

```
const esMayorDeEdad = edad >= 18;  
const tieneCupo = inscriptos < capacidad;  
const puedeIngresar = esMayorDeEdad && tieneCupo;
```

Guardar una condición intermedia no es obligatorio, pero puede volver explícita la regla del dominio.

Una condición no exige un booleano

`if`, `while`, el operador ternario y los operadores lógicos aceptan cualquier valor. Antes de decidir, JavaScript lo interpreta en contexto booleano:

```
if (nombre) {  
  console.log(`Hola, ${nombre}`);  
}
```

`nombre` puede ser un string. El lenguaje aplica conceptualmente `Boolean(nombre)`.

La lista completa de valores *falsy*

Los valores cuya conversión booleana da `false` son:

```
false  
0  
-0  
0n  
""  
null  
undefined  
NaN
```

En navegadores existe una excepción histórica muy especializada: `document.all` también se comporta como falsy. No debe utilizarse para lógica de aplicación.

Todos los demás valores son *truthy*. Algunos sorprenden:

```
Boolean("false"); // true: es una cadena no vacía  
Boolean("0");     // true  
Boolean([]);      // true  
Boolean({});      // true  
Boolean(-1);      // true
```

Truthy no significa “verdadero según el significado humano”. Solo significa que la conversión definida por el lenguaje produce `true`.

Conversión explícita

La forma descriptiva es `Boolean`:

```
Boolean(1);    // true
Boolean(0);    // false
Boolean("ok"); // true
Boolean("");   // false
```

La doble negación produce el mismo resultado:

```
!!"ok"; // true
!!0;    // false
```

La primera `!` convierte e invierte; la segunda vuelve a invertir. `Boolean(valor)` suele comunicar mejor la intención en código didáctico y de negocio; `!!valor` es una abreviatura común que conviene reconocer.

Negación con `!`

`!` siempre devuelve un booleano:

```
!"texto"; // false
!0;        // true
```

Una condición negativa puede resultar más difícil de leer, especialmente si se combina con nombres negativos:

```
if (!usuario.noEstaBloqueado) {
    // doble negación conceptual
}
```

Preferí nombres afirmativos:

```
function puedeContinuar(usuario) {
    if (usuario.estaBloqueado) return false;
    return true;
}
```

`&&`: avanzar mientras los valores sean *truthy*

El operador AND evalúa de izquierda a derecha:

1. si encuentra un valor falsy, lo devuelve y se detiene;

2. si todos son *truthy*, devuelve el último.

```
true && true;           // true
"Ana" && 20;            // 20
"" && ejecutar();       // ""; ejecutar no se llama
1 && "ok" && { id: 1 }; // { id: 1 }
```

Este comportamiento se llama **cortocircuito**. Permite proteger una operación:

```
const ciudad = usuario && usuario.direccion && usuario.direccion.ciudad;
```

Hoy suele ser más claro usar encadenamiento opcional:

```
const ciudad = usuario?.direccion?.ciudad;
```

`&&` también puede ejecutar condicionalmente un efecto:

```
estaListo && iniciar();
```

La forma con `if` suele ser preferible si el efecto es importante o si el lector podría confundir el resultado:

```
if (estaListo) iniciar();
```

`||` : buscar el primer valor *truthy*

OR también evalúa de izquierda a derecha:

1. devuelve el primer operando *truthy*;
2. si ninguno lo es, devuelve el último *falsy*.

```
"Ana" || "Anónimo";      // "Ana"
"" || "Anónimo";         // "Anónimo"
0 || false || null;      // null
configuracionA || configuracionB || configuracionBase;
```

Durante años se utilizó para valores predeterminados:

```
const cantidad = entrada || 1;
```

Pero reemplaza cualquier falsy. Si `0` es una cantidad válida, el resultado será incorrecto.

?? : ausencia nula, no falsedad

La coalescencia nula devuelve el operando derecho solo cuando el izquierdo es `null` o `undefined`:

```
0 ?? 1;           // 0
false ?? true;    // false
"" ?? "texto";    // ""
null ?? "texto";  // "texto"
```

Regla práctica:

- usá `||` cuando querés reemplazar cualquier valor falsy;
- usá `??` cuando solo querés reemplazar ausencia.

JavaScript exige paréntesis al mezclar directamente `??` con `&&` o `||`, porque la intención puede ser ambigua:

```
const valor = (preferido || alternativo) ?? predeterminado;
```

Los operadores lógicos conservan valores

La simetría ayuda a recordarlos:

```
a && b → primer falsy; si no existe, último valor
a || b → primer truthy; si no existe, último valor
```

No son funciones que siempre produzcan `true` o `false`; son operadores de control y selección. Si la API necesita un booleano real, convertí el resultado:

```
const tieneNombre = Boolean(usuario.nombre);
```

Comparaciones y orden

Los operadores relacionales producen booleanos:

```
3 < 10;  
10 >= 10;  
"Ana" < "Luis";
```

Con operandos del mismo tipo numérico, el sentido suele ser evidente. Con strings, la comparación sigue unidades de código, no el orden lingüístico humano. Con tipos diferentes puede haber coerción numérica:

```
"10" < 2; // false; "10" se convierte en 10
```

Convertí entradas antes de comparar y usé `Intl.Collator` para orden alfabético destinado a personas.

Igualdad estricta y flexible

La igualdad estricta no convierte tipos:

```
5 === 5;    // true  
5 === "5";  // false
```

La igualdad flexible aplica reglas de coerción:

```
5 == "5";      // true  
false == 0;    // true  
null == undefined; // true
```

La regla general es usar `===` y `!==`. La comparación `valor == null` es un modismo deliberado que detecta `null` o `undefined`, pero `valor === null || valor === undefined` es más explícito para quien está aprendiendo.

`NaN` es diferente de todos los valores, incluso de sí mismo:

```
NaN === NaN;      // false  
Number.isNaN(NaN); // true
```

`Object.is` distingue casos especiales:

```
Object.is(NaN, NaN); // true  
Object.is(0, -0);    // false
```

Combinar condiciones

Una regla puede construirse con condiciones pequeñas:

```
const tieneEdadValida = edad >= 18;
const tieneDocumentacion = Boolean(dni && constancia);
const noEstaSuspendido = !estaSuspendido;

const puedeInscribirse =
  tieneEdadValida &&
  tieneDocumentacion &&
  noEstaSuspendido;
```

Los saltos de línea y nombres intermedios permiten revisar cada parte. No es necesario comprimir una regla compleja en una sola expresión.

Leyes de De Morgan

Al negar una condición compuesta:

```
!(a && b) equivale a !a || !b
!(a || b) equivale a !a && !b
```

Ejemplo:

```
const noPuedeComprar = !(tieneSaldo && hayStock);
const equivalente = !tieneSaldo || !hayStock;
```

Estas leyes ayudan a transformar condiciones y escribir guardas tempranas.

Precedencia y paréntesis

En las operaciones habituales:

```
! se evalúa antes que &&
&& se evalúa antes que ||
|| se evalúa antes que el ternario
```

```
const acceso = esAdmin || esEditor && estaActivo;
// equivale a: esAdmin || (esEditor && estaActivo)
```

Aunque conozcas la precedencia, agregá paréntesis si representan una regla conceptual:

```
const acceso = esAdmin || (esEditor && estaActivo);
```

Asignación lógica

Los operadores de asignación lógica actualizan solo cuando corresponde:

```
config.tema ||= "claro";      // si tema es falsy  
config.intentos ??= 3;        // si es null o undefined  
sesion.activa &&= verificar(); // si activa es truthy
```

Son útiles, pero conservan las mismas reglas de `||`, `??` y `&&`. Antes de usarlos, decidí qué valores son válidos en el dominio.

Errores frecuentes

Interpretar una cadena como su significado humano

```
Boolean("false"); // true
```

Para una entrada textual, definí un parser:

```
function leerBooleano(texto) {  
  const normalizado = texto.trim().toLowerCase();  
  if (normalizado === "true") return true;  
  if (normalizado === "false") return false;  
  throw new TypeError("Se esperaba true o false");  
}
```

Usar `||` cuando cero, vacío o `false` son válidos

Usá `??` si solo querés cubrir ausencia.

Esperar que `&&` produzca un booleano

`usuario && usuario.nombre` puede devolver `null`, `undefined`, `""` o el nombre. Convertí si el contrato exige `boolean`.

Ocultar demasiado en una expresión

Una cadena larga de cortocircuitos puede ser compacta y difícil de depurar. Separá reglas y efectos.

Para recordar

- Solo `true` y `false` son booleanos; todos los valores tienen una interpretación booleana.
- La lista de falsy es pequeña y cerrada; arrays y objetos vacíos son truthy.
- `&&` devuelve el primer falsy o el último valor; `||`, el primer truthy o el último.
- `??` trata únicamente `null` y `undefined` como ausencia.
- Las condiciones importantes merecen nombres, paréntesis y un contrato booleano explícito.

CAPÍTULO 4

Los tipos number y bigint

Idea central

`number` sirve para la mayoría de los cálculos, pero no representa todos los decimales ni todos los enteros con exactitud; `bigint` representa enteros arbitrariamente grandes, aunque no puede mezclarse libremente con `number`. La elección correcta depende de la precisión, el rango y las operaciones del dominio.

Para trabajar con números de forma segura hay que separar cuatro preguntas:

1. ¿cómo se escribe o se convierte la entrada?
2. ¿qué puede representar el tipo?
3. ¿qué operación y precisión exige el problema?
4. ¿cómo se presenta el resultado sin confundir formato con dato?

Un único tipo para enteros y decimales

En JavaScript, `number` utiliza el formato IEEE 754 de doble precisión. El mismo tipo representa:

```
42
-7
3.14159
1.5e6
Infinity
NaN
```

No existe un tipo entero pequeño separado. Un valor puede ser entero desde el punto de vista matemático y seguir almacenado como `number`.

```
Number.isInteger(42); // true
Number.isInteger(42.5); // false
```

Literales y bases

```
const decimal = 42;
const binario = 0b101010;
const octal = 0o52;
const hexadecimal = 0x2a;

decimal === binario;    // true
binario === hexadecimal; // true
```

La base cambia cómo escribimos el literal, no el valor matemático. Los separadores numéricos mejoran la lectura:

```
const poblacion = 46_000_000;
const mascara = 0b1111_0000;
```

La notación exponencial expresa escalas grandes o pequeñas:

```
1e3;    // 1000
2.5e-3; // 0.0025
```

Operaciones aritméticas

```
10 + 3; // 13
10 - 3; // 7
10 * 3; // 30
10 / 3; // 3.333...
10 % 3; // 1
2 ** 3; // 8
```

El operador `%` produce el resto, no un módulo matemático siempre positivo:

```
-5 % 3; // -2
```

Para normalizar un valor cíclico:

```
function modulo(valor, base) {
  return ((valor % base) + base) % base;
}
```

```
modulo(-1, 7); // 6
```

La exponenciación se asocia desde la derecha:

```
2 ** 3 ** 2; // 2 ** (3 ** 2) = 512
```

JavaScript exige paréntesis para una base unaria negativa:

```
(-2) ** 2; // 4  
-(2 ** 2); // -4
```

División y valores especiales

Dividir por cero no lanza una excepción en `number`:

```
1 / 0; // Infinity  
-1 / 0; // -Infinity  
0 / 0; // NaN
```

`NaN` significa que una operación numérica no produjo un número válido. Sigue perteneciendo al tipo `number`:

```
typeof NaN; // "number"
```

No se compara consigo mismo:

```
NaN === NaN; // false
```

Para detectarlo usá `Number.isNaN`, no la función global `isNaN`, que primero convierte:

```
Number.isNaN(NaN); // true  
Number.isNaN("hola"); // false  
isNaN("hola"); // true, porque Number("hola") es NaN
```

`Number.isFinite` comprueba que un valor ya numérico sea finito:

```
Number.isFinite(10); // true  
Number.isFinite(Infinity); // false
```

```
Number.isFinite("10");    // false
```

El cero negativo

IEEE 754 distingue `0` de `-0`, aunque la igualdad estricta los considera iguales:

```
0 === -0;           // true
Object.is(0, -0);   // false
1 / 0;              // Infinity
1 / -0;             // -Infinity
```

En la mayoría de las aplicaciones no importa. Puede ser relevante en cálculos que conservan dirección o signo al aproximarse a cero.

Los decimales no siempre caben exactamente

Muchos decimales finitos en base diez son periódicos en base dos:

```
0.1 + 0.2; // 0.30000000000000004
```

No es un error exclusivo de JavaScript, sino una consecuencia de la representación finita. No compares cálculos decimales esperando siempre igualdad exacta:

```
function casiIguales(a, b, tolerancia = Number.EPSILON) {
  return Math.abs(a - b) <= tolerancia * Math.max(1, Math.abs(a), Math.abs(b));
}
```

`Number.EPSILON` es la distancia entre `1` y el siguiente número representable, no una tolerancia universal. El dominio debe definir una tolerancia coherente con su escala.

Dinero

Para dinero simple, suele ser más seguro guardar unidades mínimas enteras:

```
const precioCentavos = 19_99;
const cantidad = 3;
const totalCentavos = precioCentavos * cantidad;
```

Esto no resuelve por sí solo porcentajes, redondeos legales, monedas con distinta cantidad de decimales ni importes gigantes. Sistemas financieros serios suelen usar enteros con reglas explícitas o bibliotecas decimales.

Enteros seguros

`number` representa exactamente enteros entre:

```
Number.MIN_SAFE_INTEGER;  
Number.MAX_SAFE_INTEGER; // 9007199254740991
```

Fuera de ese rango, enteros diferentes pueden volverse indistinguibles:

```
const limite = Number.MAX_SAFE_INTEGER;  
limite + 1 === limite + 2; // true
```

Comprueba entradas que deban ser enteros exactos:

```
Number.isSafeInteger(123); // true
```

Identificadores numéricos largos —tarjetas, códigos, documentos— muchas veces no son cantidades. Guardarlos como strings conserva ceros iniciales y evita operaciones sin sentido.

Convertir a `number`

`Number` exige que toda la cadena represente un número, ignorando espacios exteriores:

```
Number("42");      // 42  
Number(" 42 ");    // 42  
Number("");        // 0  
Number("42px");    // NaN  
Number(null);      // 0  
Number(undefined); // NaN
```

El caso de la cadena vacía exige validación previa si el campo es obligatorio.

El `+` unario también convierte, pero es menos descriptivo:

```
+"42"; // 42
```

`parseInt` y `parseFloat` leen un prefijo válido:

```
parseInt("42px", 10); // 42
parseFloat("3.14kg"); // 3.14
```

Esto es útil cuando el contrato acepta unidades posteriores; es peligroso si esperamos que toda la entrada sea numérica. Indicá siempre la base de `parseInt` cuando el formato la conozca:

```
parseInt("1010", 2); // 10
```

Para convertir otras bases a texto:

```
(42).toString(2); // "101010"
(42).toString(16); // "2a"
```

Redondear y truncar

```
Math.floor(3.8); // 3, hacia -Infinity
Math.ceil(3.2); // 4, hacia +Infinity
Math.round(3.5); // 4
Math.trunc(3.8); // 3, elimina la fracción
```

Con negativos, `floor` y `trunc` difieren:

```
Math.floor(-3.2); // -4
Math.trunc(-3.2); // -3
```

Otros métodos frecuentes:

```
Math.abs(-10); // 10
Math.min(3, 7, 1); // 1
Math.max(3, 7, 1); // 7
Math.sqrt(81); // 9
Math.random(); // valor en [0, 1)
```

Para un entero aleatorio uniforme entre `min` y `max`, inclusive:

```
function enteroAleatorio(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
```

```
}
```

`Math.random` no es apropiado para seguridad, claves ni sorteos auditables.

Formatear no es calcular

`toFixed` devuelve una cadena:

```
const texto = (12.5).toFixed(2); // "12.50"  
typeof texto;                  // "string"
```

No sigas calculando sobre ese resultado sin una conversión deliberada. Para presentar números según idioma y moneda:

```
const formato = new Intl.NumberFormat("es-AR", {  
  style: "currency",  
  currency: "ARS"  
});  
  
formato.format(1234.5);
```

El valor interno sigue siendo numérico; el formato pertenece al borde de salida.

Operadores bit a bit

Los operadores bitwise de `number` convierten los operandos a enteros de 32 bits con signo:

```
0b1100 & 0b1010; // 0b1000, AND  
0b1100 | 0b1010; // 0b1110, OR  
0b1100 ^ 0b1010; // 0b0110, XOR  
~0;              // -1
```

Los desplazamientos son:

```
1 << 3;   // 8  
8 >> 1;   // 4, conserva el signo  
-1 >>> 1; // desplaza rellenando con ceros
```

La conversión a 32 bits puede truncar valores grandes y descartar fracciones:

```
5.9 | 0; // 5, pero no debe usarse como sustituto general de Math.trunc
```

Máscaras de opciones

```
const LEER = 1 << 0;
const ESCRIBIR = 1 << 1;
const BORRAR = 1 << 2;

let permisos = LEER | ESCRIBIR;           // activar
const puedeLeer = (permisos & LEER) !== 0; // consultar
permisos &= ~ESCRIBIR;                   // apagar
permisos ^= BORRAR;                       // alternar
```

Una máscara es compacta e interoperable. En lógica de negocio, un `Set` de nombres puede ser más legible. Elegí según la representación externa y las operaciones necesarias.

`bigint`: enteros sin el límite seguro de `number`

Un literal termina en `n`:

```
const poblacionGalactica = 9_007_199_254_740_993n;
const convertido = BigInt("9007199254740993");
```

Soporta aritmética entera:

```
10n + 2n; // 12n
10n * 2n; // 20n
10n ** 3n; // 1000n
10n / 3n; // 3n: descarta la fracción
```

No puede mezclarse directamente con `number`:

```
// 10n + 2; // TypeError
10n + BigInt(2); // 12n
```

La conversión debe ser consciente del rango. Convertir un `bigint` enorme a `number` puede perder precisión; convertir un decimal no entero a `bigint` lanza error.

```
BigInt(42); // 42n
// BigInt(4.2); // RangeError
```


Las comparaciones relacionales permiten mezclar ambos tipos en ciertos casos:

```
10n < 11; // true
```

La igualdad estricta conserva la diferencia:

```
10n === 10; // false
```

`Math` no acepta `bigint`, el operador `>>>` no está disponible y `JSON.stringify` no lo serializa por defecto:

```
// JSON.stringify({ valor: 10n }); // TypeError
```

Un formato externo debe decidir cómo representarlo, normalmente como string, y cómo reconstruirlo.

Validar una entrada numérica

```
function leerCantidad(texto) {
  const limpio = texto.trim();

  if (limpio === "") {
    throw new TypeError("La cantidad es obligatoria");
  }

  const cantidad = Number(limpio);

  if (!Number.isSafeInteger(cantidad) || cantidad < 0) {
    throw new RangeError("La cantidad debe ser un entero seguro no negativo");
  }

  return cantidad;
}
```

La validación separa presencia, conversión, clase de número y rango del dominio. El resto del programa puede trabajar con una garantía más fuerte.

Errores frecuentes

- esperar exactitud decimal sin definir tolerancia o redondeo;
- usar `parseInt` cuando el texto completo debería ser válido;

- considerar `NaN` mediante igualdad;
- confundir `toFixed` con una operación numérica;
- guardar identificadores largos como cantidades;
- usar bitwise sobre valores que necesitan más de 32 bits;
- mezclar `number` y `bigint` sin una conversión justificada.

Para recordar

- `number` es punto flotante de doble precisión: tiene valores especiales, errores decimales y un rango entero seguro.
- Convertir, validar, calcular y formatear son etapas distintas.
- Los operadores bitwise de `number` trabajan sobre enteros de 32 bits.
- `bigint` amplía los enteros, pero pierde fracciones, compatibilidad con `Math` y serialización JSON directa.
- La precisión correcta es una decisión del dominio, no un detalle que pueda dejarse implícito.

CAPÍTULO 5

El tipo string y Unicode

Idea central

Un string es una secuencia inmutable de unidades UTF-16, no una lista perfecta de letras visibles. Para procesar texto correctamente hay que elegir la unidad adecuada —unidad de código, punto de código o grafema—, normalizar cuando corresponda y separar comparación técnica de orden lingüístico.

Esta idea explica por qué el texto cotidiano parece sencillo hasta que aparecen acentos combinados, alfabetos diferentes o emoji.

Crear cadenas

JavaScript admite comillas simples, dobles y plantillas literales:

```
const simple = 'Hola';
const doble = "Hola";
const nombre = "Ana";
const plantilla = `Hola, ${nombre}`;
```

Las comillas simples y dobles tienen el mismo comportamiento. Elegí una convención y dejá que una herramienta de formato la aplique.

Las secuencias de escape representan caracteres especiales:

```
const linea = "primera\nsegunda";
const tabulado = "nombre\tedad";
const comillas = "Dijo: \"hola\"";
const barra = "C:\\datos\\archivo.txt";
```

También pueden escribirse puntos de código:

```
const letra = "\u0041";           // A
const emoji = "\u{1F600}";       // 😊; requiere llaves por superar FFFF
```

Plantillas literales

Las comillas invertidas permiten interpolar expresiones y escribir varias líneas:

```
const producto = "teclado";
const precio = 25000;

const mensaje = `${producto.toUpperCase(): ${precio}}`;
```

Dentro de `${...}` puede aparecer cualquier expresión, pero una plantilla muy compleja pierde legibilidad. Calculá primero las decisiones importantes:

```
const precioFormateado = formatoMoneda.format(precio);
const mensaje = `${producto}: ${precioFormateado}`;
```

Las plantillas preservan saltos y espacios de la fuente:

```
const bloque = `línea 1
línea 2`;
```

Los strings son inmutables

No se puede cambiar una posición:

```
const palabra = "casa";
palabra[0] = "C"; // no modifica el string
```

Cada método produce un nuevo valor:

```
const original = " JavaScript ";
const limpio = original.trim();
const mayusculas = limpio.toUpperCase();

original; // " JavaScript "
limpio;   // "JavaScript"
mayusculas; // "JAVASCRIPT"
```

Una variable `let` puede vincularse después con otra cadena, pero ninguna de las cadenas fue modificada internamente.

Longitud y acceso por índice

```
const texto = "JavaScript";
texto.length; // 10
texto[0];     // "J"
texto.at(-1); // "t"
```

Los índices y `length` operan sobre **unidades de código UTF-16**. Para texto ASCII y muchos caracteres del alfabeto latino, una unidad coincide con lo que esperamos. No es una garantía general.

`charAt` es una API histórica:

```
texto.charAt(0); // "J"
texto[99];       // undefined
texto.charAt(99); // ""
```

`codePointAt` obtiene el punto de código que comienza en una posición:

```
"😄".codePointAt(0); // 128512
String.fromCodePoint(128512); // "😄"
```

Extraer partes

`slice(inicio, fin)` usa un fin exclusivo y acepta índices negativos:

```
const lenguaje = "JavaScript";
lenguaje.slice(0, 4); // "Java"
lenguaje.slice(4);   // "Script"
lenguaje.slice(-6);  // "Script"
```

`substring` también usa fin exclusivo, pero transforma negativos en cero e intercambia los argumentos si están invertidos. `substr` es histórica y debe evitarse en código nuevo. `slice` ofrece el modelo más coherente con arrays.

Recordá que cortar por índices UTF-16 puede dividir un carácter compuesto o un emoji.

Buscar contenido literal

```
const frase = "Aprender JavaScript con práctica";

frase.includes("JavaScript"); // true
frase.startsWith("Aprender"); // true
frase.endsWith("práctica"); // true
frase.indexOf("JavaScript"); // 9
frase.lastIndexOf("a"); // última posición o -1
```

Estas operaciones son preferibles a una expresión regular cuando la búsqueda es exacta. Comunican mejor la intención y no introducen metacaracteres.

Transformar y normalizar la forma

```
" hola ".trim();
"hola".toUpperCase();
"HOLA".toLowerCase();
"7".padStart(3, "0"); // "007"
"x".repeat(4); // "xxxx"
```

`trimStart` y `trimEnd` actúan sobre un solo extremo. Los cambios de mayúsculas pueden depender del idioma:

```
"istanbul".toLocaleUpperCase("tr");
```

No uses `toLowerCase` como sustituto automático de todas las reglas de identidad. Nombres de usuario, rutas, correos y textos humanos tienen contratos diferentes.

Reemplazar y separar

```
"uno dos uno".replace("uno", "1"); // "1 dos uno"
"uno dos uno".replaceAll("uno", "1"); // "1 dos 1"
"a,b,c".split(","); // ["a", "b", "c"]
```

`replace` acepta strings o expresiones regulares y también una función de reemplazo. `split` no es un parser de formatos complejos: un CSV con comillas y comas internas necesita reglas adicionales o una biblioteca.

Para unir un array:

```
["Ana", "Luis", "Sofía"].join(" | ");
```

Concatenación y coerción

El operador `+` suma números o concatena si aparece una cadena:

```
1 + 2;          // 3
"1" + 2;        // "12"
1 + 2 + "3";    // "33"
"1" + 2 + 3;    // "123"
```

La evaluación ocurre de izquierda a derecha. Para mensajes, una plantilla evita depender de esa regla:

```
const mensaje = `Total: ${1 + 2}`;
```

La conversión explícita es `String(valor)`:

```
String(42);      // "42"
String(true);    // "true"
String(null);    // "null"
String(undefined); // "undefined"
```

`valor.toString()` requiere que el valor no sea `null` ni `undefined`. `String` es más seguro para una conversión general.

Tres unidades diferentes de texto

Consideremos:

```
const emoji = "😄";
```

Visualmente es un símbolo, Unicode le asigna un punto de código y UTF-16 lo almacena con dos unidades sustitutas:

```
emoji.length;      // 2 unidades UTF-16
[...emoji].length; // 1 punto de código
```

Ahora consideremos una `á` formada por `a` y un acento combinante:

```
const combinada = "a\u0301";  
[...combinada].length; // 2 puntos de código
```

Una persona ve una sola letra. Por lo tanto:

unidad UTF-16 $\neq$ punto de código $\neq$ grafema visible

Un emoji familiar también reúne varios puntos de código mediante selectores y uniones:

```
const familia = "👨👩";  
familia.length > 1; // true  
[...familia].length > 1; // true
```

Recorrer puntos de código

Un bucle por índice puede separar pares sustitutos:

```
for (let i = 0; i < "A😄B".length; i += 1) {  
  console.log("A😄B"[i]);  
}
```

`for...of` usa el iterador de strings y conserva puntos de código:

```
for (const character of "A😄B") {  
  console.log(character); // A, 😄, B  
}
```

El spread y `Array.from` aplican el mismo protocolo:

```
[..."A😄B"]; // ["A", "😄", "B"]  
Array.from("A😄B"); // ["A", "😄", "B"]
```

Recorrer grafemas con `Intl.Segmenter`

Cuando la unidad es lo que percibe la persona —por ejemplo, limitar un nombre visible—, usá segmentación de grafemas:


```
const segmentador = new Intl.Segmenter("es", {
  granularity: "grapheme"
});

function grafemas(texto) {
  return [...segmentador.segment(texto)]
    .map(parte => parte.segment);
}

grafemas("a\u0301").length; // 2
```

`Intl.Segmenter` también puede segmentar palabras y oraciones según reglas lingüísticas. Su disponibilidad depende del entorno y de los datos internacionales incluidos.

Normalización Unicode

Dos cadenas pueden verse iguales y contener secuencias distintas:

```
const compuesta = "á";
const descompuesta = "a\u0301";

compuesta === descompuesta; // false
```

Normalizar las lleva a una forma acordada:

```
compuesta.normalize("NFC") === descompuesta.normalize("NFC"); // true
```

Formas principales:

- `NFC`: composición canónica, frecuente para almacenar y comparar;
- `NFD`: descomposición canónica;
- `NFKC` y `NFKD`: incluyen equivalencias de compatibilidad y pueden cambiar distinciones visuales significativas.

Normalizar es una decisión del dominio. No elimina automáticamente acentos ni resuelve todas las equivalencias de identidad.

Comparación técnica y comparación humana

La igualdad estricta compara secuencias exactas de unidades:

```
"Ana" === "ana"; // false
```

Los operadores `<` y `>` producen un orden lexicográfico por unidades de código:

```
["2", "10"].toSorted(); // ["10", "2"]
```

Para ordenar números almacenados como texto, convertí o proporcioná un comparador numérico:

```
["2", "10"].toSorted((a, b) => Number(a) - Number(b));
```

Para texto humano:

```
const collator = new Intl.Collator("es", {  
  sensitivity: "base",  
  numeric: true  
});  
  
const ordenados = nombres.toSorted(collator.compare);
```

`sensitivity: "base"` puede ignorar diferencias de mayúsculas y acentos para comparar, según el idioma. No transforma el texto almacenado.

Tagged templates

Una función puede recibir las partes literales y los valores interpolados:

```
function inspeccionar(partes, ...valores) {  
  return { partes, valores };  
}  
  
const usuario = "Ana";  
const resultado = inspeccionar`Hola ${usuario}, tenés ${3} mensajes`;
```

Las plantillas etiquetadas permiten construir DSL, sanitizar salidas o parametrizar consultas. La seguridad depende de que la función trate cada contexto correctamente; no alcanza con “escapar todo” de una sola manera.

`String.raw` devuelve las barras como fueron escritas:

```
String.raw`C:\usuarios\ana`; // "C:\\usuarios\\ana" como contenido visible escapado
```

Es útil para ejemplos de regex y rutas, aunque `node:path` sigue siendo la herramienta adecuada para construir rutas reales.

Errores frecuentes

- usar `length` como cantidad universal de caracteres visibles;
- cortar por índice y dejar un emoji o acento partido;
- ordenar texto humano con `<` o `sort()` sin comparador;
- convertir a minúsculas sin decidir idioma y reglas de identidad;
- concatenar números y strings esperando suma;
- intentar interpretar CSV, HTML o lenguajes anidados con operaciones de texto demasiado simples;
- normalizar o quitar acentos sin conservar el original cuando tiene valor.

Para recordar

- Los strings son inmutables y sus índices trabajan con UTF-16.
- `for...of` recorre puntos de código; `Intl.Segmenter` puede recorrer grafemas.
- Normalización, cambio de mayúsculas y comparación lingüística resuelven problemas diferentes.
- Las plantillas literales expresan interpolación; las tagged templates permiten crear protocolos de procesamiento.
- Al procesar texto, primero definí qué unidad y qué equivalencia necesita el usuario.

CAPÍTULO 6

undefined, null y la ausencia

Idea central

`undefined` suele indicar que JavaScript o una operación todavía no produjo un valor; `null` suele expresar que la aplicación decidió representar una ausencia. Ninguno explica por sí solo por qué falta el dato: un programa productivo define el significado, lo valida en los límites y evita propagar estados ambiguos.

La ausencia aparece en propiedades opcionales, búsquedas sin resultado, parámetros omitidos, formularios incompletos y datos externos. Tratarla como parte del contrato es más seguro que esperar a que una operación falle lejos del origen.

`undefined` : falta de valor asignado o producido

JavaScript utiliza `undefined` en varias situaciones:

```
let pendiente;
pendiente; // undefined

const persona = {};
persona.telefono; // undefined

function saludar(nombre) {
  return nombre;
}

saludar(); // undefined
```

Una función sin `return` también devuelve `undefined`:

```
function registrar(mensaje) {
  console.log(mensaje);
}

const resultado = registrar("inicio");
// resultado es undefined
```

Esto distingue un efecto de un resultado reutilizable. Si una función promete producir un valor, todos sus caminos deberían respetar ese contrato o señalar claramente la ausencia.

`null`: ausencia intencional

`null` se escribe explícitamente:

```
const usuarioSeleccionado = null;
```

Puede significar "todavía no se seleccionó", "la relación fue eliminada" o "la búsqueda concluyó y no encontró nada". El significado pertenece a la aplicación.

```
function buscarPorLegajo(alumnos, legajo) {  
  return alumnos.find(alumno => alumno.legajo === legajo) ?? null;  
}
```

Aquí `null` no representa un error: es un resultado posible y documentado.

Una peculiaridad histórica de `typeof`

```
typeof undefined; // "undefined"  
typeof null;      // "object"
```

El resultado de `typeof null` es una incompatibilidad histórica del lenguaje. No significa que `null` sea un objeto utilizable. Para detectarlo:

```
valor === null;
```

Para `undefined`:

```
valor === undefined;  
typeof valor === "undefined";
```

La forma con `typeof` también puede consultar un identificador no declarado sin lanzar `ReferenceError`, aunque no debería usarse para ocultar dependencias globales.

Ausencia no es falsedad

`null` y `undefined` son falsy, pero comparten esa condición con valores válidos como `0`, `false` y `""`:

```
if (!cantidad) {  
  // entra tanto con ausencia como con cero  
}
```

Si cero es válido, comprobá el estado que realmente importa:

```
if (cantidad === null || cantidad === undefined) {  
  // falta la cantidad  
}
```

La forma `valor == null` detecta exactamente `null` o `undefined` mediante una regla particular de igualdad flexible:

```
null == undefined; // true  
0 == null;          // false  
"" == null;         // false
```

Puede ser un modismo útil en equipos que lo acuerdan, pero la versión explícita enseña mejor el contrato.

Coalescencia nula

`??` usa el valor derecho solo ante `null` o `undefined`:

```
const cantidad = entrada.cantidad ?? 1;  
  
0 ?? 1;          // 0  
false ?? true;  // false  
"" ?? "N/D";    // ""  
null ?? "N/D";  // "N/D"
```

Esto contrasta con `||`, que reemplaza cualquier falsy:

```
0 || 1; // 1
```

La asignación nula inicializa solo si falta:

```
config.intentos ??= 3;
```

Encadenamiento opcional

`?.` detiene una cadena de acceso cuando el valor anterior es nulo:

```
const ciudad = usuario?.direccion?.ciudad;
```

Si `usuario` o `direccion` es `null` / `undefined`, el resultado es `undefined`. No detiene ante otros falsy:

```
const longitud = ""?.length; // 0
```

Puede aplicarse a propiedades, índices y llamadas:

```
config.temas?.[0];  
observador?.(evento);
```

Usalo solo cuando la ausencia sea válida. Si una propiedad es obligatoria, el encadenamiento opcional puede ocultar un dato roto y retrasar el error:

```
// Si cliente es obligatorio, esto podría silenciar el problema.  
const nombre = pedido.cliente?.nombre;
```

En ese caso, validá el contrato al ingresar el pedido.

Parámetros predeterminados

Un valor predeterminado de parámetro se aplica ante `undefined`, no ante `null`:

```
function saludar(nombre = "Anónimo") {  
  return `Hola, ${nombre}`;  
}
```

```
saludar();           // "Hola, Anónimo"  
saludar(undefined); // "Hola, Anónimo"  
saludar(null);      // "Hola, null"
```

Si `null` también significa ausencia, normalízalo explícitamente:

```
function saludar(nombre) {  
  const visible = nombre ?? "Anónimo";  
  return `Hola, ${visible}`;
```

```
}
```

Los valores predeterminados en desestructuración siguen la misma regla:

```
const { tema = "claro" } = { tema: undefined }; // "claro"
const { idioma = "es" } = { idioma: null };      // null
```

Propiedad ausente y propiedad con `undefined`

Estos objetos devuelven lo mismo al acceder, pero no tienen la misma estructura:

```
const ausente = {};
const presente = { valor: undefined };

ausente.valor; // undefined
presente.valor; // undefined

Object.hasOwn(ausente, "valor"); // false
Object.hasOwn(presente, "valor"); // true
```

Esto importa al aplicar parches, validar campos o recorrer claves.

`in` también considera propiedades heredadas:

```
"toString" in {}; // true
Object.hasOwn({}, "toString"); // false
```

Huecos en arrays

Un array puede tener posiciones inexistentes:

```
const conHueco = [1, , 3];
const conUndefined = [1, undefined, 3];

conHueco[1];      // undefined
conUndefined[1];  // undefined
1 in conHueco;    // false
1 in conUndefined; // true
```

Algunos métodos saltan huecos, otros los materializan o preservan. Para datos de aplicación, preferí arrays densos y representé la ausencia deliberadamente.

Búsquedas y resultados opcionales

APIs diferentes usan convenciones distintas:

```
[10, 20].find(valor => valor > 50);    // undefined
[10, 20].indexOf(50);                  // -1
new Map().get("clave");                 // undefined
"texto".match(/numero/);               // null
```

No supongas una convención universal. Lee el contrato y convertí el resultado a la forma que usa tu dominio:

```
function buscarConfiguracion(mapa, clave) {
  if (!mapa.has(clave)) return { encontrada: false };
  return { encontrada: true, valor: mapa.get(clave) };
}
```

Este diseño distingue "no existe" de "existe y su valor es `undefined`".

JSON, red y bases de datos

JSON tiene `null`, pero no `undefined`:

```
JSON.stringify({ a: null, b: undefined }); // '{"a":null}'
JSON.stringify([null, undefined]);         // '[null,null]'
```

En objetos, una propiedad `undefined` se omite; en arrays se serializa como `null`. Esto puede cambiar el significado de un parche:

- propiedad ausente: "no modificar";
- propiedad con `null`: "borrar el valor";
- propiedad con dato: "reemplazar".

Cada API debe documentar su convención. Las bases de datos también tienen semánticas propias para `NULL`; no deben trasladarse automáticamente a JavaScript sin decidir el contrato.

Estados explícitos cuando la ausencia no alcanza

Una única variable con `null` puede mezclar demasiados significados:

```
let datos = null;  
// ¿todavía no se cargaron, no existen o falló la carga?
```

Un estado etiquetado conserva la diferencia:

```
let consulta = { estado: "pendiente" };  
  
consulta = { estado: "lista", datos: [] };  
consulta = { estado: "sin-resultados" };  
consulta = { estado: "error", error };
```

Este patrón evita booleanos paralelos contradictorios y permite manejar cada caso de forma exhaustiva.

Validar y normalizar en el borde

```
function normalizarPerfil(entrada) {  
  if (!entrada || typeof entrada !== "object") {  
    throw new TypeError("Se esperaba un perfil");  
  }  
  
  const nombre = entrada.nombre?.trim();  
  if (!nombre) throw new TypeError("El nombre es obligatorio");  
  
  const telefono = entrada.telefono == null  
    ? null  
    : String(entrada.telefono).trim();  
  
  return { nombre, telefono };  
}
```

Después de normalizar, el núcleo de la aplicación puede asumir que `nombre` es no vacío y `telefono` es string o `null`, sin repetir comprobaciones ambiguas.

Errores frecuentes

- usar `if (!valor)` para detectar solo ausencia;
- esperar que un parámetro predeterminado reemplace `null`;
- tratar `typeof null === "object"` como una clasificación útil;
- encadenar `?.` sobre propiedades obligatorias y ocultar datos rotos;
- no distinguir propiedad ausente de propiedad presente con `undefined`;

- serializar `undefined` esperando que JSON lo conserve;
- usar `null` para estados diferentes sin una etiqueta adicional.

Para recordar

- `undefined` suele surgir por falta de asignación o resultado; `null` suele ser una ausencia elegida por la aplicación.
- Ambos son falsy, pero no deben confundirse con `0`, `false` o `""`.
- `??`, `?.` y los valores predeterminados tienen reglas precisas y diferentes.
- Propiedad ausente, propiedad `undefined` y hueco de array no son exactamente lo mismo.
- Cuando existen varios tipos de ausencia, un estado etiquetado expresa mejor el proceso.

CAPÍTULO 7

Conversión y coerción

Idea central

Los datos externos deben convertirse y validarse explícitamente una vez; dentro del programa conviene operar con tipos estables y evitar que la coerción decida reglas del dominio. La coerción automática es parte de JavaScript y puede ser útil, pero resulta segura solo cuando conocemos qué conversión aplica cada contexto.

Conversión explícita y coerción implícita

Una conversión explícita aparece en el código:

```
const cantidad = Number(textoCantidad);  
const etiqueta = String(numeroPedido);  
const presente = Boolean(valor);
```

Una coerción ocurre porque una operación necesita otro tipo:

```
"Total: " + 10; // convierte 10 a string  
"6" - 1;        // convierte "6" a number  
if (valor) {}   // convierte valor a boolean
```

La coerción no es aleatoria. Sigue algoritmos definidos por ECMAScript. El problema práctico aparece cuando el lector o la aplicación esperaban otro significado.

Los bordes del sistema

Formularios, variables de entorno, argumentos de terminal, CSV y muchos atributos HTML llegan como texto. JSON conserva más tipos, pero sigue siendo una entrada no confiable.

```
entrada externa → normalización → conversión → validación → dato interno
```

Ejemplo:

```
function leerPuerto(texto) {
  const limpio = texto.trim();

  if (!/^\d+$/u.test(limpio)) {
    throw new TypeError("El puerto debe contener solo dígitos");
  }

  const puerto = Number(limpio);

  if (!Number.isInteger(puerto) || puerto < 1 || puerto > 65535) {
    throw new RangeError("El puerto debe estar entre 1 y 65535");
  }

  return puerto;
}
```

La regex decide la forma aceptada; `Number` convierte; la condición valida el rango del dominio.

Convertir a string

```
String(42);          // "42"
String(true);        // "true"
String(null);        // "null"
String(undefined);   // "undefined"
String(10n);         // "10"
String(Symbol("x")); // "Symbol(x)"
```

Las plantillas y la concatenación también convierten muchos valores:

```
`${42}`;           // "42"
"valor=" + 42;     // "valor=42"
```

`valor.toString()` no funciona con `null` o `undefined` y puede aceptar una base para números:

```
(255).toString(16); // "ff"
```

Los objetos usan protocolos de conversión que veremos más adelante: `Symbol.toPrimitive`, `valueOf` y `toString`.

Convertir a número

`Number` intenta convertir el valor completo:

```
Number("42");      // 42
Number(" 42 ");    // 42
Number("");        // 0
Number(" ");       // 0
Number("3.14");    // 3.14
Number("3.14px");  // NaN
Number(true);      // 1
Number(false);     // 0
Number(null);      // 0
Number(undefined); // NaN
```

Arrays y objetos producen resultados menos intuitivos debido a la conversión previa a primitivo:

```
Number([]);      // 0, porque [] se convierte en ""
Number([5]);     // 5, porque [5] se convierte en "5"
Number([1, 2]);  // NaN
Number({});      // NaN
```

Estos casos sirven para comprender el lenguaje, no para diseñar validaciones.

El `+` unario aplica conversión numérica:

```
+"5"; // 5
```

No acepta `bigint` y es menos explícito que `Number`.

`parseInt` y `parseFloat`

```
parseInt("42px", 10); // 42
parseInt("101", 2);   // 5
parseFloat("3.5kg");  // 3.5
```

Se detienen al encontrar un carácter inválido. Eso puede ser correcto para un formato deliberado y peligroso para una entrada que debería ser completamente numérica.

```
Number("12abc");      // NaN
parseInt("12abc", 10); // 12
```

Convertir a booleano

`Boolean` devuelve `false` solo para los valores falsy:

```
Boolean(0);           // false
Boolean("");          // false
Boolean(null);        // false
Boolean(undefined);   // false
Boolean(NaN);         // false
Boolean("false");     // true
Boolean([]);          // true
```

No uses `Boolean(texto)` para interpretar palabras como `"sí"`, `"no"`, `"true"` o `"false"`. Eso exige un parser del dominio.

Convertir a `bigint`

```
BigInt("9007199254740993"); // 9007199254740993n
BigInt(42);                  // 42n
BigInt(true);                // 1n
```

No acepta decimales no enteros ni texto decimal con punto:

```
// BigInt(3.5);    // RangeError
// BigInt("3.5");  // SyntaxError
```

Convertir primero un entero grande a `number` puede perder precisión antes de llegar a `BigInt`.
Convertí directamente desde el string.

Contextos de coerción

Una forma productiva de entender el lenguaje es reconocer el tipo que exige cada contexto.

Contexto booleano

```
if (valor) {}
while (valor) {}
valor ? a : b;
!valor;
valor && otro;
valor || otro;
```

Contexto numérico

La mayoría de los operadores aritméticos convierten a número:

```
"8" - "3"; // 5
"8" * "3"; // 24
"8" / 2;    // 4
"8" ** 2;   // 64
```

Si aparece un `bigint`, ambos operandos deben terminar siendo `bigint` para la aritmética correspondiente.

Contexto de string

Plantillas, `String` y concatenación cuando `+` elige texto:

```
`id:${123}`; // "id:123"
```

Contexto de propiedad

Las claves comunes de objeto se convierten en strings; los símbolos permanecen símbolos:

```
const objeto = {};
objeto[10] = "diez";
Object.keys(objeto); // ["10"]
```

La regla especial de `+`

`+` puede sumar o concatenar. De forma simplificada:

1. convierte objetos a primitivos;
2. si alguno de los primitivos es string, concatena como strings;
3. en caso contrario, realiza suma numérica compatible.

```
1 + 2;      // 3
"1" + 2;    // "12"
1 + "2";    // "12"
1 + 2 + "3"; // "33"
"1" + 2 + 3; // "123"
```

La evaluación es de izquierda a derecha. Los paréntesis cambian el primer resultado:


```
"Total: " + (1 + 2); // "Total: 3"
```

`Symbol` no se concatena implícitamente:

```
const id = Symbol("id");  
String(id); // "Symbol(id)"  
// "id=" + id; // TypeError
```

Comparaciones relacionales

Con dos strings, `<` compara lexicográficamente:

```
"20" < "3"; // true
```

Con tipos diferentes, normalmente hay conversión numérica:

```
"20" < 3; // false
```

Los resultados con `NaN` son falsos:

```
NaN < 3; // false  
NaN >= 3; // false
```

Convertí y validá antes de ordenar valores que llegan como strings.

Igualdad flexible: por qué sorprende

`==` intenta acercar tipos mediante reglas específicas:

```
0 == false; // true  
"" == false; // true  
"0" == false; // true  
null == undefined; // true  
[] == ""; // true  
[0] == 0; // true
```

Cada resultado puede explicarse, pero obliga a reconstruir varias conversiones. La igualdad estricta evita esa fase:

```
0 === false; // false
"0" === false; // false
```

Usá `===` y `!==` por defecto. Si elegís `==` para un caso específico, documentá la intención y mantené la expresión local.

Conversión de objetos a primitivos

Cuando una operación necesita un primitivo, un objeto puede participar mediante:

1. `Symbol.toPrimitive`, si existe;
2. `valueOf` y `toString`, en un orden que depende de la sugerencia de tipo.

```
const temperatura = {
  celsius: 25,
  valueOf() {
    return this.celsius;
  },
  toString() {
    return `${this.celsius} °C`;
  }
};

Number(temperatura); // 25
String(temperatura); // "25 °C"
```

Personalizar coerción puede hacer una API elegante o misteriosa. Métodos explícitos como `aCentavos()` y `formatear()` suelen ser mejores cuando existen varias interpretaciones legítimas.

Diseñar parsers del dominio

Una conversión del lenguaje no decide formatos culturales:

```
Number("1,5"); // NaN
```

Si la aplicación acepta coma decimal, debe normalizar bajo un contrato claro y rechazar mezclas ambiguas:

```
function leerDecimalEs(texto) {
  const limpio = texto.trim();
```

```
if (!/^[+-]?\d+(?:,\d+)?$/u.test(limpio)) {
  throw new TypeError("Formato decimal inválido");
}

const numero = Number(limpio.replace(",", "."));
if (!Number.isFinite(numero)) {
  throw new RangeError("Número fuera de rango");
}

return numero;
}
```

El separador de miles requiere aún más cuidado: "1.234" puede significar mil doscientos treinta y cuatro o uno con fracción.

Un pipeline de entrada completo

```
function normalizarProducto(entrada) {
  if (!entrada || typeof entrada !== "object") {
    throw new TypeError("Se esperaba un producto");
  }

  const nombre = String(entrada.nombre ?? "").trim();
  if (nombre === "") throw new TypeError("Falta el nombre");

  const precio = Number(entrada.precio);
  if (!Number.isFinite(precio) || precio < 0) {
    throw new RangeError("Precio inválido");
  }

  const stock = Number(entrada.stock);
  if (!Number.isSafeInteger(stock) || stock < 0) {
    throw new RangeError("Stock inválido");
  }

  const activo = leerBooleano(String(entrada.activo));

  return { nombre, precio, stock, activo };
}

function leerBooleano(texto) {
  const valor = texto.trim().toLowerCase();
  if (valor === "true") return true;
  if (valor === "false") return false;
  throw new TypeError("Activo debe ser true o false");
}
```

Después de este límite, ningún cálculo necesita adivinar tipos.

Errores frecuentes

- convertir una cadena vacía con `Number` y aceptar cero sin querer;
- usar `parseInt` y tolerar basura posterior;
- interpretar `"false"` con `Boolean`;
- comparar números almacenados como strings;
- depender de `+` sin saber si suma o concatena;
- convertir un entero grande a `number` antes de `BigInt`;
- personalizar la coerción de un objeto cuando métodos nombrados serían más claros.

Para recordar

- Convertir cambia la representación; validar comprueba el contrato. Son pasos diferentes.
- La coerción depende del contexto: booleano, numérico, string o clave de propiedad.
- `+` es especial porque puede sumar o concatenar.
- La igualdad flexible añade reglas de conversión; la estricta conserva los tipos.
- Un sistema productivo normaliza entradas una vez y trabaja internamente con datos estables.

CAPÍTULO 8

El tipo symbol y los protocolos

Idea central

Un `symbol` representa una identidad única; usado como clave, evita colisiones, y mediante los símbolos conocidos permite que un objeto participe en protocolos internos del lenguaje.

No es una cadena especial ni un mecanismo de privacidad.

Hay dos usos que conviene separar:

1. símbolos creados por la aplicación para obtener claves únicas;
2. símbolos conocidos definidos por JavaScript para personalizar iteración, coerción y otras operaciones.

Crear una identidad

```
const id = Symbol();  
const idConDescripcion = Symbol("id de alumno");
```

Cada llamada produce un valor diferente:

```
Symbol("id") === Symbol("id"); // false
```

La descripción es información de depuración, no identidad:

```
const clave = Symbol("interno");  
clave.description; // "interno"
```

No se crea con `new`:

```
// new Symbol(); // TypeError
```

`typeof` reconoce el tipo:

```
typeof clave; // "symbol"
```

Símbolo y string son espacios de claves distintos

Un objeto puede tener una propiedad string y otra symbol con descripciones parecidas:

```
const ID = Symbol("id");

const alumno = {
  id: "visible",
  [ID]: 123
};

alumno.id; // "visible"
alumno[ID]; // 123
```

Esto evita que dos módulos elijan accidentalmente el mismo nombre textual:

```
const METADATOS_MODULO_A = Symbol("metadatos");
const METADATOS_MODULO_B = Symbol("metadatos");
```

Aunque compartan descripción, no colisionan.

Enumeración y reflexión

Las claves symbol no aparecen en enumeraciones habituales:

```
Object.keys(alumno); // ["id"]
Object.entries(alumno); // [["id", "visible"]]
JSON.stringify(alumno); // '{"id":"visible"}'
```

Pueden descubrirse:

```
Object.getOwnPropertySymbols(alumno); // [ID]
Reflect.ownKeys(alumno); // ["id", ID]
```

Por eso un símbolo reduce interferencias accidentales, pero no oculta información frente a quien posee el objeto. Para privacidad real dentro del lenguaje usá cierres o campos privados de clase.

El spread y `Object.assign` sí copian propiedades symbol propias y enumerables:

```
const copia = { ...alumno };
copia[ID]; // 123
```

El registro global con `Symbol.for`

`Symbol.for(clave)` consulta un registro y reutiliza la identidad asociada:

```
const uno = Symbol.for("universidad.usuario");
const dos = Symbol.for("universidad.usuario");

uno === dos; // true
```

`Symbol.keyFor` recupera la clave de un símbolo registrado:

```
Symbol.keyFor(uno); // "universidad.usuario"
Symbol.keyFor(Symbol("local")); // undefined
```

Elegí según la coordinación necesaria:

- `Symbol()` para identidad local y aislada;
- `Symbol.for()` para compartir identidad mediante un nombre acordado.

El registro amplía el alcance del acuerdo. Usá prefijos con contexto para evitar que bibliotecas distintas adopten la misma clave por accidente.

Conversión y límites

La conversión explícita a string funciona:

```
const marca = Symbol("marca");
String(marca); // "Symbol(marca)"
marca.toString(); // "Symbol(marca)"
```

La concatenación implícita lanza error:

```
// "clave=" + marca; // TypeError
```

Un símbolo no se convierte a número y no tiene un orden relacional significativo:

```
// Number(marca); // TypeError
```

Estas restricciones protegen su función de identidad.

Símbolos conocidos: acuerdos con el lenguaje

JavaScript expone símbolos estáticos como `Symbol.iterator`. No se crean para una aplicación concreta: son claves compartidas que el motor consulta durante determinadas operaciones.

Un objeto implementa un **protocolo** cuando proporciona la propiedad esperada con el contrato esperado.

`Symbol.iterator`: hacer un objeto iterable

`for...of`, `spread`, desestructuración y constructores como `Array.from` consumen iterables.

El protocolo requiere un método que devuelva un iterador. El iterador tiene `next()`, que devuelve objetos `{ value, done }`.

```
const rango = {
  desde: 3,
  hasta: 5,

  [Symbol.iterator]() {
    let actual = this.desde;
    const fin = this.hasta;

    return {
      next() {
        if (actual <= fin) {
          return { value: actual++, done: false };
        }

        return { value: undefined, done: true };
      }
    };
  }
};

[...rango]; // [3, 4, 5]
```

Un generador implementa el mismo contrato con menos infraestructura:

```
const rangoSimple = {
  desde: 3,
  hasta: 5,

  *[Symbol.iterator]() {
    for (let n = this.desde; n <= this.hasta; n += 1) {
      yield n;
    }
  }
};
```



```
    }  
  }  
};
```

Arrays, strings, `Map`, `Set` y muchos objetos de plataforma ya son iterables. Un objeto común no lo es:

```
// for (const valor of { a: 1 }) {} // TypeError
```

Se puede recorrer `Object.entries(objeto)` porque ese método produce un array iterable.

Un iterable puede ofrecer recorridos diferentes

La elección de lo que se produce forma parte de la API:

```
class ListaDeAlumnos {  
  #alumnos = [];  
  
  agregar(alumno) {  
    this.#alumnos.push(alumno);  
  }  
  
  *[Symbol.iterator]() {  
    yield* this.#alumnos;  
  }  
}
```

Podría producir alumnos, ids o pares. El nombre y la documentación deben hacer previsible el recorrido predeterminado.

`Symbol.asyncIterator` : valores a través del tiempo

Un iterable asíncrono devuelve promesas o usa un generador asíncrono:

```
const paginas = {  
  async *[Symbol.asyncIterator]() {  
    let pagina = 1;  
  
    while (true) {  
      const resultado = await cargarPagina(pagina);  
      if (resultado.items.length === 0) return;  
  
      yield resultado.items;  
      pagina += 1;  
    }  
  }  
}
```

```

    }
  }
};

for await (const items of paginas) {
  procesar(items);
}

```

El protocolo expresa una secuencia cuyos elementos requieren espera, como páginas remotas, archivos por fragmentos o eventos.

Symbol.toPrimitive : decidir una coerción

```

const dinero = {
  centavos: 1250,

  [Symbol.toPrimitive](hint) {
    if (hint === "number") return this.centavos;
    return `$$${(this.centavos / 100).toFixed(2)}`;
  }
};

Number(dinero); // 1250
String(dinero); // "$12.50"

```

El método recibe una sugerencia:

- "number" para contextos numéricos;
- "string" para conversión explícita a string;
- "default" para operaciones como + o igualdad flexible.

Debe devolver un primitivo; devolver un objeto causa `TypeError`.

La coerción implícita debe ser inequívoca. Para dinero, devolver centavos ante `Number` puede sorprender a quien esperaba unidades monetarias. Métodos `aCentavos()` y `formatear()` suelen ser más explícitos.

Symbol.toStringTag : describir una clase de objeto

```

const registro = {
  get [Symbol.toStringTag]() {
    return "RegistroDeAlumnos";
  }
};

```

```
Object.prototype.toString.call(registro);  
// "[object RegistroDeAlumnos]"
```

Muchas APIs integradas usan etiquetas como `Map`, `Set` o `ArrayBuffer`. Es una ayuda descriptiva, no una validación de seguridad.

`Symbol.hasInstance`: personalizar `instanceof`

```
class EnteroPositivo {  
  static [Symbol.hasInstance](valor) {  
    return Number.isInteger(valor) && valor > 0;  
  }  
}
```

```
3 instanceof EnteroPositivo; // true  
-1 instanceof EnteroPositivo; // false
```

Aunque es posible, una función `esEnteroPositivo(valor)` comunica mejor la intención cuando no existe una relación real de instancias.

Símbolos vinculados con expresiones regulares

Los métodos de string consultan protocolos:

- `Symbol.match` para `match`;
- `Symbol.matchAll` para `matchAll`;
- `Symbol.replace` para `replace`;
- `Symbol.search` para `search`;
- `Symbol.split` para `split`.

Esto explica por qué esos métodos aceptan objetos `RegExp` y por qué un objeto especializado podría personalizar la operación.

```
const censor = {  
  [Symbol.replace](texto, reemplazo) {  
    return texto.replaceAll("secreto", reemplazo);  
  }  
};  
  
"dato secreto".replace(censor, "***"); // "dato ***"
```

Es una demostración de protocolo; para una operación concreta, una función nombrada puede ser más directa.

Symbol.isConcatSpreadable

`Array.prototype.concat` normalmente expande arrays y conserva otros objetos como una sola posición. Esta clave puede modificar la decisión:

```
const grupo = {
  0: "a",
  1: "b",
  length: 2,
  [Symbol.isConcatSpreadable]: true
};

[0].concat(grupo); // [0, "a", "b"]
```

El objeto debe tener índices y `length` coherentes con el comportamiento esperado.

Symbol.species

Algunas clases integradas consultan `Symbol.species` para decidir qué constructor utilizar en resultados derivados:

```
class MiArray extends Array {
  static get [Symbol.species]() {
    return Array;
  }
}

const valores = new MiArray(1, 2, 3);
const dobles = valores.map(x => x * 2);

dobles instanceof MiArray; // false
dobles instanceof Array;   // true
```

Es un mecanismo avanzado de interoperabilidad entre subclases y métodos que crean colecciones.

Symbol.unscopables

Controla qué propiedades quedan excluidas dentro de la sentencia histórica `with`. El modo estricto y los módulos prohíben `with`, por lo que este símbolo existe principalmente para compatibilidad del lenguaje. No debería orientar diseños nuevos.

Un ejemplo integrador

```
function crearRegistro() {
  const entradas = new Map();
  const VERSION = Symbol("version interna");

  return {
    [VERSION]: 1,

    guardar(clave, valor) {
      entradas.set(clave, valor);
    },

    obtener(clave) {
      return entradas.get(clave);
    },

    get [Symbol.toStringTag]() {
      return "Registro";
    },

    *[Symbol.iterator]() {
      yield* entradas.entries();
    }
  };
}

const registro = crearRegistro();
registro.guardar("tema", "oscuro");

for (const [clave, valor] of registro) {
  console.log(clave, valor);
}
```

El símbolo local evita colisión, la clausura encapsula el `Map`, la etiqueta mejora la descripción y el iterador integra el objeto con el lenguaje.

Errores frecuentes

- creer que dos símbolos con la misma descripción son iguales;

- tratar la descripción como una clave recuperable;
- usar símbolos como seguridad o privacidad;
- esperar que `Object.keys` o JSON los incluyan;
- concatenarlos implícitamente con strings;
- implementar un protocolo sin respetar exactamente su contrato;
- personalizar coerción o `instanceof` cuando una función nombrada sería más clara.

Para recordar

- `Symbol()` crea identidad única; la descripción no participa de la igualdad.
- `Symbol.for()` coordina identidades mediante un registro compartido.
- Una clave symbol evita colisiones y enumeración ordinaria, pero puede descubrirse.
- Los símbolos conocidos son puntos de extensión que implementan protocolos.
- Aprendé primero `Symbol.iterator`; los demás se entienden como variantes del mismo acuerdo entre objeto y lenguaje.

BLOQUE 2

Parte II. Modelar colecciones y entidades

3 capítulos en este bloque.

CAPÍTULO 9

Arrays y matrices

Idea central

Un array representa una secuencia ordenada y mutable; elegir la operación adecuada depende de si queremos consultar, transformar o cambiar esa secuencia. La productividad aparece cuando distinguimos métodos mutantes de no mutantes, evitamos arrays dispersos y controlamos las referencias compartidas en estructuras anidadas.

Cuándo elegir un array

Un array es adecuado cuando:

- el orden importa;
- cada elemento ocupa una posición;
- se recorrerá la colección completa o por rangos;
- pueden existir valores repetidos;
- las operaciones principales son agregar, filtrar, ordenar o transformar.

```
const notas = [8, 6, 10, 7];
```

Si la operación dominante es encontrar un elemento por una clave estable, un `Map` o un índice auxiliar puede ser más apropiado que recorrer siempre el array.

Crear arrays

El literal es la forma habitual:

```
const vacio = [];  
const lenguajes = ["JavaScript", "C#", "Python"];  
const mezcla = [1, "dos", true, null];
```

JavaScript permite mezclar tipos, pero una colección homogénea suele ser más fácil de procesar y documentar.

`Array.of` crea un array con sus argumentos:


```
Array.of(3);          // [3]
Array.of(1, 2, 3);    // [1, 2, 3]
```

Esto contrasta con el constructor de un solo número:

```
Array(3); // array con longitud 3 y tres huecos
```

`Array.from` convierte iterables o estructuras similares a arrays y puede transformar al mismo tiempo:

```
Array.from("A😊B"); // ["A", "😊", "B"]
Array.from({ length: 5 }, (_, indice) => indice + 1);
// [1, 2, 3, 4, 5]
```

`Array.isArray` distingue arrays de otros objetos:

```
Array.isArray([]); // true
typeof [];         // "object"
```

Índices y `length`

Los índices enteros comienzan en cero:

```
const colores = ["rojo", "verde", "azul"];

colores[0];    // "rojo"
colores[2];    // "azul"
colores[99];   // undefined
colores.length; // 3
```

`at` admite posiciones negativas:

```
colores.at(-1); // "azul"
colores.at(-2); // "verde"
```

Asignar una posición modifica el array:

```
colores[1] = "amarillo";
```

`length` no es solo una propiedad informativa. Reducirla elimina elementos; ampliarla crea huecos:

```
const valores = [1, 2, 3];
valores.length = 1; // [1]
valores.length = 4; // [1, <3 huecos>]
```

No uses esta capacidad como operación cotidiana. Métodos con nombres expresan mejor la intención.

Arrays densos y dispersos

Un hueco es una posición inexistente, diferente de una posición con `undefined`:

```
const disperso = [1, , 3];
const explicito = [1, undefined, 3];

1 in disperso; // false
1 in explicito; // true
```

Algunos métodos como `map` saltan los huecos; `for...of` produce `undefined` para ellos. Estas diferencias dificultan el razonamiento. Preferí arrays densos creados con valores explícitos.

Extraer rangos con `slice`

`slice(inicio, fin)` devuelve un nuevo array y no incluye `fin`:

```
const valores = [10, 20, 30, 40, 50];

valores.slice(1, 4); // [20, 30, 40]
valores.slice(2);    // [30, 40, 50]
valores.slice(-2);   // [40, 50]
valores.slice();     // copia superficial
```

Los elementos no se clonan. Si son objetos, ambas colecciones contienen las mismas referencias.

Agregar y quitar en los extremos

Estos métodos modifican el array:

```
const cola = ["a", "b"];

cola.push("c"); // agrega al final; devuelve nueva longitud
cola.pop();     // quita y devuelve el último
cola.unshift("z"); // agrega al comienzo
cola.shift();   // quita y devuelve el primero
```

`push` y `pop` permiten una pila LIFO. `push` y `shift` permiten una cola FIFO, aunque quitar del comienzo obliga a reindexar elementos y no escala bien para colas enormes.

`splice`: cambiar una región

`splice(inicio, cantidad, ...nuevos)` modifica el original y devuelve lo eliminado:

```
const letras = ["a", "b", "c", "d"];
const eliminadas = letras.splice(1, 2, "x", "y");

letras; // ["a", "x", "y", "d"]
eliminadas; // ["b", "c"]
```

No lo confundas con `slice`. Si querés una variante sin mutación para reemplazar una posición:

```
const indice = 1;
const nuevo = [
  ...letras.slice(0, indice),
  "reemplazo",
  ...letras.slice(indice + 1)
];
```

Buscar y comprobar

```
const numeros = [10, 20, 30, 20];

numeros.includes(20); // true
numeros.indexOf(20); // 1
numeros.lastIndexOf(20); // 3
numeros.find(n => n > 15); // 20
numeros.findIndex(n => n > 15); // 1
numeros.some(n => n < 0); // false
numeros.every(n => n > 0); // true
```

`includes` puede encontrar `NaN`, mientras que `indexOf` no:

```
[NaN].includes(NaN); // true
[NaN].indexOf(NaN); // -1
```

`find` devuelve `undefined` si no hay resultado. Si los elementos pueden ser literalmente `undefined`, usá `findIndex` o un resultado etiquetado para distinguir los casos.

Tres formas fundamentales de recorrer

for tradicional

Permite controlar índice, dirección y paso:

```
for (let indice = 0; indice < numeros.length; indice += 1) {
  console.log(indice, numeros[indice]);
}
```

Es la mejor opción cuando el índice participa en el algoritmo o el recorrido no es lineal.

for...of

Recorre valores:

```
for (const numero of numeros) {
  console.log(numero);
}
```

Para índice y valor:

```
for (const [indice, numero] of numeros.entries()) {
  console.log(indice, numero);
}
```

for...in

Recorre nombres de propiedades enumerables, incluidos los heredados. No es la herramienta para valores de arrays:

```
for (const clave in numeros) {
  console.log(clave); // strings como "0", "1"...
}
```

Una propiedad adicional o una modificación del prototipo puede aparecer en el recorrido. Usá `for...of` o métodos de array.

Transformar con `map`

`map` produce un nuevo array con un resultado por elemento:

```
const dobles = numeros.map(numero => numero * 2);
```

El callback recibe valor, índice y array original:

```
const etiquetas = numeros.map(
  (numero, indice) => `${indice + 1}: ${numero}`
);
```

No uses `map` solo para efectos si descartás su resultado. Para imprimir, enviar o registrar, `for...of` o `forEach` comunica mejor.

Seleccionar con `filter`

```
const aprobados = alumnos.filter(alumno => alumno.nota >= 6);
```

El callback se interpreta como condición. El array resultante conserva referencias a los mismos objetos seleccionados; no los clona.

Acumular con `reduce`

`reduce` combina elementos en un acumulador:

```
const suma = numeros.reduce(
  (acumulado, numero) => acumulado + numero,
  0
);
```

El valor inicial define el tipo y cubre el array vacío. Sin valor inicial, el primer elemento se usa como acumulador y un array vacío lanza `TypeError`.

Puede construir objetos, mapas o agrupaciones, aunque un bucle puede ser más legible si el acumulador tiene muchas reglas:

```
const porEstado = pedidos.reduce((grupos, pedido) => {
  const lista = grupos[pedido.estado] ?? [];

  return {
    ...grupos,
    [pedido.estado]: [...lista, pedido]
  };
}, {});
```

Crear copias en cada iteración puede ser costoso. La inmutabilidad es una herramienta, no una obligación de producir el algoritmo menos eficiente. Un acumulador local mutable que no escapa puede ser claro y seguro.

Efectos con `forEach`

```
alumnos.forEach(alumno => console.log(alumno.nombre));
```

`forEach` siempre devuelve `undefined`, no admite `break` ni `continue`, y no espera automáticamente callbacks asíncronos:

```
// No espera en secuencia:
archivos.forEach(async archivo => {
  await procesar(archivo);
});
```

Para esperar en orden:

```
for (const archivo of archivos) {
  await procesar(archivo);
}
```

Para concurrencia deliberada:

```
await Promise.all(archivos.map(procesar));
```

Aplanar y combinar

```
[[1, 2], [3, 4]].flat();           // [1, 2, 3, 4]
[1, [2, [3]]].flat(2);            // [1, 2, 3]
[1, 2].flatMap(n => [n, n]);       // [1, 1, 2, 2]
[1, 2].concat([3, 4]);            // [1, 2, 3, 4]
```

El spread también combina:

```
const combinado = [...a, ...b];
```

`flat` solo aplanar hasta la profundidad indicada y conserva las referencias de los valores internos.

Ordenar e invertir

`sort` modifica el array y, sin comparador, convierte a string:

```
[2, 10, 1].sort(); // [1, 10, 2]
```

Para números:

```
numeros.sort((a, b) => a - b);
```

El comparador debe devolver un valor negativo, cero o positivo. Para objetos:

```
alumnos.sort((a, b) => a.nota - b.nota);
```

Las variantes no mutantes son `toSorted`, `toReversed` y `toSpliced`:

```
const ordenados = numeros.toSorted((a, b) => a - b);
const invertidos = numeros.toReversed();
```

Para nombres humanos, usá `Intl.Collator`.

Referencias y copias superficiales

```
const original = [{ valor: 1 }];
const alias = original;
const copia = [...original];

alias === original; // true
copia === original; // false
copia[0] === original[0]; // true
```

La copia tiene otra estructura exterior, pero comparte los objetos internos. Para actualizar un elemento sin modificar el original:

```
const actualizado = original.map((item, indice) =>
  indice === 0 ? { ...item, valor: 2 } : item
);
```

Desestructuración

```
const [primero, segundo] = numeros;
const [cabeza, ...cola] = numeros;
const [, omitidoElPrimero, tercero = 0] = numeros;
```

Permite intercambiar variables:

```
let izquierda = "A";
let derecha = "B";
[izquierda, derecha] = [derecha, izquierda];
```

También puede anidarse, pero demasiada profundidad hace frágil el código frente a cambios de forma.

Matrices y filas compartidas

Una matriz es un array de arrays:

```
const matriz = [
  [1, 2, 3],
  [4, 5, 6]
```



```
];  
  
matriz[1][2]; // 6
```

No crees todas las filas con la misma referencia:

```
const incorrecta = Array(3).fill(Array(3).fill(0));  
incorrecta[0][0] = 1;  
// cambia la primera columna de todas las filas
```

Creá una fila por iteración:

```
const correcta = Array.from(  
  { length: 3 },  
  () => Array(3).fill(0)  
);
```

Recorrido:

```
for (let fila = 0; fila < correcta.length; fila += 1) {  
  for (let columna = 0; columna < correcta[fila].length; columna += 1) {  
    console.log(correcta[fila][columna]);  
  }  
}
```

No todas las filas tienen que medir lo mismo; si el algoritmo exige una matriz rectangular, validalo.

Caso integrador: estadísticas de notas

```
function resumirNotas(notas) {  
  if (!Array.isArray(notas) || notas.length === 0) {  
    throw new TypeError("Se necesita al menos una nota");  
  }  
  
  const invalidas = notas.filter(  
    nota => !Number.isFinite(nota) || nota < 0 || nota > 10  
  );  
  
  if (invalidas.length > 0) {  
    throw new RangeError(`Notas inválidas: ${invalidas.join(", ")}`);  
  }  
  
  const suma = notas.reduce((total, nota) => total + nota, 0);
```

```
return {
  cantidad: notas.length,
  promedio: suma / notas.length,
  minima: Math.min(...notas),
  maxima: Math.max(...notas),
  aprobadas: notas.filter(nota => nota >= 6).length,
  ordenadas: notas.toSorted((a, b) => a - b)
};
}
```

La entrada se valida una vez, los resultados tienen nombres y el array original no cambia.

Errores frecuentes

- confundir `slice` con `splice`;
- ordenar números sin comparador;
- usar `for...in` para valores;
- esperar que `spread` clone objetos internos;
- crear matrices con filas compartidas;
- usar `map` para efectos o `forEach` con `await` esperando secuencia;
- omitir el valor inicial de `reduce` sin considerar el array vacío;
- crear huecos mediante índices lejanos o cambios de `length`.

Para recordar

- Un array es una secuencia ordenada; sus métodos expresan consultas, transformaciones o mutaciones.
- `slice`, `map`, `filter` y `toSorted` producen arrays nuevos; muchos otros modifican el original.
- Las copias son superficiales salvo que se copie explícitamente cada nivel necesario.
- Evita arrays dispersos y matrices con referencias de fila compartidas.
- Elegí el recorrido según la tarea: índice, valor, transformación, acumulación o efecto.

CAPÍTULO 10

Objetos, propiedades y referencias

Idea central

Un objeto reúne propiedades para representar una entidad, y las variables que lo contienen guardan referencias a esa entidad. Para usar objetos con seguridad hay que distinguir clave de valor, identidad de contenido y copia superficial de copia profunda.

Cuándo elegir un objeto

Un objeto es una buena representación cuando un dato tiene campos conocidos con significados diferentes:

```
const alumno = {  
  legajo: 12345,  
  nombre: "Ana",  
  regular: true  
};
```

El orden de las propiedades no suele ser la operación principal. Consultamos por nombre, no por posición. Si las claves son datos dinámicos o de cualquier tipo, `Map` puede expresar mejor la colección.

Crear objetos

El literal es la forma habitual:

```
const vacio = {};  
  
const producto = {  
  codigo: "TEC-01",  
  descripcion: "Teclado",  
  precio: 25000  
};
```

Las claves literales se interpretan como strings, salvo las calculadas con símbolos:

```
const ejemplo = {  
  10: "diez",  
  "con espacios": true  
};  
  
Object.keys(ejemplo); // ["10", "con espacios"]
```

Punto y corchetes

La notación de punto requiere un identificador conocido al escribir el programa:

```
producto.precio;  
producto.precio = 26000;
```

Los corchetes evalúan una expresión y permiten claves dinámicas:

```
const campo = "precio";  
producto[campo]; // 26000  
  
producto["con espacios"] = "valor";
```

No confundas:

```
producto.campo; // busca literalmente "campo"  
producto[campo]; // busca el valor de la variable campo
```

Crear, actualizar y eliminar propiedades

```
producto.stock = 10;  
producto.precio = 27000;  
delete producto.stock;
```

`delete` elimina la propiedad, no asigna `undefined`. En objetos con una forma estable, agregar y quitar campos constantemente puede complicar el contrato. A veces conviene conservar una propiedad opcional con `null`, si ese es el modelo acordado.

Existencia de propiedades

```
Object.hasOwn(producto, "precio"); // true
"precio" in producto;               // true
```

`in` considera la cadena de prototipos; `Object.hasOwn` solo las propiedades propias.

Leer una propiedad ausente devuelve `undefined`:

```
producto.categoria; // undefined
```

Para acceso anidado opcional:

```
const ciudad = alumno.direccion?.ciudad ?? "Sin informar";
```

No uses `?.` para ocultar una propiedad obligatoria. Validá la entidad al construirla o recibirla.

Formas abreviadas

Si nombre de variable y propiedad coinciden:

```
const nombre = "Ana";
const edad = 20;
const persona = { nombre, edad };
```

Los métodos tienen sintaxis abreviada:

```
const contador = {
  valor: 0,
  incrementar() {
    this.valor += 1;
    return this.valor;
  }
};
```

Una función flecha no crea su propio `this` y no debe usarse como reemplazo automático de un método:

```
const incorrecto = {
  valor: 1,
  leer: () => this.valor
};
```

```
};
```

Claves calculadas

```
const prefijo = "nota";
const indice = 1;

const registro = {
  [`${prefijo}${indice}`]: 8
};
```

Las claves calculadas son útiles al construir índices o adaptar datos, pero una proliferación de campos dinámicos puede indicar que corresponde un `Map` o una colección anidada.

Recorrer propiedades

```
Object.keys(producto);    // claves string propias y enumerables
Object.values(producto);  // valores
Object.entries(producto); // pares [clave, valor]
```

```
for (const [clave, valor] of Object.entries(producto)) {
  console.log(clave, valor);
}
```

`Reflect.ownKeys` también incluye símbolos y claves no enumerables. La enumerabilidad y los descriptores son mecanismos avanzados; para modelos comunes, los literales producen propiedades propias, enumerables, modificables y configurables.

Orden de propiedades

JavaScript define un orden de enumeración: índices enteros válidos primero en orden numérico, luego strings en orden de inserción y finalmente símbolos en orden de inserción. Aun así, un objeto modela campos, no una secuencia. Si el orden es esencial para la operación, usá un array o `Map`.

Identidad y asignación por referencia

```
const original = { nombre: "Ana" };
const alias = original;

alias.nombre = "Beatriz";
original.nombre; // "Beatriz"
```

Ambas variables contienen una referencia al mismo objeto.

La igualdad compara identidad:

```
({ x: 1 }) === ({ x: 1 }); // false

const a = { x: 1 };
const b = a;
a === b; // true
```

JavaScript no incluye una igualdad profunda general porque "mismo contenido" depende del dominio: ¿importa el orden de arrays?, ¿los prototipos?, ¿fechas?, ¿símbolos?, ¿ciclos?

Copia superficial con spread

```
const copia = { ...original };
copia === original; // false
```

El spread copia propiedades propias y enumerables, incluidas claves symbol, pero solo una capa:

```
const configuracion = {
  tema: "claro",
  usuario: { nombre: "Ana" }
};

const copia = { ...configuracion };
copia.usuario === configuracion.usuario; // true
```

Modificar `copia.usuario.nombre` también afecta al original.

Para una actualización anidada sin mutar:

```
const actualizada = {  
  ...configuracion,  
  usuario: {  
    ...configuracion.usuario,  
    nombre: "Beatriz"  
  }  
};
```

Solo se copian las ramas que cambian; las demás pueden compartirse de manera segura si no se mutan.

Precedencia del spread

Las propiedades posteriores reemplazan anteriores:

```
const base = { tema: "claro", idioma: "es" };  
  
const configuracion = {  
  ...base,  
  tema: "oscuro"  
};
```

El orden inverso perdería el valor personalizado:

```
const incorrecta = {  
  tema: "oscuro",  
  ...base // vuelve a "claro"  
};
```

Object.assign

```
const combinado = Object.assign({}, base, { tema: "oscuro" });
```

El primer argumento es el destino y se modifica. El spread suele ser más legible para crear un objeto nuevo. Ambos realizan copia superficial y activan getters al leer valores del origen.

Copia profunda con `structuredClone`

```
const clon = structuredClone(configuracion);
clon.usuario === configuracion.usuario; // false
```

Puede clonar muchos valores integrados, referencias repetidas y estructuras cíclicas. No clona funciones, símbolos como valores ni todos los objetos de plataforma, y los prototipos personalizados no necesariamente se conservan como espera una clase.

No copies profundamente por rutina. Puede ser costoso y esconder un diseño con demasiado estado compartido. Copiá según el límite que realmente necesite aislamiento.

El truco `JSON.parse(JSON.stringify(objeto))` pierde `undefined`, símbolos, `bigint`, fechas como objetos, `Map`, `Set`, valores no finitos y ciclos. No es un clon general.

Desestructuración

```
const { nombre, edad } = persona;
```

Renombrar una variable local:

```
const { nombre: nombreCompleto } = persona;
```

Valor predeterminado ante `undefined`:

```
const { idioma = "es" } = configuracion;
```

Resto de propiedades:

```
const { id, ...datosEditables } = alumno;
```

Anidada:

```
const {
  direccion: { ciudad }
} = alumno;
```

La desestructuración anidada falla si `direccion` falta. Se puede proporcionar un objeto predeterminado:

```
const { direccion: { ciudad } = {} } = alumno;
```

Pero `ciudad` quedará `undefined`; si es obligatoria, corresponde validar.

Parámetros desestructurados

```
function presentar({ nombre, edad = 0 }) {  
  return `${nombre} tiene ${edad} años`;  
}
```

Para permitir una llamada sin argumento:

```
function configurar({ tema = "claro" } = {}) {  
  return { tema };  
}
```

Esta forma es cómoda para opciones, pero puede esconder que falta una entidad obligatoria. No agregues `= {}` automáticamente a todos los parámetros.

Métodos estáticos útiles

```
Object.fromEntries([  
  ["nombre", "Ana"],  
  ["edad", 20]  
]);  
  
Object.freeze(producto);  
Object.seal(producto);  
Object.preventExtensions(producto);
```

`freeze` impide cambios en las propiedades directas, pero es superficial:

```
const congelado = Object.freeze({ interior: { valor: 1 } });  
congelado.interior.valor = 2; // el objeto interior no está congelado
```

En modo estricto, algunas modificaciones prohibidas lanzan error; en otros contextos pueden fallar silenciosamente. La inmutabilidad por convención y una arquitectura clara siguen siendo necesarias.

JSON como formato, no como objeto JavaScript completo

```
const texto = JSON.stringify(producto);  
const recuperado = JSON.parse(texto);
```

JSON admite objetos, arrays, strings, números finitos, booleanos y `null`. No conserva métodos, prototipos, `undefined`, símbolos, `Map`, `Set`, `bigint` ni referencias compartidas. Al leer, valida de nuevo la estructura.

Caso integrador: actualizar un perfil

```
function actualizarPerfil(perfil, cambios) {  
  if (!Object.hasOwn(cambios, "nombre") &&  
      !Object.hasOwn(cambios, "preferencias")) {  
    return perfil;  
  }  
  
  const actualizado = { ...perfil };  
  
  if (Object.hasOwn(cambios, "nombre")) {  
    const nombre = cambios.nombre.trim();  
    if (!nombre) throw new TypeError("Nombre vacío");  
    actualizado.nombre = nombre;  
  }  
  
  if (Object.hasOwn(cambios, "preferencias")) {  
    actualizado.preferencias = {  
      ...perfil.preferencias,  
      ...cambios.preferencias  
    };  
  }  
  
  return actualizado;  
}
```

La función distingue propiedades ausentes, valida los cambios y copia solamente los niveles modificados.

Errores frecuentes

- creer que asignar copia el objeto;
- comparar contenido con `===`;
- esperar copia profunda de spread u `Object.assign`;

- poner una clave dinámica con punto y leer literalmente otro campo;
- invertir el orden de los spreads;
- desestructurar un camino opcional sin valor predeterminado o validación;
- usar JSON como clon universal;
- crear que `freeze` congela todo el grafo.

Para recordar

- Un objeto representa campos nombrados; una variable guarda una referencia a él.
- Punto sirve para claves conocidas; corchetes, para claves calculadas.
- Identidad no equivale a igualdad de contenido.
- Spread, `Object.assign` y `freeze` actúan superficialmente.
- La actualización inmutable copia el camino modificado y puede compartir el resto.

CAPÍTULO 11

Map, Set y colecciones especializadas

Idea central

Map modela asociaciones dinámicas y **Set** modela pertenencia sin repetidos. Ambos aceptan valores de cualquier tipo, preservan el orden de inserción y ofrecen una API de colección más explícita que un objeto o un array usados fuera de su propósito.

La elección se resume así:

```
entidad con campos conocidos → Object
secuencia ordenada           → Array
clave dinámica → valor       → Map
pertenencia única           → Set
```

Map : pares clave–valor

```
const alumnosPorLegajo = new Map();

alumnosPorLegajo.set(101, { nombre: "Ana" });
alumnosPorLegajo.set(102, { nombre: "Luis" });
```

Operaciones fundamentales:

```
alumnosPorLegajo.get(101); // { nombre: "Ana" }
alumnosPorLegajo.has(103); // false
alumnosPorLegajo.size;     // 2
alumnosPorLegajo.delete(102); // true si existía
alumnosPorLegajo.clear();   // elimina todos
```

set devuelve el propio mapa, por lo que puede encadenarse:

```
const estados = new Map()
  .set("P", "pendiente")
  .set("A", "aprobado");
```

Crear un `Map` desde pares

```
const mapa = new Map([
  ["tema", "oscuro"],
  ["idioma", "es"]
]);
```

Cada elemento exterior debe ser iterable con al menos dos posiciones. `Object.entries` permite convertir un objeto con claves string:

```
const desdeObjeto = new Map(Object.entries({ a: 1, b: 2 }));
```

La conversión inversa funciona si las claves pueden convertirse apropiadamente en propiedades:

```
const objeto = Object.fromEntries(desdeObjeto);
```

Si las claves son objetos, la conversión a objeto común pierde su identidad como claves y no es equivalente.

Claves de cualquier tipo

```
const porObjeto = new Map();
const boton = { id: "guardar" };

porObjeto.set(boton, { clicks: 0 });
porObjeto.get(boton); // funciona con la misma referencia
porObjeto.get({ id: "guardar" }); // undefined
```

La igualdad de claves sigue una comparación de identidad para objetos y una semántica similar a `SameValueZero` para primitivos. `NaN` puede funcionar como clave y `0` / `-0` se consideran la misma.

Ausencia y valores `undefined`

`get` devuelve `undefined` tanto cuando falta la clave como cuando su valor es `undefined`:

```
const mapa = new Map([["presente", undefined]]);

mapa.get("presente"); // undefined
```

```
mapa.get("ausente"); // undefined
```

Usá `has` para distinguir:

```
mapa.has("presente"); // true  
mapa.has("ausente"); // false
```

Recorrer un Map

El iterador predeterminado produce pares en orden de inserción:

```
for (const [clave, valor] of estados) {  
  console.log(clave, valor);  
}
```

También existen:

```
estados.keys();  
estados.values();  
estados.entries();
```

Los tres devuelven iteradores, no arrays. Se pueden materializar:

```
const claves = [...estados.keys()];
```

`forEach` usa el orden `(valor, clave, mapa)`, diferente al orden visual de los pares:

```
estados.forEach((valor, clave) => {  
  console.log(clave, valor);  
});
```

Patrones productivos con Map

Crear un índice

```
const porLegajo = new Map(  
  alumnos.map(alumno => [alumno.legajo, alumno])  
);
```

Una búsqueda pasa de recorrer el array a consultar por clave. El costo de construir el índice se justifica cuando habrá muchas búsquedas o actualizaciones.

Contar frecuencias

```
function contar(valores) {
  const frecuencias = new Map();

  for (const valor of valores) {
    frecuencias.set(valor, (frecuencias.get(valor) ?? 0) + 1);
  }

  return frecuencias;
}
```

Agrupar

```
function agruparPor(valores, obtenerClave) {
  const grupos = new Map();

  for (const valor of valores) {
    const clave = obtenerClave(valor);
    const grupo = grupos.get(clave) ?? [];
    grupo.push(valor);
    grupos.set(clave, grupo);
  }

  return grupos;
}
```

El array local se muta dentro de la función, pero el estado queda encapsulado y se devuelve al final. Este uso controlado puede ser más eficiente que copiar cada grupo en cada iteración.

Actualización inmutable de un Map

```
const actualizado = new Map(original);
actualizado.set(clave, valor);
```

La estructura exterior es nueva, pero claves y valores internos mantienen sus referencias. Igual que con arrays y objetos, se trata de una copia superficial.

Set : valores únicos

```
const etiquetas = new Set(["js", "node", "js"]);

etiquetas.size;          // 2
etiquetas.add("web");
etiquetas.has("node"); // true
etiquetas.delete("js");
etiquetas.clear();
```

`add` devuelve el propio conjunto y puede encadenarse.

Un `Set` mantiene el orden de la primera inserción. Agregar nuevamente un valor no lo mueve.

Eliminar duplicados

```
const unicos = [...new Set([1, 2, 2, 3, 1])];
// [1, 2, 3]
```

Con objetos, la unicidad usa identidad:

```
const a = { id: 1 };
const b = { id: 1 };

new Set([a, b]).size; // 2
new Set([a, a]).size; // 1
```

Para deduplicar por una propiedad, indexá por esa clave:

```
const porId = new Map(items.map(item => [item.id, item]));
const unicosPorId = [...porId.values()];
```

Decidí qué aparición gana. El ejemplo conserva la última para cada id.

Recorrer un Set

```
for (const etiqueta of etiquetas) {
  console.log(etiqueta);
}
```

`values()` y `keys()` son equivalentes por compatibilidad con `Map`. `entries()` produce pares `[valor, valor]`.

Operaciones de conjuntos

Podemos expresar unión, intersección, diferencia y diferencia simétrica de forma portable:

```
function union(a, b) {
  return new Set([...a, ...b]);
}

function interseccion(a, b) {
  return new Set([...a].filter(valor => b.has(valor)));
}

function diferencia(a, b) {
  return new Set([...a].filter(valor => !b.has(valor)));
}

function diferenciaSimetrica(a, b) {
  return union(diferencia(a, b), diferencia(b, a));
}
```

Para elegir qué conjunto recorrer en una intersección grande, conviene usar el más pequeño.

Subconjunto:

```
function esSubconjunto(a, b) {
  return [...a].every(valor => b.has(valor));
}
```

Actualización inmutable de un `Set`

```
const actualizado = new Set(original);
actualizado.add(nuevoValor);
```

La copia conserva referencias a objetos internos. Si los objetos se modifican, ambos conjuntos observan el cambio.

Elegir entre objeto y `Map`

Preferí un objeto cuando:

- representa una entidad con campos conocidos;
- necesitás notación literal y desestructuración;
- el formato natural de intercambio es JSON;
- las claves son nombres de propiedades.

Preferí **Map** cuando:

- las claves nacen de los datos;
- no son solo strings o símbolos;
- consultás **size**, agregás y eliminás con frecuencia;
- querés iteración directa de pares;
- necesitás distinguir claramente la colección de una entidad.

No elijas solo por una afirmación genérica de rendimiento. Medí si el volumen realmente lo exige; la semántica correcta suele ser el criterio principal.

Elegir entre array y **Set**

Preferí array cuando:

- importan posición y duplicados;
- transformás toda la secuencia;
- necesitás índices y rangos.

Preferí **Set** cuando:

- la pregunta principal es pertenencia;
- los duplicados no tienen significado;
- agregás y quitás miembros.

Un **Set** no reemplaza un array para ordenar, acceder por índice o representar varias apariciones.

WeakMap : asociar datos sin retener las claves

WeakMap acepta objetos y símbolos no registrados como claves. No impide que una clave objeto sea recolectada si no queda otra referencia fuerte:

```
const metadatos = new WeakMap();

let elemento = { id: 1 };
metadatos.set(elemento, { seleccionado: true });
metadatos.get(elemento); // { seleccionado: true }
```

```
elemento = null;  
// La entrada podrá desaparecer cuando el recolector lo determine.
```

No es iterable y no tiene `size`, porque exponer sus claves impediría una semántica previsible de recolección. Sirve para metadatos privados, cachés ligados a la vida de objetos y asociaciones que no deben prolongar esa vida.

WeakSet : marcar objetos sin retenerlos

```
const procesados = new WeakSet();  
const tarea = {};  
  
procesados.add(tarea);  
procesados.has(tarea); // true
```

Comparte las limitaciones de no enumeración y claves débiles. Es adecuado para registrar pertenencia de objetos mientras esos objetos existan.

Caso integrador: inscripciones

```
function indexarInscripciones(inscripciones) {  
  const porLegajo = new Map();  
  const cursos = new Set();  
  const duplicados = new Set();  
  
  for (const inscripcion of inscripciones) {  
    const { legajo, curso } = inscripcion;  
  
    if (porLegajo.has(legajo)) duplicados.add(legajo);  
    porLegajo.set(legajo, { ...inscripcion });  
    cursos.add(curso);  
  }  
  
  return {  
    porLegajo,  
    cursos,  
    duplicados,  
    cantidadUnica: porLegajo.size  
  };  
}
```

El contrato decide que la última inscripción reemplaza a las anteriores y conserva una lista de legajos repetidos para informar el problema.

Errores frecuentes

- usar `get` sin `has` cuando `undefined` es un valor posible;
- convertir un `Map` con claves objeto a un objeto común y esperar equivalencia;
- creer que `Set` elimina objetos con contenido igual;
- usar un objeto como diccionario dinámico sin considerar prototipos y tipos de clave;
- usar `Map` para una entidad fija y perder claridad de campos;
- esperar que una copia de `Map` o `Set` clone sus valores;
- intentar enumerar `WeakMap` o depender del momento de recolección.

Para recordar

- `Map` expresa asociaciones dinámicas; `Set`, pertenencia única.
- Ambos preservan inserción y comparan objetos por identidad.
- `has` distingue ausencia de un valor `undefined` almacenado.
- Las conversiones con arrays y objetos son útiles, pero no siempre conservan la semántica de las claves.
- `WeakMap` y `WeakSet` vinculan información con la vida de objetos sin volverla enumerable.

BLOQUE 3

Parte III. Organizar el comportamiento

5 capítulos en este bloque.

CAPÍTULO 12

Estructuras de control

Idea central

Una estructura de control debe mostrar por qué el programa toma un camino y cómo garantiza que una repetición termina. Las guardas mantienen visible el caso normal, cada bucle hace explícito su progreso y `switch` se reserva para seleccionar entre valores discretos.

Del flujo lineal a una decisión

Sin estructuras de control, las instrucciones se ejecutan de arriba hacia abajo:

```
const subtotal = precio * cantidad;
const impuesto = subtotal * 0.21;
const total = subtotal + impuesto;
```

Una condición introduce un desvío:

```
if (cantidad > stock) {
  console.log("No hay stock suficiente");
}
```

JavaScript interpreta la expresión entre paréntesis en contexto booleano. Para reglas importantes, una comparación explícita comunica mejor el motivo que un valor truthy ambiguo.

`if`: ejecutar bajo una condición

```
if (nota >= 6) {
  estado = "aprobado";
}
```

Las llaves son recomendables incluso con una sola sentencia. Evitan errores al agregar una segunda línea y hacen visible el bloque.

Una condición puede tener nombre:

```
const alcanzaAprobacion = nota >= 6;

if (alcanzaAprobacion) {
  estado = "aprobado";
}
```

No crees una variable por cada comparación trivial; hazlo cuando el nombre explica una regla.

else : dos caminos excluyentes

```
if (stock >= cantidad) {
  confirmarPedido();
} else {
  informarFaltante();
}
```

El operador condicional produce un valor:

```
const estado = stock >= cantidad ? "disponible" : "agotado";
```

Es apropiado para una selección breve. Ternarios anidados suelen ser difíciles de leer:

```
const categoria = nota >= 8
  ? "promoción"
  : nota >= 6
    ? "aprobación"
    : "desaprobación";
```

Un `if` con retornos puede expresar mejor esa clasificación.

Encadenar rangos con **else if**

```
function categoria(nota) {
  if (nota >= 8) {
    return "promoción";
  } else if (nota >= 6) {
    return "aprobación";
  } else {
    return "desaprobación";
  }
}
```


Las condiciones se evalúan en orden y solo se ejecuta la primera verdadera. Por eso los rangos deben colocarse de más restrictivo a más general.

Este orden es incorrecto:

```
function categoriaIncorrecta(nota) {
  if (nota >= 6) return "aprobación";
  if (nota >= 8) return "promoción"; // nunca se alcanza para 8 o más
  return "desaprobación";
}
```

Guardas y retorno temprano

Una guarda elimina un caso que impide continuar:

```
function procesarPedido(pedido) {
  if (!pedido) return { ok: false, error: "Pedido ausente" };
  if (pedido.items.length === 0) {
    return { ok: false, error: "Carrito vacío" };
  }
  if (!pedido.cliente) {
    return { ok: false, error: "Falta el cliente" };
  }

  const total = calcularTotal(pedido.items);
  return { ok: true, total };
}
```

Sin guardas, el camino válido quedaría dentro de varios niveles. El retorno temprano reduce el estado mental necesario: después de cada guarda sabemos qué condición ya se cumple.

Una guarda puede devolver un resultado esperado o lanzar un error si se rompió el contrato. El capítulo siguiente desarrolla esa diferencia.

Condiciones anidadas

El anidamiento es útil cuando una decisión solo tiene sentido dentro de otra:

```
if (usuario.estaAutenticado) {
  if (usuario.esAdmin) {
    mostrarPanelAdministrativo();
  }
}
```

Puede combinarse:

```
if (usuario.estaAutenticado && usuario.esAdmin) {  
    mostrarPanelAdministrativo();  
}
```

No toda anidación debe aplanarse. Si las ramas representan pasos jerárquicos con acciones diferentes, conservar la estructura puede ser más claro.

Negar condiciones compuestas

Las leyes de De Morgan ayudan a escribir guardas:

```
!(a && b) = !a || !b  
!(a || b) = !a && !b
```

```
function validarCompra(tieneSaldo, hayStock) {  
    if (!tieneSaldo || !hayStock) {  
        return { ok: false };  
    }  
  
    return { ok: true };  
}
```

equivale a rechazar cuando no se cumple `tieneSaldo && hayStock`.

switch : seleccionar por un mismo valor

```
function etiquetaEstado(estado) {  
    switch (estado) {  
        case "P":  
            return "Pendiente";  
        case "A":  
            return "Aprobado";  
        case "R":  
            return "Rechazado";  
        default:  
            return "Desconocido";  
    }  
}
```

`switch` compara el valor con cada `case` usando igualdad estricta. Es apropiado para códigos, comandos, estados o variantes discretas. Para rangos y condiciones heterogéneas, `if` comunica mejor.

`break`, `return` y *fall-through*

Si un caso no termina con `break`, `return` o `throw`, la ejecución continúa en el siguiente:

```
function tipoDeDia(dia) {  
  switch (dia) {  
    case "sábado":  
    case "domingo":  
      return "fin de semana";  
    default:  
      return "día hábil";  
  }  
}
```

Aquí el *fall-through* agrupa dos entradas. Cuando sea intencional pero no tan evidente, agregá un comentario. El olvido accidental de `break` puede ejecutar lógica de otro caso.

Alcance dentro de `switch`

Los `case` no crean bloques independientes. Dos declaraciones con el mismo nombre pueden colisionar:

```
switch (tipo) {  
  case "usuario": {  
    const resultado = cargarUsuario();  
    usar(resultado);  
    break;  
  }  
  case "curso": {  
    const resultado = cargarCurso();  
    usar(resultado);  
    break;  
  }  
}
```

Las llaves crean un alcance por caso.

Repetir: elegir el bucle por la pregunta

La elección práctica:

- `for...of` : recorrer valores de una colección iterable;
- `for` : controlar índice, rango o paso;
- `while` : repetir hasta que cambie una condición;
- `do...while` : ejecutar primero y decidir después;
- métodos de array: producir una transformación declarativa.

`while` : repetir mientras se cumpla

```
let saldo = 1000;

while (saldo >= 250) {
  saldo -= 250;
}
```

Antes de ejecutar, identifícalo:

1. estado inicial: `saldo = 1000`;
2. condición: `saldo >= 250`;
3. progreso: en cada vuelta baja `250`;
4. estado de salida: `saldo < 250`.

Un bucle infinito suele perder el progreso:

```
let intentos = 0;

while (intentos < 3) {
  ejecutar();
  // falta intentos += 1
}
```

Los bucles infinitos pueden ser deliberados en servidores o consumidores de eventos, pero necesitan un mecanismo externo de cancelación, espera y manejo de fallos.

do...while : al menos una ejecución

```
let entrada;

do {
  entrada = solicitarEntrada();
} while (!esValida(entrada));
```

Es apropiado para menús, reintentos interactivos o procesos donde la primera acción debe ocurrir antes de evaluar. No lo uses si el cuerpo podría ser inválido desde el inicio.

for : inicio, condición y actualización

```
for (let indice = 0; indice < 5; indice += 1) {
  console.log(indice);
}
```

El orden real es:

```
inicialización
→ condición
→ cuerpo
→ actualización
→ condición...
```

Un `while` equivalente:

```
let indice = 0;

while (indice < 5) {
  console.log(indice);
  indice += 1;
}
```

Elegí `for` cuando esas tres piezas pertenecen a la misma idea de recorrido.

Recorrer hacia atrás y con paso

```
for (let i = valores.length - 1; i >= 0; i -= 1) {
  console.log(valores[i]);
}
```

```
for (let par = 0; par <= 10; par += 2) {  
  console.log(par);  
}
```

Los límites y el operador (`<` frente a `<=`) merecen pruebas en el primer y último elemento.

`for...of` : valores de un iterable

```
for (const alumno of alumnos) {  
  console.log(alumno.nombre);  
}
```

Funciona con arrays, strings, `Map`, `Set`, generadores y objetos que implementan `Symbol.iterator`.

Para `Map` :

```
for (const [clave, valor] of mapa) {  
  console.log(clave, valor);  
}
```

Para índices de un array:

```
for (const [indice, alumno] of alumnos.entries()) {  
  console.log(indice, alumno.nombre);  
}
```

`for...in` : nombres de propiedades

```
for (const clave in objeto) {  
  if (Object.hasOwn(objeto, clave)) {  
    console.log(clave, objeto[clave]);  
  }  
}
```

Incluye propiedades enumerables heredadas, por eso suele requerir `Object.hasOwn`. Para objetos comunes, `Object.keys`, `values` o `entries` y `for...of` son más explícitos. No uses `for...in` para valores de arrays.

break : terminar el bucle

```
let encontrado = null;

for (const alumno of alumnos) {
  if (alumno.legajo === buscado) {
    encontrado = alumno;
    break;
  }
}
```

Para este caso concreto, `find` expresa mejor el resultado. `break` es útil cuando el recorrido incluye lógica que no cabe naturalmente en un método de colección.

continue : saltar a la siguiente vuelta

```
for (const linea of lineas) {
  if (linea.trim() === "") continue;
  procesar(linea);
}
```

En un `while`, verificá que `continue` no salte la actualización:

```
let i = 0;

while (i < valores.length) {
  const valor = valores[i];
  i += 1; // progreso antes de cualquier continue

  if (valor == null) continue;
  procesar(valor);
}
```

Bucles anidados y etiquetas

`break` termina el bucle más cercano. Una etiqueta puede señalar uno exterior:

```
buscar:
for (let fila = 0; fila < matriz.length; fila += 1) {
  for (let columna = 0; columna < matriz[fila].length; columna += 1) {
    if (matriz[fila][columna] === objetivo) {
      break buscar;
    }
  }
}
```

```
}  
}  
}
```

Las etiquetas son válidas y poco frecuentes. Muchas veces una función con `return` permite una salida más clara y devuelve el resultado encontrado.

Evitar modificar una colección durante el recorrido

Quitar elementos mientras se avanza puede saltar posiciones:

```
for (let i = 0; i < valores.length; i += 1) {  
  if (valores[i] < 0) valores.splice(i, 1);  
}
```

Después de `splice`, el siguiente elemento ocupa el índice actual, pero `i` avanza. Alternativas:

```
const noNegativos = valores.filter(valor => valor >= 0);
```

o recorrer hacia atrás cuando la mutación sea necesaria.

Caso integrador: procesar un lote

```
function procesarLineas(lineas) {  
  const resultados = [];  
  
  for (const [numero, lineaOriginal] of lineas.entries()) {  
    const linea = lineaOriginal.trim();  
  
    if (linea === "") continue;  
    if (linea === "FIN") break;  
  
    const separador = linea.indexOf(":");  
  
    if (separador === -1) {  
      resultados.push({  
        linea: numero + 1,  
        ok: false,  
        error: "Falta el separador"  
      });  
      continue;  
    }  
  
    resultados.push({
```



```
        linea: numero + 1,
        ok: true,
        clave: linea.slice(0, separador).trim(),
        valor: linea.slice(separador + 1).trim()
    });
}

return resultados;
}
```

El bucle muestra los tres desvíos: ignorar, terminar o procesar. Los resultados negativos esperados se conservan como datos.

Errores frecuentes

- confundir asignación `=` con comparación `===`;
- ordenar mal condiciones que representan rangos;
- anidar todas las reglas en lugar de usar guardas;
- olvidar `break` en un `switch`;
- crear un `while` sin progreso;
- colocar el progreso después de un `continue` posible;
- usar `for...in` para valores de array;
- modificar un array hacia adelante y saltar elementos;
- usar un ternario anidado para lógica con varias decisiones.

Para recordar

- Las guardas eliminan casos inválidos y dejan visible el camino principal.
- `if` expresa condiciones generales; `switch`, variantes discretas de un mismo valor.
- Todo bucle necesita estado inicial, condición, progreso y salida.
- `for...of` recorre valores; `for...in`, propiedades enumerables.
- `break` y `continue` deben simplificar el recorrido, no ocultar su terminación.

CAPÍTULO 13

Errores y excepciones

Idea central

Una excepción representa que una operación no pudo cumplir su contrato; debe capturarse únicamente en la capa que sabe recuperarse, agregar contexto o traducirla. Los resultados negativos esperables se modelan como datos; los fallos excepcionales se propagan sin perder su causa.

Un error cambia el flujo normal

```
const configuracion = JSON.parse(texto);
iniciar(configuracion);
```

Si `JSON.parse` encuentra sintaxis inválida, lanza una excepción. `iniciar` no se ejecuta. La excepción sube por la pila hasta encontrar un `catch` o llegar al entorno de ejecución.

llamada actual → función que llamó → capa superior → entorno

Esta propagación evita que cada función intermedia tenga que comprobar y reenviar manualmente el mismo fallo.

try...catch

```
try {
  const datos = JSON.parse(texto);
  usar(datos);
} catch (error) {
  console.error("No se pudo leer la configuración", error);
}
```

`try` intenta ejecutar el bloque. Si se lanza una excepción síncrona, el control salta al `catch`. Las líneas posteriores al fallo dentro del `try` no se ejecutan.

El parámetro puede omitirse cuando no se necesita:

```
function esJsonValido(texto) {  
  try {  
    JSON.parse(texto);  
    return true;  
  } catch {  
    return false;  
  }  
}
```

No conviertas todo error en `false` si quien llama necesita conocer la causa.

El objeto `Error`

```
const error = new Error("No se pudo guardar el pedido");  
  
error.name;    // "Error"  
error.message; // mensaje  
error.stack;   // pila, dependiente del entorno
```

Lanza objetos `Error`, no strings:

```
throw new Error("Falta el archivo de configuración");
```

Una string lanzada puede capturarse, pero no ofrece una interfaz consistente para nombre, pila y causa.

`throw` : declarar que el contrato no puede cumplirse

```
function dividir(dividendo, divisor) {  
  if (divisor === 0) {  
    throw new RangeError("El divisor no puede ser cero");  
  }  
  
  return dividendo / divisor;  
}
```

En JavaScript numérico, dividir por cero normalmente devuelve `Infinity`. La función decide imponer un contrato más estricto porque su dominio lo necesita.

`throw` acepta cualquier expresión, pero mantener objetos `Error` como convención simplifica el manejo.

Resultado esperado frente a excepción

Buscar un alumno puede no encontrarlo:

```
function buscarAlumno(alumnos, legajo) {  
  return alumnos.find(alumno => alumno.legajo === legajo) ?? null;  
}
```

El resultado `null` es una variante normal. En cambio, recibir un legajo con formato imposible puede romper el contrato:

```
function buscarAlumno(alumnos, legajo) {  
  if (!Number.isSafeInteger(legajo) || legajo <= 0) {  
    throw new TypeError("Legajo inválido");  
  }  
  
  return alumnos.find(alumno => alumno.legajo === legajo) ?? null;  
}
```

Preguntá: ¿quien llama espera decidir con frecuencia sobre este resultado? Si sí, modelalo como dato. ¿La función no puede mantener una garantía que prometía? Una excepción puede ser adecuada.

Tipos integrados de error

Algunos errores frecuentes:

- `TypeError`: valor u operación de tipo incompatible;
- `RangeError`: valor fuera del rango permitido;
- `ReferenceError`: identificador no disponible;
- `SyntaxError`: texto o código con sintaxis inválida;
- `URIError`: codificación o decodificación URI inválida;
- `AggregateError`: varios errores reunidos.

```
try {  
  const datos = JSON.parse(texto);  
} catch (error) {  
  if (error instanceof SyntaxError) {  
    informarJsonInvalido(error.message);  
  } else {  
    throw error;  
  }  
}
```

No dependas solo del texto del mensaje, que puede cambiar o localizarse. Tipos, códigos y propiedades estables son mejores para decisiones.

La pila de llamadas

```
function nivelTres() {  
  throw new Error("fallo");  
}  
  
function nivelDos() {  
  nivelTres();  
}  
  
function nivelUno() {  
  nivelDos();  
}  
  
nivelUno();
```

La pila registra cómo se llegó al punto del error. Capturar y crear un error nuevo sin causa puede perder parte de ese diagnóstico.

Errores personalizados

```
class SaldoInsuficienteError extends Error {  
  constructor({ disponible, requerido }, options) {  
    super("El saldo no alcanza para completar la operación", options);  
    this.name = "SaldoInsuficienteError";  
    this.disponible = disponible;  
    this.requerido = requerido;  
  }  
}
```

```
function debitar(cuenta, importe) {  
  if (cuenta.saldo < importe) {  
    throw new SaldoInsuficienteError({  
      disponible: cuenta.saldo,  
      requerido: importe  
    });  
  }  
  
  return { ...cuenta, saldo: cuenta.saldo - importe };  
}
```

El tipo permite tratar este caso sin analizar una string. Las propiedades ofrecen datos estructurados para interfaz, registro y pruebas.

Conservar la causa

Una capa puede traducir un error técnico a uno de dominio:

```
class ConfiguracionError extends Error {
  constructor(mensaje, options) {
    super(mensaje, options);
    this.name = "ConfiguracionError";
  }
}

function interpretarConfiguracion(texto) {
  try {
    return JSON.parse(texto);
  } catch (cause) {
    throw new ConfiguracionError(
      "El archivo de configuración no es válido",
      { cause }
    );
  }
}
```

`error.cause` mantiene el fallo original. Agregar contexto no debería borrar evidencia.

Capturar solo lo que podemos manejar

Un `try` demasiado amplio no permite saber qué operación produjo el error esperado:

```
try {
  const datos = JSON.parse(texto);
  const normalizados = normalizar(datos);
  await guardar(normalizados);
  notificar(normalizados);
} catch (error) {
  // ¿falló el JSON, la validación, el disco o la notificación?
}
```

Reducí la zona o distinguí errores por tipo y código. Una captura útil realiza al menos una de estas tareas:

- recupera con una alternativa válida;
- agrega contexto y relanza;

- traduce a un error del dominio;
- registra en una frontera del proceso;
- convierte el error en una respuesta externa apropiada.

Capturar y silenciar deja al programa continuar con un estado posiblemente incompleto.

Relanzar

```
try {
  ejecutar();
} catch (error) {
  if (error instanceof ErrorEsperado) {
    recuperar(error);
  } else {
    throw error;
  }
}
```

Usá `throw error`, no `throw new Error(error.message)`, si no necesitás envolverlo; la segunda forma reemplaza identidad y pila.

finally : limpieza garantizada

```
const recurso = await abrirRecurso();

try {
  await usarRecurso(recurso);
} finally {
  await recurso.close();
}
```

`finally` se ejecuta si el `try` termina normalmente, retorna o lanza. También se ejecuta después de un `catch`.

```
function ejemplo() {
  try {
    return "resultado";
  } finally {
    console.log("limpieza");
  }
}
```

No retornes desde `finally`: ese retorno puede reemplazar el resultado o silenciar una excepción.

`try...finally` puede usarse sin `catch` para limpiar y dejar que el error continúe.

Promesas y `async` / `await`

Una promesa rechazada se captura si se espera dentro del `try`:

```
try {
  const datos = await cargarDatos();
  usar(datos);
} catch (error) {
  manejar(error);
}
```

Crear una promesa sin esperarla permite que el `try` termine antes del rechazo:

```
try {
  cargarDatos(); // no se espera ni se devuelve
} catch (error) {
  // no captura un rechazo posterior
}
```

Corregí con `await` o devolvé la promesa a quien deba manejarla.

La forma equivalente con promesas:

```
cargarDatos()
  .then(usar)
  .catch(manejar)
  .finally(limpiar);
```

No mezcles estilos sin una razón clara; una cadena olvidada o un `await` ausente puede producir rechazos no manejados.

La sutileza de `fetch`

`fetch` rechaza ante fallos de red o cancelación, pero normalmente resuelve ante HTTP 404 o 500. Hay que comprobar el estado:


```
async function cargarUsuario(id) {
  const respuesta = await fetch(`/api/usuarios/${id}`);

  if (!respuesta.ok) {
    throw new Error(`HTTP ${respuesta.status}`);
  }

  return respuesta.json();
}
```

Una API de aplicación puede traducir estados:

```
class UsuarioNoEncontradoError extends Error {}

async function cargarUsuario(id) {
  const respuesta = await fetch(`/api/usuarios/${id}`);

  if (respuesta.status === 404) {
    throw new UsuarioNoEncontradoError(`No existe el usuario ${id}`);
  }

  if (!respuesta.ok) {
    throw new Error(`Error del servicio: ${respuesta.status}`);
  }

  return respuesta.json();
}
```

Errores concurrentes

`Promise.all` rechaza con el primer rechazo observado:

```
await Promise.all(tareas.map(ejecutar));
```

Las demás tareas no se cancelan automáticamente. Si necesitas conocer todos los resultados:

```
const resultados = await Promise.allSettled(
  tareas.map(ejecutar)
);
```

Cada elemento indica `fulfilled` con `value` o `rejected` con `reason`. `AggregateError` aparece en APIs como `Promise.any` cuando todas las alternativas fallan.

Mensaje al usuario y diagnóstico técnico

No expongas automáticamente `error.stack`, rutas, consultas o datos internos. Separá:

```
registrarError({
  operacion: "crear-pedido",
  pedidoId,
  error
});

const respuesta = {
  ok: false,
  mensaje: "No pudimos completar el pedido. Intentá nuevamente."
};
```

El registro necesita contexto técnico; la interfaz necesita una acción comprensible y un identificador de incidente cuando corresponda.

No usar excepciones como un `if` sofisticado

Esto oculta una condición esperable:

```
try {
  if (!hayStock) throw new Error("sin stock");
  reservar();
} catch {
  mostrarSinStock();
}
```

Si `sin stock` es una variante normal, modelala directamente:

```
function intentarReserva(hayStock) {
  if (!hayStock) return { ok: false, motivo: "sin-stock" };
  return { ok: true };
}
```

Una excepción puede seguir siendo válida si una función inferior prometía reservar y no pudo cumplir; el diseño depende de la capa y el contrato.

Caso integrador

```
class PedidoInvalidoError extends Error {
  constructor(mensaje, options) {
    super(mensaje, options);
    this.name = "PedidoInvalidoError";
  }
}

async function procesarPedido(texto) {
  let pedido;

  try {
    pedido = JSON.parse(texto);
  } catch (cause) {
    throw new PedidoInvalidoError("JSON inválido", { cause });
  }

  validarPedido(pedido);

  try {
    const respuesta = await enviarPedido(pedido);

    if (!respuesta.ok) {
      throw new Error(`HTTP ${respuesta.status}`);
    }

    return await respuesta.json();
  } catch (cause) {
    throw new Error(`No se pudo enviar el pedido ${pedido.id}`, { cause });
  }
}
```

Cada `try` tiene un propósito concreto y agrega el contexto disponible en esa etapa.

Errores frecuentes

- lanzar strings;
- capturar `Error` y continuar sin recuperación;
- envolver un error sin `cause` y perder diagnóstico;
- usar excepciones para decisiones normales;
- poner demasiadas operaciones dentro de un mismo `try`;
- retornar desde `finally`;
- olvidar `await` o no devolver una promesa;
- suponer que `fetch` rechaza ante todos los estados HTTP;

- mostrar detalles internos al usuario final.

Para recordar

- Una excepción indica que una operación no pudo cumplir su contrato.
- Los casos negativos esperables son datos; los fallos inesperados se propagan.
- Capturá solo donde puedas recuperar, traducir, agregar contexto o registrar.
- Conservá la causa original y usá `finally` para limpieza.
- En asincronía, una promesa solo entra al `try...catch` si se espera o se devuelve correctamente.

CAPÍTULO 14

Funciones

Idea central

Una función convierte una operación en una unidad con nombre, entradas, resultado y contrato. Cuanto más explícitas sean sus dependencias y más acotada su responsabilidad, más fácil será reutilizarla, probarla y combinarla.

Definir no es ejecutar

```
function sumar(a, b) {  
  return a + b;  
}
```

La declaración crea la función. La llamada la ejecuta:

```
const resultado = sumar(2, 3); // 5
```

Sin paréntesis obtenemos el valor función:

```
const operacion = sumar;  
operacion(10, 20); // 30
```

Esta separación permite pasar funciones como datos.

Anatomía de una función

```
function calcularPrecioFinal(precio, descuento) {  
  const rebaja = precio * descuento;  
  return precio - rebaja;  
}
```

- `calcularPrecioFinal`: nombre;
- `precio`, `descuento`: parámetros;
- cuerpo entre llaves: instrucciones;

- `return`: salida y finalización de la llamada.

Un contrato posible:

```
precio finito no negativo + descuento entre 0 y 1  
→ precio final finito no negativo
```

La sintaxis no expresa todo el contrato; nombres, validaciones, tipos, documentación y pruebas lo completan.

Parámetros y argumentos

Los parámetros son nombres locales de la definición; los argumentos son valores de una llamada:

```
function presentar(nombre, edad) {  
  return `${nombre} tiene ${edad} años`;  
}  
  
presentar("Ana", 20);
```

JavaScript no exige la cantidad exacta:

```
presentar("Ana");           // edad es undefined  
presentar("Ana", 20, 99); // el argumento adicional se ignora
```

Que el lenguaje lo permita no significa que el contrato deba aceptarlo.

Declaraciones de función

```
function duplicar(numero) {  
  return numero * 2;  
}
```

La declaración completa se eleva, por lo que puede llamarse antes en el mismo alcance:

```
duplicar(5);  
  
function duplicar(numero) {  
  return numero * 2;  
}
```

Esto permite colocar la función principal antes de detalles auxiliares. No hace falta depender de la elevación si el orden natural ya es claro.

Expresiones de función

```
const duplicar = function (numero) {  
  return numero * 2;  
};
```

La variable sigue las reglas de `const`; no puede usarse antes de la inicialización.

Una expresión puede tener nombre interno:

```
const factorial = function calcular(n) {  
  if (n <= 1) return 1;  
  return n * calcular(n - 1);  
};
```

El nombre mejora pilas de error y permite recursión sin depender del nombre exterior.

Funciones flecha

```
const duplicar = numero => numero * 2;
```

Formas:

```
const constante = () => 42;  
const sumar = (a, b) => a + b;  
const procesar = valor => {  
  const normalizado = Number(valor);  
  return normalizado * 2;  
};
```

Para devolver un objeto literal de forma implícita, hacen falta paréntesis:

```
const crearAlumno = nombre => ({ nombre, activo: true });
```

Sin paréntesis, las llaves se interpretan como cuerpo.

Flechas y funciones tradicionales no son idénticas

Las flechas:

- no crean su propio `this`;
- no tienen `arguments` propio;
- no pueden llamarse con `new`;
- no tienen `prototype` para instancias;
- no pueden ser generadores.

Son excelentes para callbacks que deben conservar el contexto exterior:

```
const temporizador = {
  segundos: 0,
  iniciar() {
    setInterval(() => {
      this.segundos += 1;
    }, 1000);
  }
};
```

La flecha usa el `this` de `iniciar`.

Para un método que recibe `this` del objeto, usá sintaxis de método o función tradicional:

```
const cuenta = {
  saldo: 100,
  depositar(importe) {
    this.saldo += importe;
  }
};
```

`this` depende de la llamada

En una función tradicional, `this` no queda determinado solo por dónde fue escrita:

```
const persona = {
  nombre: "Ana",
  saludar() {
    return `Hola, ${this.nombre}`;
  }
};

persona.saludar(); // this es persona
```


Separar el método puede perder el receptor:

```
const saludar = persona.saludar;  
// saludar(); // this es undefined en modo estricto
```

`bind` crea una función con receptor fijado:

```
const saludarAna = persona.saludar.bind(persona);
```

`call` y `apply` invocan inmediatamente con un `this` elegido:

```
persona.saludar.call({ nombre: "Luis" });
```

No uses `this` cuando parámetros explícitos harían la dependencia más clara.

Valores predeterminados

```
function saludar(nombre, saludo = "Hola") {  
  return `${saludo}, ${nombre}`;  
}
```

El predeterminado se aplica ante argumento omitido o `undefined`, no `null`. Puede depender de parámetros anteriores:

```
function crearRango(inicio, fin = inicio, paso = 1) {  
  // ...  
}
```

La expresión predeterminada se evalúa en cada llamada, lo que evita compartir accidentalmente un array creado allí:

```
function agregar(valor, lista = []) {  
  lista.push(valor);  
  return lista;  
}
```

Cada llamada sin segundo argumento recibe un array nuevo.

Parámetros rest

```
function sumar(...numeros) {  
  return numeros.reduce((total, numero) => total + numero, 0);  
}  
  
sumar(1, 2, 3); // 6
```

El rest debe ser el último parámetro y siempre es un array. Reemplaza muchos usos del objeto histórico `arguments`.

```
function registrar(nivel, ...mensajes) {  
  console.log(nivel, mensajes.join(" "));  
}
```

arguments

Las funciones tradicionales tienen un objeto similar a array:

```
function cantidadDeArgumentos() {  
  return arguments.length;  
}
```

No es un array real, aunque es iterable en entornos modernos. Rest ofrece un contrato visible, funciona en flechas y da directamente un array.

Parámetros desestructurados

```
function calcularTotal({ precio, cantidad = 1, descuento = 0 }) {  
  return precio * cantidad * (1 - descuento);  
}
```

La llamada se vuelve descriptiva:

```
calcularTotal({ precio: 100, descuento: 0.1, cantidad: 2 });
```

También arrays:

```
function distancia([x1, y1], [x2, y2]) {  
  return Math.hypot(x2 - x1, y2 - y1);  
}
```

La desestructuración no sustituye la validación. Una entrada `null` falla antes de entrar al cuerpo.

Pasaje de argumentos: siempre por valor

Con primitivos, la función recibe una copia del valor:

```
function duplicar(numero) {  
  numero *= 2;  
}  
  
let cantidad = 3;  
duplicar(cantidad);  
cantidad; // 3
```

Con objetos, el valor copiado es una referencia. La función puede modificar el mismo objeto:

```
function cumplirAnios(persona) {  
  persona.edad += 1;  
}  
  
const ana = { edad: 20 };  
cumplirAnios(ana);  
ana.edad; // 21
```

Reemplazar la referencia local no reemplaza la exterior:

```
function reemplazar(persona) {  
  persona = { edad: 0 };  
}  
  
reemplazar(ana);  
ana.edad; // sigue 21
```

Decir “los objetos se pasan por referencia” es una simplificación que confunde este último caso. Se pasa por valor una referencia.

return : resultado y salida

```
function absoluto(numero) {  
  if (numero >= 0) return numero;  
  return -numero;  
}
```

Una función sin `return`, o con `return;`, devuelve `undefined`.

`console.log` no reemplaza el retorno:

```
function dobleIncorrecto(n) {  
  console.log(n * 2);  
}  
  
const valor = dobleIncorrecto(3); // undefined
```

El valor mostrado no puede componerse en otro cálculo.

Inserción automática después de return

Un salto inmediatamente después de `return` puede terminar la sentencia:

```
function incorrecta() {  
  return  
  {  
    ok: true  
  };  
}
```

Devuelve `undefined`. La llave del objeto debe comenzar en la misma línea o ir entre paréntesis:

```
function correcta() {  
  return {  
    ok: true  
  };  
}
```

Devolver varios datos

Un objeto ofrece nombres:

```
function analizar(numeros) {  
  return {  
    minimo: Math.min(...numeros),  
    maximo: Math.max(...numeros)  
  };  
}  
  
const { minimo, maximo } = analizar([3, 1, 7]);
```

Un array es adecuado si las posiciones tienen una convención breve y estable:

```
function cocienteYResto(a, b) {  
  return [Math.trunc(a / b), a % b];  
}  
  
const [cociente, resto] = cocienteYResto(10, 3);
```

Para contratos públicos, los nombres suelen evolucionar mejor.

Alcance de función y alcance léxico

```
const tasa = 0.21;  
  
function conImpuesto(precio) {  
  const impuesto = precio * tasa;  
  return precio + impuesto;  
}
```

La función consulta `tasa` en el lugar donde fue definida, no en el lugar desde donde se llama. Esa es la base de las clausuras.

Clausuras

```
function crearMultiplicador(factor) {  
  return numero => numero * factor;  
}  
  
const duplicar = crearMultiplicador(2);  
const triplicar = crearMultiplicador(3);
```

Las funciones devueltas conservan su propio `factor`.

Una clausura puede mantener estado:

```
function crearContador(inicial = 0) {  
  let valor = inicial;  
  
  return {  
    incrementar() {  
      valor += 1;  
      return valor;  
    },  
    leer() {  
      return valor;  
    }  
  };  
}
```

`valor` no es accesible directamente. Cada llamada a `crearContador` crea una variable independiente.

La clausura conserva la variable, no una fotografía:

```
function ejemplo() {  
  let valor = 1;  
  const leer = () => valor;  
  valor = 2;  
  return leer;  
}  
  
ejemplo()(); // 2
```

Clausuras y bucles

`let` crea una vinculación por iteración:

```
const funciones = [];  
  
for (let i = 0; i < 3; i += 1) {  
  funciones.push(() => i);  
}  
  
funciones.map(fn => fn()); // [0, 1, 2]
```

Con `var`, todas compartirían la misma variable final y devolverían `3`. Esta diferencia fue una razón importante para adoptar `let`.

Funciones como valores

```
const operaciones = {
  sumar: (a, b) => a + b,
  restar: (a, b) => a - b
};

operaciones.sumar(3, 2);
```

Pueden almacenarse en arrays, objetos, mapas y pasarse a otras funciones.

Callbacks y funciones de orden superior

Una función de orden superior recibe o devuelve funciones:

```
function aplicar(operacion, a, b) {
  return operacion(a, b);
}

aplicar((a, b) => a * b, 3, 4); // 12
```

El callback puede ejecutarse inmediatamente, varias veces o en el futuro. El contrato debe aclararlo, especialmente si puede ser asíncrono.

Recursividad

```
function factorial(n) {
  if (!Number.isInteger(n) || n < 0) {
    throw new RangeError("n debe ser un entero no negativo");
  }

  if (n <= 1) return 1;
  return n * factorial(n - 1);
}
```

Toda recursión necesita caso base y reducción. Para secuencias lineales grandes, un bucle evita límites de pila. El capítulo 16 la aplica a árboles.

Funciones asíncronas

Una función `async` siempre devuelve una promesa:

```
async function cargarUsuario(id) {
  const respuesta = await fetch(`/api/usuarios/${id}`);
  if (!respuesta.ok) throw new Error(`HTTP ${respuesta.status}`);
  return respuesta.json();
}

const usuario = await cargarUsuario(10);
```

Un `return valor` se convierte en una promesa resuelta; un `throw`, en una rechazada. `await` pausa esa función, no todo el proceso.

Generadores

Una función generadora puede pausar y producir varios valores:

```
function* rango(inicio, fin) {
  for (let valor = inicio; valor <= fin; valor += 1) {
    yield valor;
  }
}

const iterador = rango(1, 3);
iterador.next(); // { value: 1, done: false }
[...rango(1, 3)]; // [1, 2, 3]
```

El cuerpo no se ejecuta al crear el iterador; avanza bajo demanda. Los generadores asíncronos combinan `async function*`, `await` y `yield` y se consumen con `for await...of`.

Diseñar una función productiva

Una buena función:

- tiene un nombre que expresa una acción o cálculo;
- realiza una responsabilidad principal;
- recibe sus dependencias relevantes;
- devuelve datos reutilizables en lugar de solo imprimir;
- no modifica argumentos sin que el contrato lo anuncie;
- valida en el nivel correcto;
- mantiene una interfaz pequeña;
- permite comprobar casos normales, límites y errores.

Ejemplo con dependencia explícita:


```
function crearServicioDeUsuarios({ repositorio, reloj, generarId }) {  
  return {  
    async registrar(datos) {  
      const usuario = {  
        id: generarId(),  
        creadoEn: reloj.ahora(),  
        ...datos  
      };  
  
      await repositorio.guardar(usuario);  
      return usuario;  
    }  
  };  
}
```

Las dependencias pueden sustituirse en pruebas sin depender de variables globales.

Errores frecuentes

- confundir la función con su ejecución;
- olvidar `return`;
- usar una flecha como método esperando un `this` propio;
- mutar un argumento sin avisar;
- depender de demasiados globales;
- crear parámetros opcionales que ocultan errores;
- escribir una función enorme con decisiones y efectos mezclados;
- olvidar `await` o no devolver la promesa;
- usar recursión sin reducción o caso base.

Para recordar

- Definir una función crea un valor; llamarla ejecuta una nueva invocación.
- Declaraciones, expresiones y flechas comparten capacidades, pero difieren en elevación, `this`, `arguments` y construcción.
- JavaScript pasa todos los argumentos por valor; para objetos, ese valor es una referencia.
- Una clausura conserva acceso al alcance léxico y puede encapsular configuración o estado.
- Una función productiva tiene contrato, dependencias explícitas y resultado comprobable.

CAPÍTULO 15

Programación funcional y pipelines

Idea central

La programación funcional organiza una solución como composición de transformaciones, favorece datos inmutables y concentra los efectos en los bordes. En JavaScript no es una obligación de pureza absoluta: es un conjunto de herramientas para reducir estados implícitos y volver comprobable cada paso.

Funciones como datos

La base es que una función puede almacenarse, pasarse y devolverse:

```
const duplicar = numero => numero * 2;

function aplicar(funcion, valor) {
  return funcion(valor);
}

aplicar(duplicar, 5); // 10
```

Una función que recibe o devuelve funciones es de orden superior:

```
function mayorQue(limite) {
  return valor => valor > limite;
}

const mayorQueDiez = mayorQue(10);
```

Las funciones especializadas conservan configuración mediante clausuras.

Pureza: misma entrada, mismo resultado

Una función pura:

1. devuelve el mismo resultado para las mismas entradas;
2. no cambia estado observable fuera de ella.

```
function calcularIva(precio, tasa) {  
  return precio * tasa;  
}
```

Esta función depende de estado externo y produce resultados diferentes con la misma entrada:

```
let tasaActual = 0.21;  
  
function calcularIvaImplicito(precio) {  
  return precio * tasaActual;  
}
```

Pasar la tasa vuelve visible la dependencia.

Otros efectos incluyen:

- modificar argumentos o variables globales;
- escribir archivos o bases de datos;
- enviar una solicitud;
- leer el reloj o generar azar;
- imprimir;
- lanzar una excepción observable.

Los efectos no son malos: una aplicación debe interactuar con el mundo. El objetivo es saber dónde ocurren.

Núcleo funcional, bordes imperativos

```
const texto = await readFile(ruta, "utf8"); // efecto de entrada  
const datos = JSON.parse(texto);           // transformación, puede fallar  
const resumen = resumir(datos);           // núcleo puro  
await writeFile(salida, JSON.stringify(resumen)); // efecto de salida
```

El núcleo puro puede probarse con valores en memoria. Los bordes son responsables de recursos, errores e integración.

Inmutabilidad

En lugar de cambiar un valor compartido, se produce una nueva versión:

```
const alumno = { nombre: "Ana", nota: 7 };

const actualizado = {
  ...alumno,
  nota: 8
};
```

Con arrays:

```
const numeros = [1, 2, 3];
const conCuatro = [...numeros, 4];
const dobles = numeros.map(numero => numero * 2);
```

La inmutabilidad aporta:

- historial de estados más fácil de reconstruir;
- menos interferencia entre consumidores;
- comparación por identidad para detectar cambios;
- pruebas sin preparación y limpieza compartida.

No exige clonar profundamente todo. Se copia el camino modificado y se comparten ramas que nadie mutará.

Mutación local controlada

Esta función es pura desde afuera aunque use un acumulador mutable local:

```
function indexarPorId(items) {
  const indice = new Map();

  for (const item of items) {
    indice.set(item.id, item);
  }

  return indice;
}
```

La pureza observable importa más que prohibir toda asignación interna. Para la misma entrada y sin mutarla, devuelve un mapa equivalente y no cambia estado exterior.

map : una salida por entrada

```
const preciosConIva = precios.map(precio => precio * 1.21);
```

Con objetos:

```
const alumnosConEstado = alumnos.map(alumno => ({
  ...alumno,
  estado: alumno.nota >= 6 ? "aprobado" : "desaprobado"
}));
```

Una implementación didáctica:

```
function map(valores, transformar) {
  const resultado = [];

  for (let i = 0; i < valores.length; i += 1) {
    resultado.push(transformar(valores[i], i, valores));
  }

  return resultado;
}
```

El contrato garantiza orden y cantidad, no que el callback sea puro.

filter : conservar lo que cumple

```
const mayores = alumnos.filter(alumno => alumno.edad >= 18);
```

Implementación:

```
function filter(valores, predicado) {
  const resultado = [];

  for (let i = 0; i < valores.length; i += 1) {
    const valor = valores[i];
    if (predicado(valor, i, valores)) resultado.push(valor);
  }

  return resultado;
}
```

El predicado se interpreta en contexto booleano. Para reglas críticas, devolvé un booleano claro.

reduce : acumular cualquier estructura

Suma:

```
const total = importes.reduce(
  (suma, importe) => suma + importe,
  0
);
```

Conteo por categoría:

```
const conteos = productos.reduce((mapa, producto) => {
  const anterior = mapa.get(producto.categoria) ?? 0;
  mapa.set(producto.categoria, anterior + 1);
  return mapa;
}, new Map());
```

Implementación didáctica:

```
function reduce(valores, combinar, inicial) {
  let acumulador = inicial;

  for (let i = 0; i < valores.length; i += 1) {
    acumulador = combinar(acumulador, valores[i], i, valores);
  }

  return acumulador;
}
```

reduce es poderoso y puede ocultar intención. Si el acumulador tiene muchas mutaciones y bifurcaciones, un bucle y variables nombradas pueden ser más fáciles de mantener.

Consultas especializadas

```
alumnos.find(alumno => alumno.legajo === 10);
alumnos.findIndex(alumno => alumno.legajo === 10);
alumnos.some(alumno => alumno.nota >= 8);
alumnos.every(alumno => alumno.asistencia >= 75);
```

`some` se detiene al primer `true`; `every`, al primer `false`. Son preferibles a reducir booleanos porque expresan la pregunta y permiten cortocircuito.

forEach : efecto, no transformación

```
alumnos.forEach(alumno => registrar(alumno));
```

Usalo cuando el objetivo sea ejecutar un efecto por elemento. Si necesitas un array, `map`; si necesitas control de `break` o `await` secuencial, `for...of`.

flatMap : transformar a cero, uno o varios elementos

```
const materias = alumnos.flatMap(alumno => alumno.materias);
```

Puede filtrar y transformar en un paso:

```
const errores = filas.flatMap((fila, indice) => {  
  const error = validar(fila);  
  return error ? [{ fila: indice + 1, error }] : [];  
});
```

Solo aplana un nivel.

Ordenar sin mutar

```
const porNota = alumnos.toSorted((a, b) => a.nota - b.nota);
```

Para varios criterios:

```
const porCursoYNombre = alumnos.toSorted((a, b) => {  
  const porCurso = a.curso.localeCompare(b.curso, "es");  
  if (porCurso !== 0) return porCurso;  
  return a.nombre.localeCompare(b.nombre, "es");  
});
```

Un comparador debe ser coherente: comparar el mismo par debe mantener el sentido, y elementos equivalentes deben producir cero.

Encadenar un pipeline

```
const totalDeAprobados = alumnos
  .filter(alumno => alumno.nota >= 6)
  .map(alumno => alumno.nota)
  .reduce((suma, nota) => suma + nota, 0);
```

Se lee como una secuencia:

seleccionar → transformar → acumular

Cada etapa debería tener una responsabilidad. Si el callback es complejo, nombralo:

```
const estaAprobado = alumno => alumno.nota >= 6;
const obtenerNota = alumno => alumno.nota;
const sumar = (a, b) => a + b;

const total = alumnos
  .filter(estaAprobado)
  .map(obtenerNota)
  .reduce(sumar, 0);
```

No extraigas una función si el nombre no agrega información y solo obliga a saltar por el archivo.

Composición de funciones

```
const recortar = texto => texto.trim();
const minusculas = texto => texto.toLowerCase();
const quitarEspacios = texto => texto.replaceAll(" ", "-");

const slug = texto => quitarEspacios(minusculas(recortar(texto)));
```

Una utilidad de composición de izquierda a derecha:

```
const pipe = (...funciones) => valor =>
  funciones.reduce((actual, funcion) => funcion(actual), valor);

const crearSlug = pipe(recortar, minusculas, quitarEspacios);
```

Para funciones con errores, asincronía o varios parámetros, una utilidad demasiado genérica puede ocultar el flujo. La composición explícita sigue siendo válida.

Negar y especializar predicados

```
const negar = predicado => valor => !predicado(valor);

const esPar = numero => numero % 2 === 0;
const esImpar = negar(esPar);
```

Combinar:

```
const y = (...predicados) => valor =>
  predicados.every(predicado => predicado(valor));

const esAdulto = persona => persona.edad >= 18;
const estaActivo = persona => persona.activo;
const puedeIngresar = y(esAdulto, estaActivo);
```

Mantén estas abstracciones si el vocabulario del dominio las vuelve legibles.

Equivalencias aproximadas con LINQ

Para quien viene de C#:

LINQ	JavaScript
Where	filter
Select	map
Aggregate	reduce
Any	some
All	every
FirstOrDefault	find con undefined
SelectMany	flatMap
Skip	slice(inicio)
Take	slice(0, cantidad)
Distinct	new Set(valores)
OrderBy	toSorted(comparador)

Ejemplos:

```
const suma = numeros.reduce((a, b) => a + b, 0);

const promedio = numeros.length === 0
  ? null
  : suma / numeros.length;

const pagina = items.slice(desde, desde + cantidad);

const distintos = [...new Set(valores)];
```

Las equivalencias son semánticas, no idénticas. LINQ sobre `IEnumerable` suele ser diferido; los métodos de arrays de JavaScript son inmediatos.

Evaluación inmediata y arrays intermedios

```
const resultado = datos
  .filter(condicion)
  .map(transformacion)
  .slice(0, 10);
```

`filter` recorre y crea un array; `map` vuelve a recorrer y crea otro; `slice` crea un tercero. Para colecciones habituales, el costo puede ser irrelevante y la claridad valiosa.

Si una medición demuestra un problema, un solo bucle evita intermedios:

```
const resultado = [];

for (const dato of datos) {
  if (!condicion(dato)) continue;
  resultado.push(transformacion(dato));
  if (resultado.length === 10) break;
}
```

La versión también deja de recorrer al obtener diez resultados, algo que el pipeline de arrays no logra antes de filtrar y mapear todo.

Evaluación diferida con generadores

```
function* filtrar(iterable, predicado) {
  for (const valor of iterable) {
    if (predicado(valor)) yield valor;
  }
}
```

```

}

function* transformar(iterable, funcion) {
  for (const valor of iterable) {
    yield funcion(valor);
  }
}

function* tomar(iterable, cantidad) {
  if (cantidad <= 0) return;

  let usados = 0;
  for (const valor of iterable) {
    yield valor;
    usados += 1;
    if (usados === cantidad) return;
  }
}

```

```

const pares = filtrar(rango(1, 1_000_000), n => n % 2 === 0);
const cuadrados = transformar(pares, n => n * n);
const primeros = [...tomar(cuadrados, 5)];

```

Solo se producen los valores consumidos. La evaluación diferida agrega complejidad; usala para secuencias grandes, infinitas o costosas, no como ritual.

Callbacks reciben más de un argumento

```
["10", "10", "10"].map(parseInt);
```

Puede producir `[10, NaN, 2]` porque `map` pasa `(valor, indice)` y `parseInt` interpreta el segundo argumento como base.

```
["10", "10", "10"].map(texto => parseInt(texto, 10));
```

No pases una función existente como callback sin comprobar que su firma sea compatible.

Flechas y `return`

```

const correcto = numeros.map(numero => numero * 2);

const tambienCorrecto = numeros.map(numero => {

```

```
    return numero * 2;
  });

const incorrecto = numeros.map(numero => {
  numero * 2;
}); // [undefined, ...]
```

Las llaves eliminan el retorno implícito.

Caso integrador

```
function resumirVentas(ventas) {
  const validas = ventas.filter(venta =>
    Number.isFinite(venta.precio) &&
    Number.isSafeInteger(venta.cantidad) &&
    venta.cantidad > 0
  );

  const detalle = validas.map(venta => ({
    id: venta.id,
    vendedor: venta.vendedor,
    total: venta.precio * venta.cantidad
  }));

  const totalGeneral = detalle.reduce(
    (suma, venta) => suma + venta.total,
    0
  );

  const porVendedor = detalle.reduce((mapa, venta) => {
    mapa.set(
      venta.vendedor,
      (mapa.get(venta.vendedor) ?? 0) + venta.total
    );
    return mapa;
  }, new Map());

  return {
    validas: detalle,
    descartadas: ventas.length - validas.length,
    totalGeneral,
    porVendedor
  };
}
```

Las etapas separan validación, proyección y agregación. La mutación del `Map` es local y no cambia la entrada.

Errores frecuentes

- llamar “pura” a una función que lee reloj, azar o estado global;
- copiar solo el array y luego mutar objetos compartidos;
- usar `reduce` para cualquier problema aunque oculte la intención;
- olvidar `return` en una flecha con llaves;
- pasar callbacks con una firma incompatible, como `parseInt` directo a `map`;
- suponer evaluación diferida en métodos de array;
- construir muchos intermedios en una ruta medida como crítica;
- intentar eliminar todos los efectos en lugar de aislarlos.

Para recordar

- Pureza e inmutabilidad reducen dependencias y cambios invisibles.
- Los efectos son necesarios; concentrarlos en los bordes vuelve comprobable el núcleo.
- `map`, `filter`, `reduce`, `find`, `some` y `every` responden preguntas diferentes.
- Los métodos de array son inmediatos; generadores y otros iterables permiten evaluación diferida.
- Elegí la abstracción más clara y optimizá después de medir.

CAPÍTULO 16

Recursividad y árboles binarios

Idea central

La recursividad resulta natural cuando un problema contiene versiones más pequeñas de sí mismo. Para que sea correcta, cada llamada debe acercarse a un caso base; para que sea productiva, la forma de la función debe reflejar la estructura de los datos y mantener explícitas sus invariantes.

Los árboles son el ejemplo central: cada nodo contiene un valor y referencias a subárboles que obedecen la misma definición.

Qué ocurre en una llamada recursiva

Una función puede llamarse a sí misma:

```
function cuentaRegresiva(numero) {  
  if (numero < 0) return;  
  
  console.log(numero);  
  cuentaRegresiva(numero - 1);  
}
```

Cada llamada crea un marco en la pila con sus parámetros y variables locales. La llamada actual queda suspendida hasta que termine la siguiente.

```
cuentaRegresiva(2)  
  cuentaRegresiva(1)  
    cuentaRegresiva(0)  
      cuentaRegresiva(-1) → termina
```

La pila tiene un límite. Una recursión sin fin o con demasiada profundidad produce un error de tamaño máximo de pila.

Las tres preguntas obligatorias

Antes de escribir una función recursiva:

1. **Caso base:** ¿qué entrada puedo resolver sin otra llamada?
2. **Reducción:** ¿cómo garantizo que la siguiente entrada es más pequeña o más cercana al final?
3. **Combinación:** ¿cómo se integra el resultado pequeño en la respuesta actual?

Factorial:

```
function factorial(n) {  
  if (!Number.isSafeInteger(n) || n < 0) {  
    throw new RangeError("n debe ser un entero no negativo");  
  }  
  
  if (n <= 1) return 1;  
  return n * factorial(n - 1);  
}
```

- caso base: `0!` y `1!` valen `1`;
- reducción: `n - 1`;
- combinación: multiplicar `n` por el resultado menor.

Recursión sobre una secuencia

```
function sumar(numeros, indice = 0) {  
  if (indice === numeros.length) return 0;  
  return numeros[indice] + sumar(numeros, indice + 1);  
}
```

Funciona, pero un bucle es más directo y no consume un marco por elemento:

```
function sumarIterativo(numeros) {  
  let total = 0;  
  
  for (const numero of numeros) {  
    total += numero;  
  }  
  
  return total;  
}
```

La recursividad no es “mejor” por ser más abstracta. Es especialmente útil cuando la estructura de la entrada también es recursiva.

Estructuras anidadas

```
const documento = {
  titulo: "Curso",
  secciones: [
    {
      titulo: "Unidad 1",
      secciones: [
        { titulo: "Tema 1", secciones: [] }
      ]
    }
  ]
};
```

Cada sección contiene secciones. Una función puede aplicar la misma operación en cada nivel:

```
function contarSecciones(seccion) {
  return 1 + seccion.secciones.reduce(
    (total, hija) => total + contarSecciones(hija),
    0
  );
}
```

El array vacío funciona como base implícita: su suma adicional es cero.

Definir un nodo binario

```
function Nodo(valor, menor = null, mayor = null) {
  return { valor, menor, mayor };
}
```

Un nodo tiene como máximo dos hijos. `null` representa un subárbol vacío.

```
const raiz = Nodo(
  20,
  Nodo(15, Nodo(10)),
  Nodo(30, Nodo(25))
);
```

```

  20
 /  \
15   30
```


/
10 25

El invariante de un árbol binario de búsqueda

Para cada nodo:

- todos los valores del subárbol **menor** se comparan antes que el valor del nodo;
- todos los valores del subárbol **mayor** se comparan después;
- ambos subárboles cumplen la misma regla.

Hay que decidir qué hacer con duplicados:

- rechazarlos;
- ubicarlos siempre en una rama;
- almacenar una frecuencia;
- permitir varios registros bajo la misma clave.

Sin una política consistente, buscar, insertar y eliminar dejarán de concordar.

Recorrido inorden

```
function recorrerEnOrden(nodo, resultado = []) {  
  if (nodo === null) return resultado;  
  
  recorrerEnOrden(nodo.menor, resultado);  
  resultado.push(nodo.valor);  
  recorrerEnOrden(nodo.mayor, resultado);  
  
  return resultado;  
}  
  
recorrerEnOrden(raiz); // [10, 15, 20, 25, 30]
```

El caso base es el árbol vacío. El orden es:

subárbol menor → nodo → subárbol mayor

En un árbol de búsqueda válido, produce los valores ordenados.

Preorden y postorden

Preorden procesa el nodo antes de sus hijos:

```
function preorden(nodo, resultado = []) {  
  if (nodo === null) return resultado;  
  
  resultado.push(nodo.valor);  
  preorden(nodo.menor, resultado);  
  preorden(nodo.mayor, resultado);  
  return resultado;  
}
```

Es útil para copiar o serializar conservando la raíz antes de las ramas.

Postorden procesa hijos antes del nodo:

```
function postorden(nodo, resultado = []) {  
  if (nodo === null) return resultado;  
  
  postorden(nodo.menor, resultado);  
  postorden(nodo.mayor, resultado);  
  resultado.push(nodo.valor);  
  return resultado;  
}
```

Sirve cuando el resultado del padre depende de ambos hijos o cuando se liberan estructuras desde las hojas.

Recorridos como generadores

Un generador evita construir un array completo y permite cortar el consumo:

```
function* valoresEnOrden(nodo) {  
  if (nodo === null) return;  
  
  yield* valoresEnOrden(nodo.menor);  
  yield nodo.valor;  
  yield* valoresEnOrden(nodo.mayor);  
}  
  
for (const valor of valoresEnOrden(raiz)) {  
  console.log(valor);  
}
```

La recursión sigue creando marcos por profundidad, pero los valores se producen de manera diferida.

Buscar descartando una rama

```
function compararNumeros(a, b) {
  return a - b;
}

function buscar(nodo, valor, comparar = compararNumeros) {
  if (nodo === null) return null;

  const orden = comparar(valor, nodo.valor);

  if (orden === 0) return nodo.valor;

  return orden < 0
    ? buscar(nodo.menor, valor, comparar)
    : buscar(nodo.mayor, valor, comparar);
}
```

Cada comparación elige una sola rama. En un árbol equilibrado con n nodos, la profundidad típica es proporcional a $\log_2(n)$. En el peor caso puede ser n .

Versión iterativa:

```
function buscarIterativo(raiz, valor, comparar = compararNumeros) {
  let actual = raiz;

  while (actual !== null) {
    const orden = comparar(valor, actual.valor);

    if (orden === 0) return actual.valor;
    actual = orden < 0 ? actual.menor : actual.mayor;
  }

  return null;
}
```

La versión iterativa mantiene profundidad constante en la pila y refleja que solo seguimos un camino.

Insertar modificando el árbol

```
function insertarMutable(nodo, valor, comparar = compararNumeros) {
  if (nodo === null) return Nodo(valor);

  if (comparar(valor, nodo.valor) < 0) {
    nodo.menor = insertarMutable(nodo.menor, valor, comparar);
  } else {
    nodo.mayor = insertarMutable(nodo.mayor, valor, comparar);
  }

  return nodo;
}
```

Es importante reasignar la rama al resultado recursivo: cuando llega a `null`, la llamada devuelve el nuevo nodo.

```
let arbol = null;

for (const valor of [20, 15, 10, 30, 25]) {
  arbol = insertarMutable(arbol, valor);
}
```

Si el árbol estaba vacío, también hay que reasignar la raíz.

Insertar sin modificar la versión anterior

```
function insertar(nodo, valor, comparar = compararNumeros) {
  if (nodo === null) return Nodo(valor);

  if (comparar(valor, nodo.valor) < 0) {
    return {
      ...nodo,
      menor: insertar(nodo.menor, valor, comparar)
    };
  }

  return {
    ...nodo,
    mayor: insertar(nodo.mayor, valor, comparar)
  };
}
```

Solo se copian los nodos del camino. El otro subárbol se comparte porque no cambia:

```
const nuevo = insertar(raiz, 12);

nuevo !== raiz;           // true
nuevo.mayor === raiz.mayor; // true, rama compartida
nuevo.menor !== raiz.menor; // true, camino modificado
```

Este **compartir estructural** permite conservar versiones sin clonar todo el árbol.

Altura y cantidad

```
function cantidad(nodo) {
  if (nodo === null) return 0;
  return 1 + cantidad(nodo.menor) + cantidad(nodo.mayor);
}

function altura(nodo) {
  if (nodo === null) return 0;
  return 1 + Math.max(altura(nodo.menor), altura(nodo.mayor));
}
```

Ambas funciones deben visitar todos los nodos: su costo es lineal en la cantidad de elementos. La profundidad de la recursión depende de la altura.

Mínimo y máximo

El menor valor está en la rama izquierda más profunda:

```
function minimo(nodo) {
  if (nodo === null) return null;

  let actual = nodo;
  while (actual.menor !== null) actual = actual.menor;
  return actual.valor;
}
```

El máximo sigue la rama derecha. Estas operaciones aprovechan el invariante y no necesitan recorrer todo.

Eliminar un valor

La eliminación tiene tres casos:

1. nodo hoja: devolver `null`;
2. un solo hijo: devolver ese hijo;
3. dos hijos: reemplazar por un sucesor y eliminar ese sucesor de su rama original.

```
function eliminar(nodo, valor, comparar = compararNumeros) {
  if (nodo === null) return null;

  const orden = comparar(valor, nodo.valor);

  if (orden < 0) {
    return { ...nodo, menor: eliminar(nodo.menor, valor, comparar) };
  }

  if (orden > 0) {
    return { ...nodo, mayor: eliminar(nodo.mayor, valor, comparar) };
  }

  if (nodo.menor === null) return nodo.mayor;
  if (nodo.mayor === null) return nodo.menor;

  const sucesor = minimo(nodo.mayor);

  return {
    valor: sucesor,
    menor: nodo.menor,
    mayor: eliminar(nodo.mayor, sucesor, comparar)
  };
}
```

Esta versión supone que la política de duplicados y el comparador hacen identificable el sucesor. Para registros con claves no únicas, el contrato debe precisar qué elimina.

Validar el invariante

No alcanza con comparar cada nodo solo con sus hijos. Un valor puede violar un límite heredado desde un ancestro:

```
function esArbolDeBusqueda(
  nodo,
  comparar = compararNumeros,
  minimoPermitido = null,
  maximoPermitido = null
) {
  if (nodo === null) return true;

  if (minimoPermitido !== null &&
    comparar(nodo.valor, minimoPermitido) < 0) {
```

```
    return false;
  }

  if (maximoPermitido !== null &&
      comparar(nodo.valor, maximoPermitido) >= 0) {
    return false;
  }

  return esArbolDeBusqueda(
    nodo.menor,
    comparar,
    minimoPermitido,
    nodo.valor
  ) && esArbolDeBusqueda(
    nodo.mayor,
    comparar,
    nodo.valor,
    maximoPermitido
  );
}
```

La función adopta la política “menores a la izquierda y mayores o iguales a la derecha”. Los límites deben adaptarse a la política real y a claves que puedan coincidir con los centinelas; una implementación genérica puede usar indicadores separados en lugar de `null`.

Comparadores y datos compuestos

```
const compararPorLegajo = (a, b) => a.legajo - b.legajo;

let alumnos = null;
alumnos = insertar(alumnos, { legajo: 20, nombre: "Ana" }, compararPorLegajo);
alumnos = insertar(alumnos, { legajo: 10, nombre: "Luis" }, compararPorLegajo);
```

Buscar requiere un valor comparable:

```
buscar(alumnos, { legajo: 10 }, compararPorLegajo);
```

El comparador debe ser consistente y definir un orden total apropiado. Si dos registros tienen el mismo legajo, el árbol necesita una política de duplicados.

Encapsular el árbol

```
function crearArbol(comparar = compararNumeros) {
  let raiz = null;

  return {
    insertar(valor) {
      raiz = insertar(raiz, valor, comparar);
      return this;
    },

    buscar(valor) {
      return buscar(raiz, valor, comparar);
    },

    eliminar(valor) {
      raiz = eliminar(raiz, valor, comparar);
      return this;
    },

    valores() {
      return [...valoresEnOrden(raiz)];
    },

    get cantidad() {
      return cantidad(raiz);
    }
  };
}
```

La clausura conserva la raíz y garantiza que todas las operaciones usan el mismo comparador.

Equilibrio y complejidad

Insertar valores ya ordenados crea una cadena:

```
10
 \
  20
   \
    30
```

La búsqueda pierde la ventaja y cuesta hasta n comparaciones. Un orden mezclado puede producir una altura cercana a $\log_2(n)$.

Árboles AVL y rojo-negro realizan rotaciones para mantener balance. Implementarlos excede este capítulo, pero dejan una lección: la complejidad prometida depende de mantener no solo el orden, sino también una forma suficientemente equilibrada.

Para muchos programas JavaScript, `Map` es la estructura productiva para búsquedas por clave. Implementar un árbol es valioso para comprender recursividad, invariantes, orden y complejidad; se elige en producción cuando sus capacidades específicas justifican el costo.

Recorridos iterativos con pila

Para árboles de profundidad externa o potencialmente enorme, una pila explícita evita desbordar la pila de llamadas:

```
function inordenIterativo(raiz) {
  const resultado = [];
  const pendientes = [];
  let actual = raiz;

  while (actual !== null || pendientes.length > 0) {
    while (actual !== null) {
      pendientes.push(actual);
      actual = actual.menor;
    }

    actual = pendientes.pop();
    resultado.push(actual.valor);
    actual = actual.mayor;
  }

  return resultado;
}
```

La estructura explícita reproduce los marcos que la recursión guardaba implícitamente.

Depurar una recursión

Registrá entrada, profundidad y caso base:

```
function buscarConTraza(nodo, valor, profundidad = 0) {
  console.log({ profundidad, actual: nodo?.valor ?? null });

  if (nodo === null || nodo.valor === valor) return nodo;

  return valor < nodo.valor
```

```
? buscarConTraza(nodo.menor, valor, profundidad + 1)
: buscarConTraza(nodo.mayor, valor, profundidad + 1);
}
```

Probá:

- estructura vacía;
- un nodo;
- valor en la raíz;
- valor en cada rama;
- valor ausente;
- árbol degenerado;
- duplicados según la política elegida.

Errores frecuentes

- no definir caso base;
- hacer una llamada que no reduce el problema;
- olvidar retornar el resultado recursivo;
- no reasignar una rama después de insertar o eliminar;
- mezclar políticas de duplicados;
- verificar solo padres e hijos y no límites de ancestros;
- asumir búsqueda logarítmica sin controlar equilibrio;
- usar recursión profunda con entrada externa no limitada.

Para recordar

- Recursión correcta significa caso base, reducción y combinación.
- La estructura recursiva de un árbol se refleja en la función que lo recorre.
- Un árbol de búsqueda necesita una política de orden y duplicados compartida por todas sus operaciones.
- Inmutabilidad puede lograrse copiando el camino y compartiendo ramas no modificadas.
- El equilibrio determina si buscar cuesta aproximadamente $\log n$ o se degrada a n .

BLOQUE 4

Parte IV. Aplicar e integrar

3 capítulos en este bloque.

CAPÍTULO 17

Fundamentos de computación digital

Idea central

Una computadora representa información con estados discretos y construye cálculos complejos al componer operaciones lógicas simples. Los bits adquieren significado mediante una codificación; las funciones booleanas transforman entradas en salidas; las puertas y circuitos implementan físicamente esas funciones.

Este modelo conecta el nivel físico con las operaciones bit a bit de un lenguaje y muestra una idea que atraviesa toda la programación:

componentes simples + composición + capas de abstracción
→ sistemas complejos

Dos estados distinguibles

Una computadora física trabaja con señales continuas, pero diseña rangos que interpreta como estados discretos. Podemos nombrarlos:

0 / 1
falso / verdadero
apagado / encendido
bajo / alto

Lo importante no es que exista exactamente cero o cinco voltios, sino que el circuito pueda distinguir con margen dos regiones. Esa separación aporta tolerancia frente a pequeñas variaciones y ruido.

Un **bit** es una unidad capaz de representar una de dos alternativas.

Un bit no tiene significado por sí solo

El patrón **1** podría significar:

- una respuesta afirmativa;
- un píxel encendido;
- que una puerta está abierta;

- el permiso de lectura;
- la cifra uno en un número binario.

La interpretación depende del convenio. Programar siempre incluye diseñar representaciones.

Cantidad de combinaciones

Con un bit existen dos combinaciones. Con dos:

```
00
01
10
11
```

Con n bits existen 2^n patrones. Tres bits ofrecen ocho estados:

```
000 001 010 011 100 101 110 111
```

Un dado necesita seis estados, por lo que tres bits alcanzan:

Cara	Código posible
1	001
2	010
3	011
4	100
5	101
6	110

000 y 111 pueden quedar reservados. Esto ilustra dos decisiones:

1. la cantidad de bits determina la capacidad;
2. el formato decide qué significa cada patrón y qué estados son inválidos.

Cuántos bits hacen falta

Para representar k estados necesitamos el menor n que cumpla:

$$2^n \geq k$$

Ejemplos:

- 2 estados → 1 bit;
- 6 estados → 3 bits;
- 256 estados → 8 bits;
- 1000 estados → 10 bits, porque $2^{10} = 1024$.

No todos los patrones deben utilizarse. Los sobrantes pueden reservarse para errores, extensiones o control.

Del dato al cálculo

Supongamos dos dados codificados con tres bits cada uno. Queremos responder si suman siete. El sistema recibe seis bits y produce uno:

$$f(A, B) \rightarrow \{0, 1\}$$

$1 + 6 \rightarrow 1$
 $2 + 5 \rightarrow 1$
 $3 + 4 \rightarrow 1$
 $1 + 1 \rightarrow 0$
 $2 + 2 \rightarrow 0$

Un cálculo digital puede entenderse como una función que transforma un patrón de entrada en otro de salida. El número de entradas posibles puede ser enorme, pero la idea es la misma.

Funciones booleanas

Cuando cada variable vale 0 o 1, usamos álgebra de Boole.

NOT

Invierte un bit:

A	NOT A
0	1
1	0

Se escribe $\neg A$.

AND

Solo vale uno si ambas entradas valen uno:

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

Se escribe $A \wedge B$.

OR

Vale uno si al menos una entrada vale uno:

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

Se escribe $A \vee B$.

XOR

Vale uno cuando las entradas son diferentes:

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

XOR es útil en suma binaria, paridad, alternancia y cifrados elementales.

Componer operaciones

$(A \text{ AND } B) \text{ OR } (\text{NOT } C)$

equivale a:

$(A \wedge B) \vee \neg C$

La salida de una operación se convierte en entrada de otra. Este principio se repite:

puertas $\rightarrow$ bloques aritméticos $\rightarrow$ procesadores $\rightarrow$ computadoras
funciones $\rightarrow$ módulos $\rightarrow$ aplicaciones $\rightarrow$ sistemas

La abstracción permite usar un bloque por lo que hace sin reconstruir todos sus componentes cada vez.

Tabla de verdad

Una tabla enumera todas las entradas y la salida. Para dos variables tiene cuatro filas:

A	B	f(A, B)
0	0	0
0	1	1
1	0	1
1	1	0

Esta tabla describe XOR completamente. Para n entradas habrá 2^n filas.

Una expresión permite calcular sin almacenar toda la tabla; la tabla permite especificar y verificar la expresión.

Construir una expresión desde una tabla

Supongamos que una función vale uno en estas filas:

A=0, B=1
A=1, B=0

Cada fila verdadera produce un término AND que fija todos los valores:

$$\begin{aligned} &(\text{NOT } A \text{ AND } B) \\ &(A \text{ AND NOT } B) \end{aligned}$$

Unimos los términos con OR:

$$(\neg A \wedge B) \vee (A \wedge \neg B)$$

Esta es una **forma normal disyuntiva**: un OR de términos AND. Permite construir una expresión para cualquier tabla booleana finita, aunque no siempre sea la forma mínima.

Simplificar expresiones

Algunas identidades:

$$\begin{aligned} A \text{ AND } 1 &= A \\ A \text{ AND } 0 &= 0 \\ A \text{ OR } 0 &= A \\ A \text{ OR } 1 &= 1 \\ A \text{ AND } A &= A \\ A \text{ OR } A &= A \\ A \text{ AND NOT } A &= 0 \\ A \text{ OR NOT } A &= 1 \end{aligned}$$

Distributividad:

$$\begin{aligned} A \text{ AND } (B \text{ OR } C) &= (A \text{ AND } B) \text{ OR } (A \text{ AND } C) \\ A \text{ OR } (B \text{ AND } C) &= (A \text{ OR } B) \text{ AND } (A \text{ OR } C) \end{aligned}$$

Leyes de De Morgan:

$$\begin{aligned} \text{NOT } (A \text{ AND } B) &= (\text{NOT } A) \text{ OR } (\text{NOT } B) \\ \text{NOT } (A \text{ OR } B) &= (\text{NOT } A) \text{ AND } (\text{NOT } B) \end{aligned}$$

Estas leyes sirven en circuitos, condiciones de programas y consultas.

NAND y NOR: completitud funcional

NAND es NOT de AND. NOR es NOT de OR.

Con solo puertas NAND puede construirse NOT, AND y OR; por lo tanto, cualquier función booleana. Lo mismo ocurre con NOR.

Por ejemplo, con NAND:

```
NOT A = A NAND A
AND(A, B) = NOT(A NAND B)
```

La **completitud funcional** muestra que una colección mínima de operaciones puede construir todas las demás. En ingeniería, reducir tipos de componentes puede simplificar fabricación, aunque los diseños reales equilibran velocidad, consumo y cantidad de transistores.

Implementación física

Una puerta lógica puede construirse con transistores usados como interruptores controlados. Los detalles eléctricos incluyen tiempos de propagación, consumo, capacidad, ruido y niveles de tensión.

Desde el nivel lógico, abstraemos esos detalles:

```
entrada 0/1 → puerta → salida 0/1
```

La salida no cambia instantáneamente. Todo circuito tiene retardo. En sistemas sincronizados, un reloj coordina cuándo se considera estable el nuevo estado.

Circuitos combinacionales y secuenciales

Un circuito **combinacional** produce salida según la entrada actual:

```
salida = f(entrada actual)
```

Ejemplos: sumadores, comparadores, multiplexores.

Un circuito **secuencial** también depende del estado anterior:

```
salida y próximo estado = f(entrada, estado actual)
```

Registros, contadores y memoria requieren almacenar estado. Esta diferencia se parece a una función pura frente a un objeto o proceso con memoria.

Números binarios posicionales

En decimal, cada posición pesa una potencia de diez. En binario, una potencia de dos:

$$\begin{aligned}
 10110_2 &= 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \\
 &= 16 + 4 + 2 \\
 &= 22_{10}
 \end{aligned}$$

En JavaScript:

```
0b10110; // 22
(22).toString(2); // "10110"
```

Suma binaria

Reglas de un bit:

```
0 + 0 = 0, acarreo 0
0 + 1 = 1, acarreo 0
1 + 0 = 1, acarreo 0
1 + 1 = 0, acarreo 1
```

La suma del bit es XOR y el acarreo es AND.

Medio sumador

Entradas **A** y **B**; salidas **S** y **C**:

```
S = A XOR B
C = A AND B
```

A	B	Suma	Acarreo
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Se llama “medio” porque no recibe un acarreo anterior.

Sumador completo

Para sumar varios bits, cada posición —salvo la primera— recibe `Cin`, el acarreo previo:

```
S = A XOR B XOR Cin
Cout = (A AND B) OR (Cin AND (A XOR B))
```

Al encadenar sumadores completos se construye un sumador de varios bits. El acarreo se propaga de una posición a la siguiente; diseños más avanzados aceleran esa propagación.

Representar enteros con signo

Una representación común es complemento a dos. Con un ancho fijo de `n` bits:

- el bit más significativo participa del signo;
- el rango es -2^{n-1} a $2^{n-1} - 1$;
- negar consiste en invertir y sumar uno.

Con 8 bits:

```
5   = 00000101
-5  = 11111011
```

El ancho fijo es esencial. El patrón no tiene un valor con signo independiente del número de bits.

Los operadores bitwise de `number` en JavaScript trabajan con enteros de 32 bits con signo, salvo `>>>`, que interpreta el desplazamiento sin signo.

Overflow

Con un ancho fijo, una suma puede necesitar un bit adicional. Si se descarta, el resultado “da la vuelta”. Para enteros sin signo de `n` bits, la aritmética se comporta módulo 2^n .

JavaScript `number` no es un entero fijo de 32 bits para la aritmética normal, pero las operaciones bitwise sí convierten a 32 bits. Typed arrays como `Uint8Array` también aplican rangos de ancho fijo al almacenar.

Bytes, palabras y memoria

Ocho bits forman un byte, capaz de 256 patrones. La memoria se organiza en bytes direccionables y los procesadores operan también sobre grupos mayores llamados palabras.

Un mismo conjunto de bytes puede interpretarse como:

- entero;
- número de punto flotante;
- instrucción;
- parte de un texto UTF-8;
- canales de un color;
- muestra de audio.

Otra vez, la representación y el contexto dan significado.

Máscaras de bits en JavaScript

Cada posición representa una opción:

```
const LEER = 1 << 0;    // 0001
const CREAR = 1 << 1;   // 0010
const EDITAR = 1 << 2;  // 0100
const BORRAR = 1 << 3;  // 1000
```

Activar con OR:

```
let permisos = LEER | CREAR;
```

Consultar con AND:

```
function tiene(permisos, permiso) {
  return (permisos & permiso) !== 0;
}
```

Apagar:

```
permisos &= ~CREAR;
```

Alternar con XOR:

```
permisos ^= EDITAR;
```

Presentar:

```
permisos.toString(2).padStart(4, "0");
```

Operadores lógicos y bitwise no son equivalentes

```
5 && 2; // 2, selección por truthiness
5 & 2;  // 0, AND bit a bit

5 || 2; // 5
5 | 2;  // 7
```

`&&` y `||` preservan operandos y cortocircuitan. `&` y `|` convierten a enteros y siempre evalúan ambos lados.

Del circuito al software productivo

Comprender bits ayuda a:

- interpretar codificaciones y formatos binarios;
- diseñar flags y protocolos;
- razonar sobre rangos y overflow;
- entender por qué texto, imagen y números son interpretaciones de bytes;
- reconocer que una interfaz simple puede ocultar muchas capas.

No significa reemplazar automáticamente estructuras legibles por máscaras. La representación de bajo nivel se justifica cuando hay interoperabilidad, almacenamiento compacto o una operación bitwise real.

Para recordar

- n bits ofrecen 2^n patrones; la codificación asigna significado.
- Una función booleana puede especificarse con tabla de verdad y construirse mediante composición.
- NOT, AND y OR son una base; NAND o NOR por sí solas también son completas.
- La suma binaria surge de XOR, AND y propagación de acarreo.

- Bits, bytes y señales son la base; las capas de abstracción permiten trabajar productivamente por encima de ella.

CAPÍTULO 18

Expresiones regulares

Idea central

Una expresión regular describe un conjunto de secuencias de texto; se construye combinando átomos, cantidades, alternativas y posiciones. Es productiva para buscar, validar formas, extraer y reemplazar patrones locales, pero no sustituye un parser ni valida el significado del dominio.

La forma más segura de diseñarla es progresiva:

```
ejemplos válidos e inválidos
→ piezas literales
→ clases y cantidades
→ agrupación
→ anclas
→ pruebas y casos límite
```

Antes de una regex: búsqueda literal

No todo problema de texto necesita un lenguaje de patrones:

```
texto.includes("error");
texto.startsWith("TUP-");
texto.endsWith(".md");
texto.indexOf(":");
texto.replaceAll(" ", "-");
```

Estas operaciones son claras cuando la secuencia es fija. Usá una regex cuando deben variar caracteres, cantidad, posición o alternativas.

Crear un objeto `RegExp`

Literal:

```
const patron = /javascript/i;
```

Constructor:


```
const patronDinamico = new RegExp("javascript", "i");
```

Ambos producen objetos:

```
typeof patron;           // "object"  
patron instanceof RegExp; // true
```

El literal se analiza al cargar el código y es preferible para un patrón fijo. El constructor permite incorporar datos en tiempo de ejecución, pero las barras invertidas atraviesan primero la sintaxis del string:

```
const literal = /\d+/u;  
const construido = new RegExp("\\d+", "u");
```

Texto dinámico: escapar antes de interpolar

Si el usuario busca `a.b`, el punto no debería significar "cualquier carácter". Hay que escapar los metacaracteres:

```
function escaparRegex(texto) {  
  return texto.replace(/[\.*+?^${}()|[\]\]/gu, "\\$&");  
}  
  
const termino = "a.b";  
const patron = new RegExp(escaparRegex(termino), "giu");
```

El escape depende del contexto. Insertar dentro de una clase `[]` puede exigir reglas diferentes de insertar como patrón general.

Caracteres literales

```
/casa/u.test("la casa azul"); // true
```

La regex busca en cualquier posición salvo que agreguemos anclas. Las mayúsculas importan sin el flag `i`.

Metacaracteres

Tienen significado especial fuera de clases:

```
. ^ $ * + ? ( ) [ ] { } | \
```

Para buscar uno literalmente, se escapa:

```
/\./u.test("archivo.txt");  
/\/u.test("¿qué?");  
/\\/u.test("C:\\datos");
```

Dentro de `[]`, las reglas cambian: `-`, `]`, `^` y `\\` pueden necesitar atención según su posición.

El punto

`.` coincide con cualquier carácter salvo terminadores de línea en el modo habitual:

```
/c.sa/u.test("casa"); // true  
/c.sa/u.test("cosa"); // true
```

El flag `s` activa *dotAll* y permite incluir saltos:

```
/inicio.*fin/su.test("inicio\\nfin"); // true
```

Una clase negada suele expresar mejor un límite que `. * ?`:

```
/"[^"]*/u;
```

Clases de caracteres

```
/[abc]/u;      // a, b o c  
/[0-9]/u;      // dígito ASCII  
/[a-zA-Z]/u;   // letra ASCII inglesa  
/[^0-9]/u;     // cualquier carácter excepto dígito ASCII
```

El `^` niega solo si aparece al comienzo de la clase. Un guion al final o escapado se interpreta literalmente:

```
/[A-Z-]/u;
```

Los rangos siguen puntos de código, no categorías lingüísticas completas. `[A-z]` incluye caracteres entre `Z` y `a` que no son letras; no lo uses como abreviatura de ambos alfabetos.

Clases abreviadas

```
\d  dígito ASCII
\D  no dígito
\w  letra ASCII, dígito o guion bajo
\W  no carácter de palabra
\s  espacio o separador reconocido
\S  no espacio
```

```
/\d+/u.test("Legajo 123"); // true
```

`\w` no representa todas las letras Unicode. Para alfabetos generales, usá propiedades Unicode con `u`:

```
/\p{L}+/u; // una o más letras Unicode
/\p{N}+/u; // números Unicode
/\p{Script=Greek}+/u;
```

La negación usa `\P{...}`.

Cuantificadores

Se aplican al átomo anterior:

```
x?    cero o una x
x*    cero o más x
x+    una o más x
x{3}  exactamente tres x
x{2,5} entre dos y cinco x
x{2,} dos o más x
```

```
/ab+/u;      // a seguida de una o más b
/(ab)+/u;    // una o más secuencias "ab"
/colou?r/u;   // u opcional
```

```
/\d{2,4}/u;    // entre dos y cuatro dígitos
```

`*` y `?` permiten cero coincidencias. Un patrón puede coincidir con la cadena vacía, lo que tiene consecuencias al repetir búsquedas globales.

Cuantificadores codiciosos y perezosos

Por defecto intentan consumir lo máximo y retroceden si hace falta:

```
const texto = "<b>uno</b><i>dos</i>";
texto.match(/<.*>/u)?.[0]; // puede abarcar todo
```

Agregar `?` los vuelve perezosos:

```
texto.match(/<.*?>/u)?.[0]; // "<b>"
```

Para este caso, una clase con límite explícito es más precisa:

```
texto.match(/<[^>]*>/u)?.[0];
```

Ninguna de estas expresiones convierte a regex en un parser HTML correcto; atributos, comentarios y contenido especial requieren una herramienta estructural.

Anclas: posiciones, no caracteres

`^` representa el comienzo y `$` el final:

```
/^hola/u.test("hola mundo"); // true
/mundo$/u.test("hola mundo"); // true
```

Para validar el texto completo:

```
const legajo = /\d{5}$/u;

legajo.test("12345"); // true
legajo.test("X12345"); // false
legajo.test("123456"); // false
```

Sin anclas, `/\d{5}/` solo pregunta si existe una subsecuencia de cinco dígitos.

Con flag `m`, `^` y `$` también operan al inicio y final de cada línea. No confundas texto completo con cada línea.

Frontera de palabra

`\b` coincide entre una posición de palabra y una de no palabra:

```
/\bgato\b/u.test("un gato negro"); // true
/\bgato\b/u.test("gatopardo");      // false
```

Su idea de “palabra” está vinculada a `\w` y no cubre de manera intuitiva todos los idiomas. Para segmentación lingüística, `Intl.Segmenter` es más apropiado.

Agrupar

Los paréntesis agrupan una secuencia para aplicar cantidad o alternativas:

```
/(ab)+/u;
/^(rojo|verde|azul)$/u;
```

Sin grupo:

```
/rojo|verde$/u;
```

se interpreta como `rojo` en cualquier posición o `verde` al final. Agrupar hace visible el alcance de la alternativa.

Capturas

Los grupos también guardan la parte coincidente:

```
const fecha = "28/08/2026";
const coincidencia = fecha.match(/^(\\d{2})\\(\\d{2})\\(\\d{4})$/u);

coincidencia?.[0]; // texto completo
coincidencia?.[1]; // "28"
coincidencia?.[2]; // "08"
coincidencia?.[3]; // "2026"
```

Un grupo no capturante organiza sin agregar una posición:

```
/(?:https?|ftp):\\//u;
```

Usalo cuando no necesitarás el contenido del grupo.

Grupos con nombre

```
const patronFecha = /^(?<dia>\d{2})\/(?<mes>\d{2})\/(?<anio>\d{4})$/u;
const resultado = "28/08/2026".match(patronFecha);

if (resultado) {
  const { dia, mes, anio } = resultado.groups;
  console.log({ dia, mes, anio });
}
```

Los nombres resisten mejor cambios de orden y documentan la extracción.

Referencias a capturas

Una referencia posterior exige repetir el mismo texto:

```
/\b(\p{L}+)\s+\1\b/giu;
```

Con nombre:

```
/\b(?<palabra>\p{L}+)\s+\k<palabra>\b/giu;
```

Puede detectar palabras consecutivas repetidas. El flag `i` aplica comparación sin distinguir mayúsculas según las reglas del motor.

Lookahead y lookbehind

Comprueban contexto sin consumirlo.

Lookahead positivo:

```
/\d+(?=\s*ARS)/u; // número seguido de ARS
```

Lookahead negativo:

```
/^(?!admin$)[a-z]+$ /u; // palabra minúscula excepto admin
```

Lookbehind positivo:

```
/(?<=\$)\d+(?:\.\d+)? /u; // número precedido por $
```

Lookbehind negativo:

```
/(?<!\d)-\d+ /u; // signo menos no precedido por dígito
```

Son potentes y pueden reducir legibilidad. A veces capturar el contexto y procesarlo después es más portable y fácil de depurar.

Flags

Flag	Efecto
<code>g</code>	todas las coincidencias y uso de <code>lastIndex</code>
<code>i</code>	ignora diferencias de mayúsculas según reglas del motor
<code>m</code>	anclas por línea
<code>s</code>	<code>.</code> incluye terminadores de línea
<code>u</code>	semántica Unicode y sintaxis Unicode estricta
<code>y</code>	coincidencia adherida a <code>lastIndex</code>
<code>d</code>	índices de los rangos coincidentes

```
const patron = /hola/giu;  
patron.flags; // "giu" en un orden canónico  
patron.global; // true  
patron.source; // "hola"
```

El flag `u` debería ser habitual en patrones modernos que procesan texto Unicode.

test

```
/error/iu.test("ERROR de conexión"); // true
```

Devuelve un booleano. Con `g` o `y`, el objeto conserva `lastIndex`:

```
const global = /a/g;

global.test("a"); // true; lastIndex pasa a 1
global.test("a"); // false; comienza desde 1
```

Para comprobaciones independientes, evita `g` o reiniciá deliberadamente el estado.

exec

```
const patronNumero = /\d+/gu;
const coincidencia = patronNumero.exec("A12 B34");

coincidencia[0]; // "12"
coincidencia.index; // 1
patronNumero.lastIndex; // posición posterior
```

Con `g`, llamadas sucesivas recorren coincidencias. Si el patrón puede coincidir con vacío, asegurará progreso para evitar bucles infinitos en código manual.

match y matchAll

Sin `g`, `match` devuelve detalle de la primera coincidencia:

```
"A12 B34".match(/(?<numero>\d+)/u);
```

Con `g`, devuelve solo textos completos:

```
"A12 B34".match(/\d+/gu); // ["12", "34"]
```

`matchAll` devuelve un iterador con detalles para cada coincidencia y requiere `g`:


```
const resultados = [
  ..."A12 B34".matchAll(/(?<numero>\d+)/gu)
];
```

Es la forma cómoda de obtener grupos e índices de todas las apariciones.

replace

Con referencias:

```
"28/08/2026".replace(
  /^(\d{2})\/(\d{2})\/(\d{4})$/u,
  "$3-$2-$1"
); // "2026-08-28"
```

Con grupos nombrados:

```
"28/08/2026".replace(
  /^(?<d>\d{2})\/(?<m>\d{2})\/(?<a>\d{4})$/u,
  "$<a>-$<m>-$<d>"
);
```

Con callback:

```
"precios: 10 y 20".replace(/\d+/gu, texto =>
  String(Number(texto) * 2)
);
```

Sin `g`, solo reemplaza la primera coincidencia.

search y split

```
"abc 123".search(/\d/u); // 4
"uno, dos; tres".split(/\s*[;,]\s*/u);
// ["uno", "dos", "tres"]
```

Si `split` contiene grupos capturantes, los separadores capturados pueden aparecer en el resultado. Usá `(?:...)` si no los necesitas.

Construir una validación paso a paso

Código `ABC-1234`:

```
tres letras ASCII mayúsculas → [A-Z]{3}
guion literal → -
cuatro dígitos ASCII → \d{4}
texto completo → ^[A-Z]{3}-\d{4}$
```

```
const codigo = /^[A-Z]{3}-\d{4}$/u;
```

Casos de prueba:

```
const casos = new Map([
  ["ABC-1234", true],
  ["abc-1234", false],
  ["AB-1234", false],
  ["ABC-12345", false],
  ["XABC-1234", false]
]);

for (const [texto, esperado] of casos) {
  console.assert(codigo.test(texto) === esperado, texto);
}
```

Validar forma y luego significado

Esta regex comprueba una fecha con dos dígitos por componente:

```
/^\d{2}\d{2}\d{4}$/u;
```

También acepta `99/99/0000`. Después de extraer:

```
function leerFecha(texto) {
  const match = texto.match(
    /^(?<dia>\d{2})\/(?<mes>\d{2})\/(?<anio>\d{4})$/u
  );

  if (!match) return null;

  const dia = Number(match.groups.dia);
  const mes = Number(match.groups.mes);
  const anio = Number(match.groups.anio);
```

```
const fecha = new Date(Date.UTC(anio, mes - 1, dia));

const valida =
  fecha.getUTCFullYear() === anio &&
  fecha.getUTCMonth() === mes - 1 &&
  fecha.getUTCDate() === dia;

return valida ? fecha : null;
}
```

Regex valida forma; el código valida calendario.

Email: no exagerar

Una comprobación práctica puede detectar errores obvios:

```
const formaBasica = /^[^\s@]+@[^\s@]+\.[^\s@]+$/u;
```

No prueba que la dirección exista ni cubre necesariamente todas las variantes permitidas por estándares. La verificación real es enviar un mensaje o usar un flujo de confirmación. Ajustá la sintaxis a lo que la aplicación realmente admite, sin prometer validación universal.

Patrones útiles

Normalizar espacios:

```
texto.trim().replace(/\s+/gu, " ");
```

Extraer números:

```
[...texto.matchAll(/[+-]?[0-9]+(?:[.,]\d+)?/gu)]
  .map(match => match[0]);
```

Hashtags Unicode:

```
[...texto.matchAll(/#(<etiqueta>[\p{L}\p{N}_]+)/gu)]
  .map(match => match.groups.etiqueta);
```

Hora de 24 horas:

```
/^(?:[01]\d|2[0-3]):[0-5]\d$/u;
```

Nombre y número:

```
const linea = /^(?<nombre>\p{L}+(?:[ '\-]\p{L}+)*)\s*:\s*(?<valor>\d+)\$/u;
```

El patrón refleja un contrato concreto; no pretende reconocer todos los nombres del mundo.

Rendimiento y retroceso excesivo

Algunos motores exploran alternativas mediante retroceso. Patrones ambiguos con cuantificadores anidados pueden crecer de forma extrema ante entradas que casi coinciden:

```
/(a+)+$/u;
```

Con muchas `a` seguidas de un carácter inválido, puede requerir gran cantidad de intentos. Para entradas externas:

- evitá cuantificadores anidados ambiguos;
- usá clases y límites más específicos;
- limitá el tamaño de entrada;
- medí patrones críticos con casos adversos;
- no aceptes patrones arbitrarios de usuarios sin aislamiento y límites.

Una regex corta no es necesariamente barata.

Regex y autómatas

En su forma teórica, una expresión regular describe un lenguaje regular que puede reconocerse con un autómata finito. JavaScript agrega extensiones como referencias a capturas y lookahead que superan parte de esa definición clásica.

El modelo útil sigue siendo:

```
estado actual + próximo símbolo → nuevo estado
```

Las anclas restringen posiciones, las clases aceptan conjuntos de símbolos, los cuantificadores repiten transiciones y `|` ofrece caminos alternativos.

Cuándo no usar regex

No es la herramienta principal para:

- JSON: `JSON.parse`;
- HTML o XML general: parser estructural;
- CSV completo: parser con comillas y saltos;
- código fuente: tokenizador y parser;
- estructuras anidadas arbitrariamente;
- validación semántica de fechas, dominios o identidades.

Puede participar en una fase pequeña, como tokenizar un fragmento o comprobar una forma antes del parser.

Errores frecuentes

- olvidar anclas al validar el texto completo;
- no escapar texto dinámico;
- confundir `\d` y `\w` con todas las categorías Unicode;
- capturar grupos que solo servían para agrupar;
- usar `g` con `test` y olvidar `lastIndex`;
- esperar grupos detallados de `match` global;
- usar `.*` cuando existe un delimitador específico;
- validar solo forma y asumir significado;
- crear patrones vulnerables a retroceso excesivo;
- usar regex como parser de una gramática anidada.

Para recordar

- Una regex describe posibilidades; no “entiende” el significado del texto.
- Construí desde átomos, cantidades, grupos y posiciones, guiado por ejemplos.
- Anclas convierten una búsqueda en validación completa; capturas convierten coincidencias en datos.
- Flags y métodos cambian estado y forma del resultado; `g` merece atención especial.
- Regex es excelente para patrones locales y peligrosa cuando reemplaza parsers, validación semántica o límites de seguridad.

CAPÍTULO 19

Gestión de archivos con Node.js

Idea central

Una operación de archivos atraviesa cuatro capas: una ruta identifica el recurso, el sistema entrega bytes, una codificación puede convertirlos en texto y un formato puede transformar ese texto o esos bytes en datos. Un programa productivo controla cada capa, evita sobrescrituras y eliminaciones accidentales, y elige entre lectura completa, manejadores o streams según el tamaño y la concurrencia.

ruta → archivo → bytes → codificación → texto → formato → datos

Confundir capas produce errores como interpretar un PDF con `readFile(..., "utf8")`, asumir que `.json` garantiza JSON válido o creer que una ruta relativa parte del archivo JavaScript.

Archivo, directorio, ruta y metadatos

- **Archivo:** secuencia de bytes con metadatos.
- **Directorio:** estructura que asocia nombres con entradas del sistema.
- **Ruta:** expresión que permite localizar una entrada.
- **Metadatos:** tamaño, tipo, permisos, fechas y otra información del sistema.

La extensión es parte del nombre. Ayuda a inferir un formato, pero no transforma ni valida el contenido.

datos.json puede contener texto inválido
imagen.jpg puede no ser JPEG
informe.txt puede contener cualquier secuencia de bytes

Node.js como intermediario

El lenguaje JavaScript no define acceso general al disco. Node.js aporta módulos como:

```
import { readFile } from "node:fs/promises";  
import path from "node:path";
```

El navegador restringe el sistema de archivos y ofrece APIs diferentes por seguridad.

Tres estilos de API

Promesas

```
import { readFile } from "node:fs/promises";

const texto = await readFile("datos.txt", "utf8");
```

Es el estilo principal de este capítulo: no bloquea el hilo mientras espera y se integra con `async` / `await`.

Callbacks

```
import { readFile } from "node:fs";

readFile("datos.txt", "utf8", (error, texto) => {
  if (error) {
    console.error(error);
    return;
  }

  console.log(texto);
});
```

Sigue presente en APIs y código histórico.

Sincrónico

```
import { readFileSync } from "node:fs";

const texto = readFileSync("datos.txt", "utf8");
```

Bloquea el hilo hasta terminar. Puede ser razonable en un script corto, durante el arranque o antes de aceptar trabajo concurrente. En un servidor puede detener todas las solicitudes mientras el disco responde.

Preparar un módulo ejecutable

Podés usar un archivo `.mjs`:

```
node programa.mjs
```

O un proyecto con `package.json`:

```
{
  "type": "module"
}
```

Entonces los archivos `.js` usan `import`. Los proyectos CommonJS utilizan `require`; no mezcles formatos sin comprender cómo los carga Node.js.

Rutas relativas y directorio de trabajo

```
await readFile("datos/entrada.txt", "utf8");
```

La ruta se interpreta desde `process.cwd()`, el directorio de trabajo del proceso:

```
console.log(process.cwd());
```

No necesariamente coincide con la carpeta del módulo. Puede cambiar según desde dónde se ejecute:

```
node herramientas/procesar.mjs
```

Ruta relativa al módulo

```
import path from "node:path";
import { fileURLToPath } from "node:url";

const archivoActual = fileURLToPath(import.meta.url);
const directorioActual = path.dirname(archivoActual);
const rutaDatos = path.join(directorioActual, "datos", "entrada.txt");
```

Elegí la base según el contrato:

- `process.cwd()` para una herramienta que trabaja sobre el proyecto invocado;
- `import.meta.url` para recursos ubicados junto al módulo;
- una ruta recibida explícitamente para mayor control y prueba.

Construir rutas con `node:path`

```
path.join("datos", "2026", "alumnos.json");
path.resolve("datos", "entrada.txt");
path.dirname(ruta);
path.basename(ruta);
path.extname(ruta);
path.parse(ruta);
path.format({ dir: "/tmp", name: "informe", ext: ".txt" });
```

`join` combina segmentos y normaliza. `resolve` construye una ruta absoluta procesando de derecha a izquierda hasta encontrar una base absoluta o usar el directorio de trabajo.

No concatenes separadores manualmente:

```
// const ruta = carpeta + "/" + archivo;
const ruta = path.join(carpeta, archivo);
```

Diferencias entre sistemas

Windows y sistemas tipo Unix difieren en:

- separadores (`\` frente a `/`);
- raíces y letras de unidad;
- sensibilidad habitual a mayúsculas;
- caracteres permitidos;
- permisos y enlaces;
- convenciones de fin de línea.

`path` usa las reglas de la plataforma actual. `path.posix` y `path.win32` permiten manipular expresiones de una plataforma específica cuando se procesa un formato externo.

No cambies mayúsculas ni reemplaces caracteres de una ruta sin conocer el sistema y el contrato.

Crear directorios

```
import { mkdir } from "node:fs/promises";

await mkdir("salidas", { recursive: true });
```

Con `recursive`, crea padres faltantes y no falla si el directorio ya existe. Sin esa opción, un padre ausente produce error y una entrada existente puede producir `EEXIST`.

Escribir texto

```
import { writeFile } from "node:fs/promises";

await writeFile("salida.txt", "Hola\n", "utf8");
```

Por defecto, `writeFile` crea o reemplaza. Si sobrescribir es peligroso, pedilo de forma explícita:

```
await writeFile("salida.txt", "Hola\n", {
  encoding: "utf8",
  flag: "wx"
});
```

`wx` crea de manera exclusiva y falla con `EEXIST` si ya existe. Evita la carrera de "comprobar y luego crear".

Crear un archivo vacío:

```
await writeFile("vacio.txt", "", { flag: "wx" });
```

Agregar sin borrar

```
import { appendFile } from "node:fs/promises";

await appendFile("eventos.log", "inicio\n", "utf8");
```

Varias operaciones concurrentes sobre el mismo archivo no forman automáticamente una transacción. Los límites de atomicidad dependen del sistema, el tamaño y el modo de apertura. Si el orden es esencial, serializá escrituras o utilizá un almacenamiento diseñado para concurrencia.

Bytes, Buffer y texto

Sin codificación, `readFile` devuelve un `Buffer`:

```
const bytes = await readFile("imagen.png");

Buffer.isBuffer(bytes); // true
bytes.length;           // cantidad de bytes
```

Un `Buffer` es una vista de bytes de Node.js. Puede convertirse:

```
const texto = bytes.toString("utf8");
const buffer = Buffer.from("mañana", "utf8");
```

Con codificación en `readFile`, Node.js devuelve string:

```
const texto = await readFile("datos.txt", "utf8");
```

Unicode no es UTF-8

Unicode asigna puntos de código. UTF-8 define cómo codificarlos en bytes. JavaScript representa strings internamente mediante UTF-16.

```
const texto = "😄";

texto.length; // 2 unidades UTF-16
Buffer.byteLength(texto, "utf8"); // 4 bytes UTF-8
[...texto].length; // 1 punto de código
```

No supongas un byte por carácter ni que `string.length` mide almacenamiento.

Codificación incorrecta

Si bytes UTF-8 se decodifican como otra codificación, aparecen caracteres corruptos. No existe una detección universal infalible; varias codificaciones pueden interpretar cualquier byte y producir texto aparentemente válido.

El origen debe declarar el contrato. Para decodificación UTF-8 estricta:

```
const decodificador = new TextDecoder("utf-8", { fatal: true });
```

```
const texto = decodificador.decode(bytes);
```

Con `fatal`, una secuencia inválida lanza en vez de insertar el carácter de reemplazo.

BOM

Algunos archivos comienzan con una marca de orden de bytes. UTF-8 no la necesita, pero puede incluir `EF BB BF`. Algunas herramientas la eliminan o interpretan; otras la conservan como `U+FEFF` al inicio.

Si el primer encabezado de un CSV o JSON parece tener un carácter invisible, inspecciona el BOM. No elimines automáticamente cualquier `U+FEFF` interior, porque puede formar parte real del contenido.

Fines de línea

Convenciones comunes:

```
Unix/macOS moderno: \n
Windows:             \r\n
```

Para dividir líneas de ambos:

```
const lineas = texto.split(/\r?\n/u);
```

Eso puede dejar una última línea vacía si el archivo termina con salto. Decidí si representa contenido o solo terminación.

Para escribir con la convención de la plataforma:

```
import os from "node:os";

const contenido = lineas.join(os.EOL);
```

En repositorios, suele acordarse `\n` independientemente del sistema para reducir diferencias.

Leer y transformar un archivo pequeño

```
async function numerarLineas(origen, destino) {
  const texto = await readFile(origen, "utf8");

  const numerado = texto
    .split(/\r?\n/u)
    .map((linea, indice) => `${indice + 1}: ${linea}`)
    .join("\n");

  await writeFile(destino, numerado, {
    encoding: "utf8",
    flag: "wx"
  });
}
```

Es simple y correcto cuando el contenido completo cabe cómodamente en memoria.

Escritura más segura mediante archivo temporal

Reemplazar un archivo crítico directamente puede dejarlo incompleto si el proceso falla durante la escritura. Un patrón común:

1. escribir un temporal en el mismo directorio;
2. cerrar y, si el nivel de garantía lo exige, sincronizar;
3. renombrar el temporal sobre el destino.

```
import { rename } from "node:fs/promises";

async function escribirReemplazo(ruta, contenido) {
  const temporal = `${ruta}.tmp-${process.pid}`;

  await writeFile(temporal, contenido, {
    encoding: "utf8",
    flag: "wx"
  });

  await rename(temporal, ruta);
}
```

La atomicidad y reemplazo de `rename` dependen del sistema; debe ocurrir en el mismo volumen. Una implementación robusta también limpia temporales, evita colisiones y considera permisos y sincronización.

Listar un directorio

```
import { readdir } from "node:fs/promises";

const nombres = await readdir("datos");
```

Con tipos:

```
const entradas = await readdir("datos", { withFileTypes: true });

for (const entrada of entradas) {
  if (entrada.isFile()) console.log("archivo", entrada.name);
  if (entrada.isDirectory()) console.log("directorio", entrada.name);
  if (entrada.isSymbolicLink()) console.log("enlace", entrada.name);
}
```

Un `Dirent` evita consultar `stat` para la clasificación básica de cada entrada.

Metadatos con `stat` y `lstat`

```
import { stat, lstat } from "node:fs/promises";

const info = await stat(ruta);

info.isFile();
info.isDirectory();
info.size;
info.mtime;
```

`stat` sigue un enlace simbólico; `lstat` informa sobre el enlace mismo. Esta diferencia es importante al recorrer o eliminar árboles, porque seguir enlaces puede salir del directorio esperado o crear ciclos.

Los metadatos pueden cambiar inmediatamente después de consultarlos. No los trates como una garantía permanente.

Comprobar existencia sin crear una carrera

Esto tiene una ventana:

```
comprobar → otro proceso cambia el archivo → operar
```

Intentá la operación y manejá el código esperado:

```
async function leerOpcional(ruta) {
  try {
    return await readFile(ruta, "utf8");
  } catch (error) {
    if (error.code === "ENOENT") return null;
    throw error;
  }
}
```

`access` existe, pero suele servir para diagnósticos o comprobaciones informativas, no para autorizar una operación posterior.

Códigos de error frecuentes

- `ENOENT` : ruta inexistente;
- `EACCES` o `EPERM` : permiso insuficiente;
- `EEXIST` : ya existe;
- `EISDIR` : se esperaba archivo y era directorio;
- `ENOTDIR` : un segmento no era directorio;
- `ENOTEMPTY` : directorio no vacío;
- `EXDEV` : movimiento entre volúmenes no soportado por `rename` ;
- `EMFILE` : demasiados descriptores abiertos.

```
try {
  await operacion();
} catch (error) {
  if (error.code === "ENOENT") {
    // recuperación específica
  } else {
    throw error;
  }
}
```

No dependas solo del mensaje, que cambia entre plataformas.

Recorrer directorios recursivamente

```
async function* recorrer(directorio) {
  const entradas = await readdir(directorio, { withFileTypes: true });
```

```
for (const entrada of entradas) {
  const ruta = path.join(directorio, entrada.name);

  if (entrada.isDirectory()) {
    yield* recorrer(ruta);
  } else if (entrada.isFile()) {
    yield ruta;
  }
}
```

El generador produce a medida que recorre. La política ignora enlaces simbólicos; hay que decidir deliberadamente si seguirlos, detectar ciclos y limitar profundidad.

Buscar por nombre o extensión

```
async function buscarMarkdown(raiz) {
  const resultados = [];

  for await (const ruta of recorrer(raiz)) {
    if (path.extname(ruta).toLowerCase() === ".md") {
      resultados.push(ruta);
    }
  }

  return resultados;
}
```

La extensión no valida el contenido, pero puede filtrar candidatos. Para patrones complejos se usan globbers con reglas claras sobre separadores, archivos ocultos y enlaces.

Buscar texto en archivos pequeños

```
async function buscarTexto(ruta, patron) {
  const texto = await readFile(ruta, "utf8");

  return texto
    .split(/\r?\n/u)
    .flatMap((linea, indice) =>
      patron.test(linea)
        ? [{ numero: indice + 1, linea }]
        : []
    );
}
```


Si `patron` tiene flag `g`, `test` conserva `lastIndex` y puede alternar resultados. Para comprobar cada línea, eliminá `g`, cloná el patrón apropiadamente o reiniciá su estado.

Leer archivos grandes por líneas

```
import { createReadStream } from "node:fs";
import { createInterface } from "node:readline";

async function contarCoincidencias(ruta, patron) {
  const entrada = createReadStream(ruta, { encoding: "utf8" });
  const lineas = createInterface({
    input: entrada,
    crlfDelay: Infinity
  });

  let cantidad = 0;

  for await (const linea of lineas) {
    if (patron.test(linea)) cantidad += 1;
  }

  return cantidad;
}
```

`readline` conserva fragmentos hasta completar una línea y entiende `\r\n` con `crlfDelay: Infinity`.

Un fragmento de stream no es una línea

Un stream entrega chunks según buffers y disponibilidad, no según límites semánticos:

```
chunk 1: "primera\nsegu"
chunk 2: "nda\ntercera"
```

Tampoco debe dividirse un carácter UTF-8 manualmente sin un decodificador que conserve bytes incompletos. APIs de texto del stream y `readline` resuelven parte de ese trabajo.

Copiar con streams y `pipeline`

```
import { createReadStream, createWriteStream } from "node:fs";
import { pipeline } from "node:stream/promises";
```

```
await pipeline(  
  createReadStream(origen),  
  createWriteStream(destino, { flags: "wx" })  
);
```

`pipeline` propaga errores y coordina cierre y contrapresión. Es preferible a conectar eventos manualmente para un flujo lineal.

La **contrapresión** evita que el productor lea mucho más rápido de lo que el consumidor puede escribir o procesar.

Manejadores de archivo

```
import { open } from "node:fs/promises";  
  
const archivo = await open(ruta, "r");  
  
try {  
  const buffer = Buffer.alloc(100);  
  const { bytesRead } = await archivo.read(buffer, 0, 100, 0);  
  usar(buffer.subarray(0, bytesRead));  
} finally {  
  await archivo.close();  
}
```

Un `FileHandle` permite lecturas parciales, posiciones explícitas, sincronización y varias operaciones sobre la misma apertura. Siempre debe cerrarse, incluso ante error.

Modos comunes:

- `r`: lectura; debe existir;
- `r+`: lectura y escritura; debe existir;
- `w`: escritura; crea o trunca;
- `wx`: escritura exclusiva;
- `a`: agregar; crea si falta;
- `ax`: agregar de forma exclusiva.

Elegir el modo equivocado puede borrar contenido.

JSON: sintaxis y estructura

```
async function leerJson(ruta) {
  const texto = await readFile(ruta, "utf8");
  const datos = JSON.parse(texto);

  if (!datos || typeof datos !== "object" || Array.isArray(datos)) {
    throw new TypeError("Se esperaba un objeto JSON");
  }

  return datos;
}
```

`JSON.parse` comprueba sintaxis. La aplicación debe validar propiedades, tipos, rangos y relaciones.

Escribir:

```
await writeFile(
  ruta,
  `${JSON.stringify(datos, null, 2)}\n`,
  { encoding: "utf8", flag: "wx" }
);
```

JSON no admite comentarios, comas finales, `undefined`, símbolos, `bigint`, `Map`, `Set` ni referencias cíclicas sin una representación personalizada.

CSV no es simplemente `split(",")`

Una línea puede contener:

```
123,"Pérez, Ana","texto con ""comillas"""
```

El formato real permite separadores dentro de comillas, comillas escapadas y a veces saltos dentro de un campo. Usá una biblioteca CSV probada y configurá:

- delimitador;
- codificación;
- encabezados;
- política de filas mal formadas;
- conversión y validación por columna.

Para un formato didáctico deliberadamente simple, documentá que no acepta comillas ni separadores internos.

Extraer datos de un log

```
const patron = /^(?<fecha>\S+)\s+(?<nivel>INFO|WARN|ERROR)\s+(?<mensaje>.*$)/u;

function interpretarLinea(linea) {
  const match = linea.match(patron);
  if (!match) return { ok: false, linea };

  return {
    ok: true,
    ...match.groups
  };
}
```

Un parser de línea puede devolver éxitos y errores como datos para que una fila inválida no detenga todo el lote. Los fallos de lectura siguen siendo excepciones de la operación.

Formatos binarios

PDF, DOCX, imágenes y audio no deben convertirse enteros a UTF-8. Se leen como bytes y se procesan con bibliotecas que entienden su formato.

```
const contenido = await readFile("informe.pdf");
```

Una firma inicial puede ayudar a detectar un candidato, pero validar un formato completo requiere interpretar su estructura.

Copiar

```
import { copyFile, cp } from "node:fs/promises";
import { constants } from "node:fs";

await copyFile(origen, destino);
await copyFile(origen, destino, constants.COPYFILE_EXCL);

await cp(origenDir, destinoDir, {
  recursive: true,
```

```
errorIfExists: true,  
force: false  
});
```

Definí si se reemplaza, si se siguen enlaces, qué metadatos se preservan y qué ocurre ante un fallo parcial.

Renombrar y mover

```
import { rename } from "node:fs/promises";  
  
await rename(origen, destino);
```

Dentro del mismo sistema de archivos suele ser una operación atómica sobre el nombre. Entre volúmenes puede fallar con `EXDEV`; una estrategia de copiar y eliminar debe contemplar que la copia termine y sea verificable antes de borrar el origen.

Eliminar

```
import { unlink, rmdir, rm } from "node:fs/promises";  
  
await unlink(rutaArchivo);  
await rmdir(directorioVacio);  
await rm(arbol, { recursive: true });
```

La eliminación recursiva es peligrosa. Antes de usarla:

1. resolvé una ruta absoluta;
2. verificá que esté dentro de una raíz permitida;
3. rechazá raíz, carpeta de trabajo y destinos demasiado amplios;
4. ofrecé simulación o papelera cuando sea una herramienta de usuario;
5. registrá exactamente qué se eliminará;
6. tratá enlaces según una política explícita.

No dependas de una variable vacía, glob o concatenación para identificar un destino destructivo.

Impedir que una ruta escape

```
function resolverDentroDe(base, entrada) {
  const raiz = path.resolve(base);
  const candidata = path.resolve(raiz, entrada);
  const relativa = path.relative(raiz, candidata);

  if (relativa === "" ||
      relativa.startsWith(`..${path.sep}`) ||
      relativa === ".." ||
      path.isAbsolute(relativa)) {
    throw new Error("Ruta fuera del directorio permitido");
  }

  return candidata;
}
```

La condición `relativa === ""` rechaza la propia raíz si la operación no debe apuntarle. Ajustala si leer la raíz es válido.

Esta comprobación textual no resuelve por sí sola enlaces simbólicos, cambios concurrentes ni diferencias de mayúsculas del sistema. Para entradas hostiles se necesitan controles adicionales y privilegios mínimos.

Concurrencia y límites

Procesar miles de archivos con `Promise.all` los abre casi simultáneamente y puede producir `EMFILE`:

```
await Promise.all(rutas.map(procesar));
```

La alternativa secuencial es segura pero puede ser lenta:

```
for (const ruta of rutas) {
  await procesar(ruta);
}
```

Un pool con concurrencia limitada equilibra recursos y rendimiento. El límite adecuado depende de disco, red, tamaño y operación.

No ejecutes escrituras concurrentes sobre el mismo destino sin coordinación. El orden de finalización puede diferir del orden de inicio.

Programa integrador: analizar una carpeta

```
async function analizarCarpeta(raiz) {
  const resumen = {
    archivos: 0,
    bytes: 0,
    lineas: 0,
    errores: []
  };

  for await (const ruta of recorrer(raiz)) {
    try {
      const info = await stat(ruta);
      resumen.archivos += 1;
      resumen.bytes += info.size;

      if (path.extname(ruta).toLowerCase() === ".txt") {
        const texto = await readFile(ruta, "utf8");
        resumen.lineas += texto === ""
          ? 0
          : texto.split(/\r?\n/u).length;
      }
    } catch (error) {
      resumen.errores.push({
        ruta,
        codigo: error.code ?? "DESCONOCIDO",
        mensaje: error.message
      });
    }
  }

  return resumen;
}
```

Esta primera versión prioriza claridad. Para producción habría que decidir tamaño máximo para lectura completa, seguimiento de enlaces, concurrencia, cancelación y política ante errores.

Criterios para elegir una técnica

Necesidad	Técnica inicial
archivo pequeño completo	<code>readFile</code>
texto grande por líneas	<code>createReadStream</code> + <code>readline</code>
copiar flujo	<code>pipeline</code>
leer posiciones específicas	<code>FileHandle</code>
script corto de arranque	API síncrona, si bloquear es aceptable
muchas rutas	iterador y concurrencia limitada
crear sin reemplazar	flag exclusivo <code>x</code>
reemplazar dato crítico	temporal + <code>rename</code> , con garantías documentadas

Errores frecuentes

- creer que extensión y formato son lo mismo;
- asumir que una ruta relativa parte del módulo;
- concatenar separadores manualmente;
- omitir codificación y esperar string;
- suponer un byte por carácter;
- sobrescribir con `writeFile` sin intención;
- comprobar existencia y luego actuar como si nada pudiera cambiar;
- leer archivos gigantes completos;
- creer que un chunk es una línea;
- dividir cualquier CSV por comas;
- seguir enlaces sin política;
- lanzar concurrencia sin límite;
- capturar y silenciar todos los códigos;
- eliminar recursivamente una ruta no validada.

Para recordar

- Ruta, bytes, codificación y formato son capas separadas.
- La ruta relativa depende del directorio de trabajo; los recursos del módulo requieren otra base.
- `readFile` simplifica archivos pequeños; streams y manejadores controlan tamaño y acceso parcial.

- Los flags de apertura expresan si crear, truncar, agregar o exigir exclusividad.
- Seguridad y corrección exigen validar rutas, limitar concurrencia y tratar sobrescritura y eliminación como decisiones explícitas.

BLOQUE 5

Apéndices

2 capítulos en este bloque.

APÉNDICE A

Byte Pair Encoding: construir un tokenizador

Idea central

Byte Pair Encoding (BPE) convierte texto en una secuencia de tokens al comenzar con unidades universales y aprender, en orden, fusiones de pares frecuentes. El modelo aprendido no es solo un diccionario: es una lista ordenada de reglas que debe aplicarse de la misma forma al codificar y deshacerse al decodificar.

Este apéndice integra conceptos del libro:

- strings, Unicode y UTF-8;
- arrays, `Map` y conteos;
- funciones puras y clases;
- recursividad;
- archivos JSON;
- invariantes y pruebas.

La implementación es didáctica. Permite comprender el mecanismo, observarlo y experimentar; no intenta reemplazar un tokenizador de producción.

A.1. El problema: convertir texto en números

Los modelos y muchos algoritmos numéricos no operan directamente sobre palabras visibles. Necesitan identificadores:

```
"hola mundo" → [391, 82, 1042]
```

La conversión debe resolver requisitos en tensión:

1. representar cualquier texto;
2. no crear un vocabulario imposible de mantener;
3. producir secuencias razonablemente cortas;
4. ser reversible o conservar la información necesaria;
5. aplicar exactamente el mismo modelo durante entrenamiento e inferencia.

A.2. Por qué no usar solo palabras

Un token por palabra parece natural:

```
"casa" → 1  
"casas" → 2  
"casita" → 3
```

Pero aparecen:

- formas flexionadas;
- errores de escritura;
- nombres propios;
- URLs, código y números;
- idiomas diferentes;
- palabras que no estaban en el corpus.

Un vocabulario de palabras crece mucho y todavía necesita un token desconocido.

A.3. Por qué no usar solo caracteres o bytes

Unidades pequeñas representan cualquier texto, pero alargan la secuencia. Una palabra frecuente se repite como varios pasos en lugar de una unidad reutilizable.

BPE busca un compromiso:

```
unidades pequeñas universales  
+ composiciones frecuentes aprendidas  
= cobertura completa con secuencias más compactas
```

A.4. Texto, puntos de código y UTF-8

Un string JavaScript usa unidades UTF-16. Para obtener una base finita y universal, esta implementación lo codifica como UTF-8:

```
const encoder = new TextEncoder();  
const decoder = new TextDecoder("utf-8", { fatal: true });  
  
const bytes = [...encoder.encode("mañana")];  
const recuperado = decoder.decode(Uint8Array.from(bytes));
```

Cada byte es un entero entre 0 y 255. Por lo tanto, los ids base son:

0, 1, 2, ... 255

Un byte no equivale a un carácter. ñ ocupa más de un byte en UTF-8 y un emoji suele ocupar cuatro. BPE puede aprender a fusionar esos bytes si aparecen con frecuencia.

A.5. Qué es un token

En este modelo, un token es un id que representa:

- un byte base; o
- la concatenación de dos tokens anteriores.

token 256 = token 97 + token 110

Si 97 representa a y 110 representa n en UTF-8/ASCII, el nuevo token representa an.

Una regla posterior puede usar el token 256:

token 257 = token 98 + token 256 → "ban"

El vocabulario forma un grafo de composiciones dirigido hacia unidades anteriores.

A.6. La idea del entrenamiento

Dada una secuencia:

banana banana

la convertimos en bytes y repetimos:

1. contar pares adyacentes;
2. elegir el más frecuente;
3. asignarle un id nuevo;
4. reemplazar todas sus apariciones no superpuestas;
5. guardar la regla;
6. volver a contar sobre la secuencia resultante.

El proceso se detiene al alcanzar un número máximo de fusiones o cuando ningún par alcanza la frecuencia mínima.

A.7. Contar pares

Una clave de texto permite usar `Map` :

```
function clavePar(izquierda, derecha) {
  return `${izquierda},${derecha}`;
}

function contarPares(ids) {
  const conteos = new Map();

  for (let indice = 0; indice < ids.length - 1; indice += 1) {
    const clave = clavePar(ids[indice], ids[indice + 1]);
    conteos.set(clave, (conteos.get(clave) ?? 0) + 1);
  }

  return conteos;
}
```

Para:

```
[1, 2, 1, 2, 3]
```

cuenta:

```
(1,2) → 2
(2,1) → 1
(2,3) → 1
```

La representación de clave es suficiente para ids enteros. Una implementación optimizada podría evitar crear strings en cada par.

A.8. Elegir el par más frecuente

```
function parMasFrecuente(ids) {
  const conteos = contarPares(ids);
  let mejorPar = null;
  let mejorFrecuencia = 0;

  for (const [clave, frecuencia] of conteos) {
```

```

    if (frecuencia > mejorFrecuencia) {
        mejorPar = clave.split(",").map(Number);
        mejorFrecuencia = frecuencia;
    }
}

return mejorPar === null
    ? null
    : { par: mejorPar, frecuencia: mejorFrecuencia };
}

```

El desempate es determinista: `Map` conserva el orden de primera inserción y solo reemplazamos al encontrar una frecuencia estrictamente mayor. Por lo tanto, gana el par que apareció primero entre los máximos.

La política de desempate forma parte del modelo. Otra implementación que elija distinto aprenderá reglas diferentes aunque use el mismo corpus.

A.9. Fusionar sin superposición

```

function fusionar(ids, [izquierda, derecha], nuevoId) {
    const resultado = [];

    for (let indice = 0; indice < ids.length; ) {
        const coincide =
            ids[indice] === izquierda &&
            ids[indice + 1] === derecha;

        if (coincide) {
            resultado.push(nuevoId);
            indice += 2;
        } else {
            resultado.push(ids[indice]);
            indice += 1;
        }
    }

    return resultado;
}

```

```

fusionar([1, 2, 1, 2, 1], [1, 2], 256);
// [256, 256, 1]

```

El índice avanza dos posiciones ante una fusión. Así las coincidencias no se superponen.

Para `[1, 1, 1]` y el par `[1, 1]`, el resultado es `[256, 1]`, no una fusión doble que reutilice el elemento central.

A.10. Representar una regla

```
const regla = {
  izquierda: 97,
  derecha: 110,
  id: 256
};
```

El id nuevo será `256 + cantidadDeFusiones`. La lista queda contigua y ordenada. Una regla solo puede referirse a bytes o a tokens creados antes.

Invariantes:

```
id >= 256
izquierda < id
derecha < id
id de la regla en posición i = 256 + i
```

Estos invariantes permiten localizar una regla mediante `fusiones[id - 256]` y garantizan que la expansión recursiva termina.

A.11. Clase completa

```
class BPE {
  constructor() {
    this.fusiones = [];
  }

  train(texto, maximoFusiones = 100, frecuenciaMinima = 2) {
    if (!Number.isSafeInteger(maximoFusiones) || maximoFusiones < 0) {
      throw new RangeError("maximoFusiones debe ser no negativo");
    }

    if (!Number.isSafeInteger(frecuenciaMinima) || frecuenciaMinima < 2) {
      throw new RangeError("frecuenciaMinima debe ser al menos 2");
    }

    let ids = [...new TextEncoder().encode(texto)];
    this.fusiones = [];

    for (let paso = 0; paso < maximoFusiones; paso += 1) {
```



```

    const candidato = parMasFrecuente(ids);

    if (!candidato || candidato.frecuencia < frecuenciaMinima) break;

    const id = 256 + this.fusiones.length;
    const [izquierda, derecha] = candidato.par;

    this.fusiones.push({ izquierda, derecha, id });
    ids = fusionar(ids, candidato.par, id);
  }

  return ids;
}

encode(texto) {
  let ids = [...new TextEncoder().encode(texto)];

  for (const regla of this.fusiones) {
    ids = fusionar(
      ids,
      [regla.izquierda, regla.derecha],
      regla.id
    );
  }

  return ids;
}

decode(ids) {
  const bytes = [];

  for (const id of ids) {
    bytes.push(...this.#expandir(id));
  }

  return new TextDecoder("utf-8", { fatal: true })
    .decode(Uint8Array.from(bytes));
}

#expandir(id) {
  if (!Number.isSafeInteger(id) || id < 0) {
    throw new RangeError(`Token inválido: ${id}`);
  }

  if (id < 256) return [id];

  const regla = this.fusiones[id - 256];

  if (!regla || regla.id !== id) {
    throw new RangeError(`Token desconocido: ${id}`);
  }

  return [

```

```
...this.#expandir(regla.izquierda),
...this.#expandir(regla.derecha)
];
}
}
```

A.12. Entrenamiento paso a paso

```
function entrenarConTraza(texto, maximoFusiones = 10) {
  let ids = [...new TextEncoder().encode(texto)];
  const reglas = [];

  for (let paso = 0; paso < maximoFusiones; paso += 1) {
    const candidato = parMasFrecuente(ids);
    if (!candidato || candidato.frecuencia < 2) break;

    const id = 256 + reglas.length;
    const antes = ids.length;

    ids = fusionar(ids, candidato.par, id);
    reglas.push({
      id,
      izquierda: candidato.par[0],
      derecha: candidato.par[1]
    });

    console.log({
      paso: paso + 1,
      par: candidato.par,
      frecuencia: candidato.frecuencia,
      antes,
      despues: ids.length,
      id
    });
  }

  return { ids, reglas };
}
```

La traza permite observar que una fusión frecuente reduce la longitud y que los conteos deben recalcularse después de cada cambio.

A.13. Por qué el orden importa

Supongamos:

```

256 = (97, 110)    → "an"
257 = (98, 256)    → "ban"
258 = (257, 256)   → "banan"

```

La regla 257 no puede aplicarse antes de crear 256. Ordenar las reglas por frecuencia, texto o ids internos cambiaría la tokenización.

encode debe recorrer exactamente en el orden aprendido. Guardar solo un diccionario de bytes finales sin las prioridades no siempre permite reconstruir esa conducta.

A.14. Decodificación recursiva

Un token base devuelve un byte. Un token compuesto expande sus dos hijos:

```

expandir(258)
→ expandir(257) + expandir(256)
→ expandir(98) + expandir(256) + expandir(97) + expandir(110)
→ bytes originales

```

El caso base es `id < 256`. El invariante "cada regla referencia tokens anteriores" impide ciclos y garantiza terminación.

Una implementación más eficiente puede precalcular la secuencia de bytes de cada regla al cargar el modelo y evitar expandir repetidamente.

A.15. La propiedad fundamental

Para cualquier string que pueda codificarse como UTF-8:

```
decode(encode(texto)) === texto
```

```

const bpe = new BPE();
bpe.train("banana banana mañana mañana 🍌🍌 ", 50);

for (const texto of [
  "",
  "banana",
  "mañana",
  "🍌 banana",
  "texto nunca visto"
]) {
  const ids = bpe.encode(texto);

```

```
const recuperado = bpe.decode(ids);

console.assert(recuperado === texto, {
  texto,
  ids,
  recuperado
});
}
```

Un texto no visto sigue siendo representable porque los 256 tokens base cubren cualquier secuencia de bytes. Tal vez use más tokens, pero no necesita un token desconocido.

A.16. Otras propiedades para comprobar

```
function validarModelo(bpe) {
  for (let indice = 0; indice < bpe.fusiones.length; indice += 1) {
    const regla = bpe.fusiones[indice];
    const idEsperado = 256 + indice;

    if (regla.id !== idEsperado) {
      throw new Error(`Id fuera de secuencia: ${regla.id}`);
    }

    if (regla.izquierda >= regla.id || regla.derecha >= regla.id) {
      throw new Error(`Referencia futura en token ${regla.id}`);
    }
  }
}
```

También:

- entrenar con texto vacío no crea reglas;
- ninguna fusión aumenta la longitud;
- una regla aprendida tiene frecuencia inicial al menos igual al mínimo;
- ids desconocidos se rechazan;
- entrenar de nuevo reinicia el modelo;
- mismo corpus, límites y desempate producen las mismas reglas.

A.17. Guardar el modelo

```
import { writeFile } from "node:fs/promises";

async function guardarModelo(ruta, bpe) {
```

```

validarModelo(bpe);

const datos = {
  version: 1,
  algoritmo: "bpe-didactico",
  base: "utf-8-bytes",
  fusiones: bpe.fusiones
};

await writeFile(
  ruta,
  `${JSON.stringify(datos, null, 2)}\n`,
  { encoding: "utf8", flag: "wx" }
);
}

```

`version`, `algoritmo` y `base` evitan interpretar silenciosamente un modelo con convenciones diferentes.

A.18. Cargar y validar

```

import { readFile } from "node:fs/promises";

async function cargarModelo(ruta) {
  const datos = JSON.parse(await readFile(ruta, "utf8"));

  if (datos.version !== 1 ||
    datos.algoritmo !== "bpe-didactico" ||
    datos.base !== "utf-8-bytes" ||
    !Array.isArray(datos.fusiones)) {
    throw new TypeError("Modelo BPE incompatible");
  }

  const bpe = new BPE();
  bpe.fusiones = datos.fusiones.map((regla, indice) => {
    const id = 256 + indice;

    if (!regla ||
      regla.id !== id ||
      !Number.isSafeInteger(regla.izquierda) ||
      !Number.isSafeInteger(regla.derecha) ||
      regla.izquierda < 0 ||
      regla.derecha < 0 ||
      regla.izquierda >= id ||
      regla.derecha >= id) {
      throw new TypeError(`Regla inválida en la posición ${indice}`);
    }
  });

  return {

```

```
        izquierda: regla.izquierda,  
        derecha: regla.derecha,  
        id  
    };  
});  
  
validarModelo(bpe);  
return bpe;  
}
```

No basta con comprobar que `fusiones` sea un array. Los archivos externos no son confiables y una referencia futura podría crear una expansión inválida.

A.19. Medir la tokenización

```
function estadisticas(bpe, texto) {  
    const bytes = new TextEncoder().encode(texto).length;  
    const tokens = bpe.encode(texto).length;  
  
    return {  
        bytes,  
        tokens,  
        bytesPorToken: tokens === 0 ? 0 : bytes / tokens  
    };  
}
```

Más bytes por token indica una secuencia más corta para ese texto, pero no es una métrica completa de calidad. También importan tamaño del vocabulario, distribución, idioma, preprocesamiento y costo del modelo.

Compará corpus:

- repetitivo;
- prosa natural;
- código fuente;
- texto con emoji;
- texto de un dominio diferente al entrenamiento.

A.20. Complejidad de la versión didáctica

En cada fusión:

1. se recorre la secuencia para contar;

2. se recorre el mapa para elegir;
3. se recorre la secuencia para fusionar.

Con muchas fusiones y un corpus grande, repetir estos recorridos es costoso. `encode` también recorre el texto una vez por regla, incluso si una regla no aparece.

Tokenizadores de producción usan índices, estructuras de prioridad, pretokenización y algoritmos especializados. La versión didáctica privilegia que cada paso sea visible y verificable.

A.21. Decisiones simplificadas

Este apéndice omite o fija:

- normalización Unicode;
- tratamiento especial de espacios;
- división inicial en palabras o fragmentos;
- tokens reservados;
- límites de vocabulario basados en frecuencia y tamaño;
- entrenamiento sobre varios documentos sin permitir fusiones entre límites;
- desempates compatibles con modelos externos;
- codificación optimizada;
- procesamiento distribuido;
- límites ante archivos o modelos hostiles.

Cada decisión cambia los tokens. Dos implementaciones llamadas BPE no son necesariamente compatibles.

A.22. Separar documentos

Concatenar corpus permite que un par cruce el final de un documento y el inicio del siguiente. Si eso no tiene significado, se necesita una frontera que no pueda fusionarse o contar pares documento por documento.

Una estrategia didáctica:

```
function contarParesEnDocumentos(documentos) {
  const total = new Map();

  for (const ids of documentos) {
    for (const [clave, cantidad] of contarPares(ids)) {
      total.set(clave, (total.get(clave) ?? 0) + cantidad);
    }
  }
}
```

```
return total;  
}
```

Después de elegir una regla, se fusiona dentro de cada documento por separado.

A.23. Tokens especiales

Un sistema puede reservar ids para comienzo, fin, relleno o separación. No deben confundirse con bytes ni generarse accidentalmente mediante fusiones.

Una organización posible:

0..255	bytes
256..N	fusiones
N+1...	especiales registrados en el modelo

Otra reserva ids especiales antes de las fusiones. Lo importante es que el formato documento rangos y que codificador y decodificador compartan la convención.

A.24. Experimentos productivos

1. Mostrá en cada paso el par, su representación en bytes y la reducción lograda.
2. Cambiá el criterio de desempate y compará modelos.
3. Entrená con `banana banana` y dibujá el árbol de cada token compuesto.
4. Compará NFC y NFD para palabras con acentos.
5. Evitá fusiones entre documentos.
6. Agregá tokens especiales sin colisionar ids.
7. Precalculá bytes por token al cargar.
8. Limitá profundidad y tamaño durante la validación de modelos externos.
9. Escribí pruebas generativas de ida y vuelta con strings aleatorios.
10. Medí dónde se consume el tiempo antes de optimizar.

A.25. Modelo mental final

```
ENTRENAR  
texto  
→ bytes  
→ contar pares
```



```
→ elegir par
→ crear token
→ fusionar
→ repetir
→ guardar reglas ordenadas
```

CODIFICAR

```
texto
→ bytes
→ aplicar reglas en orden
→ ids
```

DECODIFICAR

```
ids
→ expandir tokens hasta bytes
→ decodificar UTF-8
→ texto
```

Para recordar

- Los bytes ofrecen cobertura universal; las fusiones aprenden unidades frecuentes.
- Una regla crea un token a partir de dos tokens anteriores.
- El orden y el desempate son parte del modelo.
- La expansión recursiva termina gracias al orden de ids y referencias.
- `decode(encode(texto)) == texto` es la propiedad central, pero no la única validación necesaria.
- Comprender primero la versión simple permite reconocer qué optimizan y qué convenciones agregan los tokenizadores reales.

APÉNDICE B

Evaluar expresiones mediante pilas

Idea central

Una expresión aritmética puede evaluarse de izquierda a derecha si se guardan en pilas los valores y los operadores que todavía no pueden resolverse. La prioridad de los operadores determina cuándo se retira una operación pendiente; los paréntesis funcionan como fronteras que aíslan cada grupo.

Este apéndice conecta varios conceptos del libro:

- clases y encapsulamiento;
- arrays usados como estructuras de datos;
- recorridos de izquierda a derecha;
- condiciones y ciclos;
- funciones como valores;
- invariantes y validación de entradas.

La implementación es deliberadamente pequeña. Evalúa números positivos o negativos escritos como literales y los operadores binarios `+`, `-`, `*` y `/`. No pretende ser un parser completo de JavaScript.

B.1. El problema: respetar la estructura de la expresión

Evaluar esta expresión estrictamente de izquierda a derecha produce un resultado incorrecto:

```
3 + 4 * 2
```

La multiplicación debe resolverse antes que la suma, por lo que el resultado correcto es `11`, no `14`. Los paréntesis agregan otra regla:

```
3 + 4 * (2 - 1) → 7
```

El algoritmo necesita conservar información mientras recorre los tokens:

1. los números se convierten en valores;
2. los operadores quedan pendientes;

3. un operador nuevo puede obligar a resolver el anterior;
4. un paréntesis de cierre resuelve todo el grupo que abrió el paréntesis;
5. al llegar al final se resuelven las operaciones que todavía quedan.

La estructura adecuada para ese comportamiento es una pila: el último elemento que se guarda es el primero que se retira.

B.2. Implementar una pila

Una pila expone dos operaciones principales:

- `push`, que agrega un elemento en el tope;
- `pop`, que retira y devuelve el elemento del tope.

También conviene poder consultar el tope sin retirarlo y saber si la pila está vacía. La clase oculta cómo se representa la estructura: el resto del programa solo depende de ese contrato.

```
class Stack {
  constructor(maximo = 20) {
    this.elementos = new Array(maximo);
    this.contador = 0;
  }

  push(elemento) {
    if (this.contador === this.elementos.length) {
      throw new Error("La pila alcanzó su capacidad máxima");
    }

    this.elementos[this.contador] = elemento;
    this.contador += 1;
  }

  pop() {
    if (this.empty) {
      throw new Error("No se puede retirar un elemento de una pila vacía");
    }

    this.contador -= 1;
    const elemento = this.elementos[this.contador];
    this.elementos[this.contador] = undefined;
    return elemento;
  }

  get top() {
    if (this.empty) {
      return undefined;
    }
  }
}
```

```

    return this.elementos[this.contador - 1];
  }

  get empty() {
    return this.contador === 0;
  }
}

```

El contador representa la próxima posición disponible. Por eso también coincide con la cantidad de elementos almacenados:

```

elementos: [A, B, C, _, _]
contador:      3
               ↑ próxima posición disponible

```

En una pila no se accede a un elemento arbitrario. La única entrada válida es el tope. Esta restricción permite razonar sobre el orden de procesamiento y mantiene independiente la implementación interna.

B.3. Dos pilas para una expresión

La evaluación usa dos pilas con responsabilidades diferentes:

Pila	Contenido
valores	números y resultados parciales
operadores	operadores pendientes y paréntesis de apertura

Cuando se resuelve una operación se retiran tres elementos:

1. el operador;
2. el operando derecho **b**;
3. el operando izquierdo **a**.

Después se calcula **a operador b** y se apila el resultado. El orden es fundamental: **8 - 3** no es igual que **3 - 8**, y **8 / 2** no es igual que **2 / 8**.

La tabla de prioridad expresa una regla del dominio, no una propiedad de la pila:

```

const prioridad = {
  "+": 1,
  "-": 1,

```

```
"*": 2,  
"/": 2  
};
```

B.4. Recorrer y resolver

Antes de evaluar, la expresión se separa en tokens. Esta versión espera que los paréntesis estén separados de los números y operadores:

```
3 + 4 * ( 2 - 1 )
```

La expresión regular `/\s+/u` permite aceptar uno o varios espacios y también saltos de línea. Los operadores se conservan como strings y los números se convierten a `number`.

La función `resolver` implementa una única operación pendiente:

```
function resolver(valores, operadores, operacion) {  
  const operador = operadores.pop();  
  const b = valores.pop();  
  const a = valores.pop();  
  valores.push(operacion[operador](a, b));  
}
```

El algoritmo completo sigue estas reglas:

- número: se apila en `valores`;
- `(`: se apila en `operadores`;
- `)`: se resuelve hasta encontrar `(` y luego se descarta ese paréntesis;
- operador: se resuelven operadores pendientes de prioridad mayor o igual, siempre dentro del grupo actual, y después se apila el operador nuevo;
- fin: se resuelven todos los operadores restantes.

La condición “dentro del grupo actual” es importante. Un paréntesis de apertura no es un operador y no debe compararse con la tabla de prioridades.

```
function evaluar(expresion) {  
  const texto = expresion.trim();  
  
  if (texto === "") {  
    throw new Error("La expresión no puede estar vacía");  
  }  
  
  const valores = new Stack();
```

```
const operadores = new Stack();

const prioridad = {
  "+": 1,
  "-": 1,
  "*": 2,
  "/": 2
};

const operacion = {
  "+": (a, b) => a + b,
  "-": (a, b) => a - b,
  "*": (a, b) => a * b,
  "/": (a, b) => a / b
};

const tokens = texto.split(/\s+/u).map((token) => {
  return Number.isNaN(Number(token)) ? token : Number(token);
});

for (const token of tokens) {
  if (typeof token === "number") {
    valores.push(token);
    continue;
  }

  if (token === "(") {
    operadores.push(token);
    continue;
  }

  if (token === ")") {
    while (!operadores.empty && operadores.top !== "(") {
      resolver(valores, operadores, operacion);
    }

    if (operadores.empty) {
      throw new Error("Hay un paréntesis de cierre sin apertura");
    }

    operadores.pop();
    continue;
  }

  if (!(token in prioridad)) {
    throw new Error(`Token no reconocido: ${token}`);
  }

  while (
    !operadores.empty &&
    operadores.top !== "(" &&
    prioridad[operadores.top] >= prioridad[token]
  ) {
    resolver(valores, operadores, operacion);
  }

  operadores.push(token);
}
```

```
        resolver(valores, operadores, operacion);
    }

    operadores.push(token);
}

while (!operadores.empty) {
    if (operadores.top === "(") {
        throw new Error("Hay un paréntesis de apertura sin cierre");
    }

    resolver(valores, operadores, operacion);
}

if (valores.contador !== 1) {
    throw new Error("La expresión no tiene una estructura válida");
}

return valores.pop();
}

console.log(evaluar("3 + 4 * ( 2 - 1 )"));
// 7
```

El analizador sigue suponiendo que la entrada tiene la forma esperada: un número, un operador, otro número y así sucesivamente. Las comprobaciones finales mejoran los mensajes de error, pero no convierten este código en un parser general.

B.5. Evaluación paso a paso

Consideremos la expresión:

```
1 + 2 * ( 3 * 4 + 5 )
```

Sus tokens son:

```
[1, "+", 2, "*", "(", 3, "*", 4, "+", 5, ")"]
```

En la tabla, la base de cada pila está a la izquierda y el tope está a la derecha.

Token	Acción	valores	operadores
Inicio	Ambas pilas están vacías	[]	[]
1	Apilar el número	[1]	[]
+	Apilar el operador	[1]	[+]
2	Apilar el número	[1, 2]	[+]
*	Tiene mayor prioridad; se apila	[1, 2]	[+, *]
(	Abrir un grupo	[1, 2]	[+, *, (]
3	Apilar el número	[1, 2, 3]	[+, *, (]
*	Apilar el operador del grupo	[1, 2, 3]	[+, *, (, *]
4	Apilar el número	[1, 2, 3, 4]	[+, *, (, *]
+	Resolver $3 * 4 = 12$; apilar +	[1, 2, 12]	[+, *, (, +]
5	Apilar el número	[1, 2, 12, 5]	[+, *, (, +]
)	Resolver $12 + 5 = 17$; retirar (	[1, 2, 17]	[+, *]
Fin	Resolver $2 * 17 = 34$	[1, 34]	[+]
Fin	Resolver $1 + 34 = 35$	[35]	[]

El único valor restante es el resultado:

```
evaluar("1 + 2 * ( 3 * 4 + 5 )"); // 35
```

B.6. Qué queda fuera de esta versión

La implementación sirve para estudiar el mecanismo, pero tiene límites explícitos:

- los tokens deben estar separados por espacios;
- no distingue entre resta binaria y signo unario;
- no admite funciones como `sin(2)`;
- no incorpora el operador de potencia ni otros operadores;
- no verifica de manera detallada si faltan operandos;
- la capacidad de cada pila es fija.

Por ejemplo, `-3 + 2` no se interpreta como una expresión con signo unario, porque el primer token debería ser un número. Para soportarlo habría que agregar una etapa de tokenización y decidir, según el token anterior, si `-` es un operador binario o parte de un literal negativo.

Separar las etapas ayuda a extender el programa sin mezclar responsabilidades:

texto → tokens → validación → evaluación → resultado

La evaluación también puede reemplazarse por el algoritmo de Shunting Yard, que transforma la expresión infija en una expresión posfija. El enfoque de este apéndice evita construir esa segunda representación para concentrarse en el uso de las pilas.

B.7. Complejidad

Cada token se apila y se retira una cantidad acotada de veces. Por lo tanto, para n tokens:

- el tiempo es $O(n)$;
- el espacio adicional es $O(n)$ en el peor caso.

La profundidad de los paréntesis y la cantidad de operadores pendientes son los principales factores que determinan cuánto ocupan las pilas.

Para recordar

- Una pila implementa el comportamiento “último en entrar, primero en salir”.
- **valores** conserva números y resultados parciales; **operadores** conserva operaciones pendientes.
- La prioridad determina cuándo resolver un operador.
- Los paréntesis detienen la resolución hasta cerrar el grupo correspondiente.
- En una operación, el segundo valor retirado es el operando izquierdo.
- La tokenización, la validación y la evaluación son responsabilidades diferentes.
- La versión presentada es didáctica y debe extenderse antes de aceptar entradas generales.